

NGUYỄN CÔNG AN

KHOA HỌC MÁY TÍNH

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC
NGÀNH KHOA HỌC MÁY TÍNH

XÂY DỰNG ỨNG DỤNG GIAO HÀNG TRỰC
TUYẾN

Giảng viên hướng dẫn : **ThS. NGUYỄN VĂN TÙNG**
Sinh viên : **Nguyễn Công An**
Mã sinh viên : **2022604005**

Hà Nội, 03/2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc tới thầy Nguyễn Văn Tùng, người đã tận tình hướng dẫn, chỉ bảo và luôn đồng hành cùng em trong suốt quá trình thực hiện đồ án tốt nghiệp. Những góp ý quý báu cùng sự hỗ trợ nhiệt tình của thầy đã giúp em từng bước hoàn thiện đề tài, đồng thời nâng cao kiến thức và kỹ năng của bản thân.

Em cũng xin trân trọng cảm ơn Trường Công Nghệ Thông Tin và Truyền Thông, Trường Đại học Công Nghiệp Hà Nội cùng toàn thể quý thầy cô trong khoa đã truyền đạt những kiến thức nền tảng và chuyên môn quan trọng trong suốt thời gian học tập. Đây chính là hành trang quý giá giúp em có thể hoàn thành tốt đồ án của mình. Trong quá trình thực hiện đề tài, mặc dù đã cố gắng hết sức trong thời gian có hạn, nhưng chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự đóng góp ý kiến từ quý thầy cô để có thể hoàn thiện hơn về kiến thức cũng như kinh nghiệm trong lĩnh vực này. Em xin chân thành cảm ơn!

MỤC LỤC

DANH MỤC BẢNG BIỂU	VII
DANH MỤC HÌNH ẢNH	VIII
MỞ ĐẦU.....	10
1. Lý do chọn đề tài.....	10
2. Ý nghĩa của đề tài.....	11
2.1 Về mặt xã hội và thực tiễn	11
2.2 Về mặt công nghệ và học thuật	11
3. Mục tiêu nghiên cứu.....	12
4. Đối tượng và phạm vi nghiên cứu.....	12
4.1 Đối tượng nghiên cứu.....	12
4.2 Phạm vi nghiên cứu.....	13
5. Kết quả mong muốn đạt được của đề tài.....	13
6. Cấu trúc báo cáo	13
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT	15
1.1. Giới thiệu chung về đề tài	15
1.2 Khảo sát thị trường giao đồ ăn trực tuyến.....	17
1.2.1. Thực trạng thị trường	17
1.2.2. Đánh giá hệ thống hiện tại	18
1.2.3. Định hướng phát triển	18
1.3. Giới thiệu hệ thống phần mềm.....	19
1.3.1. Phạm vi của đề tài	19
1.3.2. Các chức năng chính của hệ thống.....	20
1.3.3. Đặc điểm kỹ thuật nổi bật	21
1.4. Phương pháp thực hiện.....	22
1.4.1. Lý do lựa chọn mô hình Agile/Scrum.....	22
1.4.2. Các giai đoạn thực hiện chính.....	23
1.4.3. Công cụ hỗ trợ phát triển	32
1.5. Đối tượng sử dụng.....	33
1.5.1. Khách hàng (Customer)	33

1.5.2. Nhà hàng (Restaurant).....	33
1.5.3. Tài xế giao hàng (Shipper).....	34
1.6. Kiến trúc và mô hình phát triển ứng dụng	34
1.6.1. Kiến trúc hệ thống.....	35
1.6.2. Mô hình phát triển ứng dụng.....	36
1.7. Công cụ và công nghệ sử dụng phát triển phần mềm.	37
1.7.1. Công cụ phát triển phần mềm.	37
1.7.1.1. Visual Studio Code (VS Code)	37
1.7.1.3. Công cụ quản lý cơ sở dữ liệu.....	40
1.7.1.4. Công cụ kiểm thử API (Postman)	43
1.7.1.5. Công cụ Quản lý mã nguồn (Git và GitHub)	46
1.7.2. Công nghệ phát triển phần mềm	48
1.7.2.1. Flutter SDK và Dart.	48
1.7.2.2. Backend (Node.js và Express.js).....	52
1.7.2.3. Lớp dữ liệu Prisma ORM và PostgreSQL (Supabase).....	55
1.7.2.4. Các dịch vụ hỗ trợ và Tích hợp bên thứ ba.....	55
CHƯƠNG II. PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU ỨNG DỤNG	62
2.1. Cơ sở lý thuyết phân tích và đặc tả yêu cầu ứng dụng.....	62
2.2. Mô tả bài toán.....	64
2.3. Xác định tác nhân hệ thống.....	65
2.3.1. Khách hàng.....	65
2.3.2. Nhà hàng	66
2.3.3. Shipper	66
2.4. Phân tích yêu cầu	67
2.4.1. Yêu cầu chức năng	67
2.4.1.1. Về phía khách hàng.....	67
2.4.1.2. Về phía nhà hàng.....	68
2.4.1.3. Về phía shipper	68
2.4.2. Yêu cầu phi chức năng	68
2.4.2.1. Giao diện người dùng.....	68
2.4.2.2. Tính bảo mật	69

2.5. Mô hình hóa Use case	70
2.5.1. Mô hình Use case tổng quát	70
2.5.2. Use case phía khách hàng	70
2.5.3. Use case phía nhà hàng	70
2.5.4. Use case phía shipper	71
2.6. Đặc tả Use case	71
2.6.1. Use case “Xác thực người dùng”	71
2.6.2. Use case “Đặt đơn hàng”	73
2.6.3. Use case “Xác nhận đơn hàng”	75
2.6.4. Use case “Giao hàng”	77
2.6.5. Use case “Quản lý tài khoản”	79
2.6.6. Use case “Tìm kiếm và xem thực đơn”	81
2.6.7. Use case “Quản lý món yêu thích”	83
2.6.8. Use case “Quản lý giỏ hàng”	84
2.6.9. Use case “Theo dõi đơn hàng”	85
2.6.10. Use case “Xác thực tài khoản nhà hàng”	87
2.6.10. Use case “Quản lý món ăn”	88
2.6.11. Use case “Xác nhận món đã nấu xong”	90
2.6.12. Use case “Xem thống kê doanh thu”	91
2.6.13. Use case “Xác thực tài khoản shipper”	93
2.6.14. Use case “Bật chế độ bắt đơn”	94
2.6.15. Use case “Giao thành công”	96
2.6.16. Use case “Xem thống kê thu nhập”	97
2.7. Thiết kế biểu đồ lớp (Class Diagram)	99
2.7.1. Biểu đồ lớp tổng quan	99
2.7.2. Các lớp chính của hệ thống	100
2.8. Biểu đồ trình tự (Sequence Diagram)	110
2.8.1. Biểu đồ trình tự use case “Xác thực người dùng”	111
2.8.2. Biểu đồ trình tự use case “Đặt đơn hàng”	112
2.8.3. Biểu đồ trình tự use case “Xác nhận đơn hàng”	113
2.8.4. Biểu đồ trình tự use case “Giao hàng”	114

2.8.5. Biểu đồ trình tự use case “Quản lý tài khoản và Địa chỉ”	115
2.8.6. Biểu đồ trình tự use case “Tìm kiếm và xem thực đơn”	116
2.8.7. Biểu đồ trình tự use case “Quản lý món yêu thích”	117
2.8.8. Biểu đồ trình tự use case “Quản lý giỏ hàng”	118
2.8.9. Biểu đồ trình tự use case “Theo dõi đơn hàng”	120
2.8.10. Biểu đồ trình tự use case “Xác thực tài khoản nhà hàng”	121
2.8.11. Biểu đồ trình tự use case “Quản lý món ăn”	122
2.8.12. Biểu đồ trình tự use case ”Xác nhận món đã nấu xong”	124
2.8.13. Biểu đồ trình tự use case “Xem thống kê doanh thu”	126
2.8.14. Biểu đồ trình tự use case “Xác thực tài khoản shipper”	127
2.8.15. Biểu đồ trình tự use case “Bật chế độ bắt đơn”	128
2.8.16. Biểu đồ trình tự use case “Giao thành công và xem thống kê thu nhập”	130
2.8.17. Biểu đồ trình tự luồng trạng thái đơn hàng	132
2.8.2. Biểu đồ trạng thái (State Diagram)	134
CHƯƠNG III. THIẾT KẾ ỨNG DỤNG.....	136
3.1. Thiết kế Back-end	136
3.2. Thiết kế mô hình dữ liệu	138
3.2.1. Biểu đồ ERD mức logic	139
3.2.2. Biểu đồ ERD mức vật lý	140
3.3. Thiết kế giao diện hệ thống.....	141
3.3.1. Giao diện phía khách hàng.....	142
3.3.1.1. Màn hình đăng nhập và đăng kí.....	143
3.3.1.2. Màn hình trang chủ	144
3.3.1.3. Màn hình chi tiết nhà hàng và thực đơn.....	145
3.3.1.4. Màn hình giỏ hàng và thanh toán.....	146
3.3.1.5. Màn hình theo dõi đơn hàng	147
3.3.1.6. Màn hình hồ sơ.....	148
3.3.2. Giao diện phía nhà hàng.....	148
3.3.2.1. Màn hình quản lý đơn hàng	149
3.3.2.2. Màn hình quản lý thực đơn.....	150

3.3.2.3. Màn hình thống kê thu nhập	151
3.3.2.4. Màn hình hồ sơ cửa hàng	152
3.3.3. Giao diện phía shipper	152
3.3.3.1. Màn hình danh sách đơn hàng đang chờ.....	153
3.3.3.2. Màn hình chi tiết đơn hàng	154
3.3.3.3. Màn hình thống kê thu nhập	155
CHƯƠNG IV. CÀI ĐẶT VÀ KIỂM THỬ.....	156
4.1. Cài đặt	156
4.1.1. Cài đặt Back-end	156
4.1.1.1. Cài đặt môi trường và công nghệ sử dụng	156
4.1.1.2. Mô hình kiến trúc.....	157
4.1.1.3. Kết quả đạt được	157
4.1.2. Cài đặt Front-end.....	157
4.1.2.1. Cấu hình môi trường	157
4.1.2.2. Tổ chức giao diện và cấu trúc hệ thống	158
4.1.2.3. Kết quả đạt được của hệ thống Front-end.....	159
4.2. Kiểm thử.....	159
4.2.1. Kế hoạch kiểm thử	159
4.2.2. Kịch bản kiểm thử.....	162
4.2.3. Kết quả kiểm thử.....	163
4.3. Hạn chế và hướng cải thiện.....	164
4.3.1. Các hạn chế hiện tại của hệ thống.....	164
4.3.2. Định hướng phát triển và mở rộng.....	165
4.3.2.1. Số hóa luồng vận hành dòng tiền.....	165
4.3.2.2. Nâng cấp thuật toán vận tải và tối ưu giao hàng.....	166
4.3.2.3. Tích hợp hệ thống giao tiếp nội bộ	167
4.3.2.4. Ứng dụng phân tích dữ liệu và trí tuệ nhân tạo.....	167
4.3.2.5. Nâng cao khả năng chịu tải và mở rộng hệ thống.....	167
KẾT LUẬN	169
TÀI LIỆU THAM KHẢO	171

DANH MỤC BẢNG BIỂU

Bảng 1. Bảng product backlog	24
Bảng 2. Bảng chia sprint	28
Bảng 3. Bảng tiêu chí so sánh các công nghệ.....	49
Bảng 4. Bảng tóm tắt các dịch vụ tích hợp.....	60
Bảng 5. Mô tả chi tiết use case đăng ký tài khoản người dùng	71
Bảng 6. Mô tả chi tiết use case đặt đơn hàng	73
Bảng 7. Mô tả chi tiết use case xác nhận đơn hàng.....	75
Bảng 8. Mô tả chi tiết use case giao hàng	77
Bảng 9. Mô tả chi tiết use case quản lý tài khoản	79
Bảng 10. Bảng mô tả chi tiết use case tìm kiếm và xem thực đơn.....	81
Bảng 11. Bảng mô tả chi tiết use case quản lý món yêu thích	83
Bảng 12. Bảng mô tả chi tiết use case quản lý giỏ hàng	84
Bảng 13. Bảng mô tả chi tiết use case theo dõi đơn hàng	85
Bảng 14. Bảng mô tả chi tiết use case xác thực tài khoản nhà hàng	87
Bảng 15. Bảng mô tả chi tiết use case quản lý món ăn	88
Bảng 16. Bảng mô tả chi tiết use case xác nhận món đã nấu xong	90
Bảng 17. Bảng mô tả chi tiết use case xem thống kê doanh thu	91
Bảng 18. Bảng mô tả chi tiết use case xác thực tài khoản shipper.....	93
Bảng 19. Bảng mô tả chi tiết use case bật chế độ bắt đơn.....	94
Bảng 20. Bảng mô tả chi tiết use case giao thành công	96
Bảng 21. Bảng mô tả chi tiết use case Xem thống kê thu nhập	97
Bảng 22. Lớp user	100
Bảng 23. Lớp address	101
Bảng 24. Lớp restaurant	102
Bảng 25. Lớp shipper	103
Bảng 26. Lớp food.....	104
Bảng 27. Lớp category	104
Bảng 28. Lớp food option group	105

Bảng 29. Lớp food option.....	105
Bảng 30. Lớp order.....	106
Bảng 31. Lớp order item.....	107
Bảng 32. Lớp cart item	108
Bảng 33. Lớp favorite.....	109
Bảng 34. Danh mục kịch bản kiểm thử	162

DANH MỤC HÌNH ẢNH

Hình 1. Sơ đồ kiến trúc hệ thống.....	36
Hình 2. Giao diện minh họa của Visual Studio Code.....	40
Hình 3. Hình ảnh minh họa Dashboard Supabase	41
Hình 4. Hình ảnh minh họa Dashboard Prisma Studio	43
Hình 5. Công cụ quản lý mã nguồn Git và GitHub	46
Hình 6. Các nền tảng phổ biến với Flutter	48
Hình 7. Kiến trúc Backend sử dụng Node.js và Express.js	52
Hình 8. Prisma ORM và cơ sở dữ liệu PostgreSQL.....	55
Hình 9. Tích hợp OpenStreetMap và định vị GPS trong ứng dụng Flutter....	57
Hình 10. Kiến trúc của Supabase Realtime	59
Hình 11. Luồng vận hành của hệ thống.....	60
Hình 12. Mô hình use case tổng quát	70
Hình 13. Mô hình use case phía khách hàng	70
Hình 14. Mô hình use case phía nhà hàng.....	70
Hình 15. Mô hình use case phía shipper.....	71
Hình 16. Biểu đồ lớp tổng quan	99
Hình 17. Biểu đồ trình tự xác thực người dùng.....	111
Hình 18. Biểu đồ trình tự đặt đơn hàng.....	112
Hình 19. Biểu đồ trình tự xác nhận đơn hàng.....	113
Hình 20. Biểu đồ trình tự giao hàng	114
Hình 21. Biểu đồ trình tự quản lý tài khoản và địa chỉ	115
Hình 22. Biểu đồ trình tự tìm kiếm và xem thực đơn.....	116

Hình 23. Biểu đồ trình tự quản lý món yêu thích.....	117
Hình 24. Biểu đồ trình tự quản lý giỏ hàng.....	118
Hình 25. Biểu đồ trình tự theo dõi đơn hàng.....	120
Hình 26. Biểu đồ trình tự xác thực tài khoản nhà hàng.....	121
Hình 27. Biểu đồ trình tự quản lý món ăn.....	122
Hình 28. Biểu đồ trình tự xác nhận món đã nấu xong.....	124
Hình 29. Biểu đồ trình tự xem thống kê doanh thu.....	126
Hình 30. Biểu đồ trình tự xác thực tài khoản shipper.....	127
Hình 31. Biểu đồ trình tự bắt chế độ bắt đơn	128
Hình 32. Biểu đồ trình tự giao hàng thành công và xem thống kê doanh thu	130
Hình 33. Biểu đồ trình tự luồng trạng thái đơn hàng.....	132
Hình 34. Biểu đồ trạng thái đơn hàng	134
Hình 35. Cấu trúc thư mục backend.....	136
Hình 36. Biểu đồ ERD mức logic.....	139
Hình 37. Biểu đồ ERD mức vật lí	140
Hình 38. Màn hình đăng nhập và đăng kí	143
Hình 39. Màn hình trang chủ.....	144
Hình 40. Màn hình chi tiết nhà hàng và thực đơn	145
Hình 41. Màn hình giỏ hàng và thanh toán	146
Hình 42. Màn hình theo dõi đơn hàng.....	147
Hình 43. Màn hình hồ sơ.....	148
Hình 44. Màn hình quản lý đơn hàng.....	149
Hình 45. Màn hình quản lý thực đơn.....	150
Hình 46. Màn hình thống kê thu nhập.....	151
Hình 47. Màn hình hồ sơ cửa hàng.....	152
Hình 48. Màn hình danh sách đơn hàng đang chờ	153
Hình 49. Màn hình chi tiết đơn hàng.....	154
Hình 50. Màn hình thống kê thu nhập.....	155

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và nền kinh tế số, các dịch vụ trực tuyến ngày càng trở nên phổ biến và đóng vai trò quan trọng trong đời sống của con người. Đặc biệt, dịch vụ giao đồ ăn trực tuyến đã trở thành một xu hướng tiêu dùng hiện đại, giúp người dùng có thể dễ dàng đặt món và nhận hàng tận nơi một cách nhanh chóng và tiện lợi. Các nền tảng như GrabFood, ShopeeFood hay Gojek đã góp phần thay đổi thói quen tiêu dùng của người dân, đồng thời mở ra nhiều cơ hội phát triển cho các cửa hàng kinh doanh ẩm thực.

Tuy nhiên, việc vận hành một hệ thống giao đồ ăn trực tuyến không hề đơn giản. Hệ thống cần phải quản lý và phối hợp hoạt động giữa nhiều đối tượng khác nhau như khách hàng, cửa hàng và người giao hàng. Nếu việc quản lý đơn hàng và trạng thái giao nhận vẫn thực hiện thông qua các phương thức truyền thống như gọi điện hoặc nhắn tin thủ công thì sẽ dễ dẫn đến tình trạng sai sót thông tin, chậm trễ trong quá trình xử lý đơn hàng và khó khăn trong việc theo dõi tiến trình giao nhận.

Bên cạnh đó, yếu tố thời gian thực ngày càng trở nên quan trọng trong trải nghiệm của người dùng. Khách hàng không chỉ muốn đặt món mà còn mong muốn theo dõi trạng thái đơn hàng, biết được đơn hàng đang được chuẩn bị hay đang được giao đến đâu. Việc cung cấp thông tin cập nhật liên tục về vị trí của shipper hoặc trạng thái đơn hàng trên bản đồ sẽ giúp tăng tính minh bạch và nâng cao sự hài lòng của người sử dụng dịch vụ.

Xuất phát từ những nhu cầu thực tế đó, việc xây dựng một hệ thống ứng dụng giao đồ ăn trực tuyến có khả năng xử lý và đồng bộ dữ liệu theo thời gian thực là một giải pháp cần thiết. Vì vậy, em quyết định thực hiện đề tài “Xây dựng hệ thống ứng dụng giao đồ ăn trực tuyến thời gian thực”. Thông qua đề tài này, em có cơ hội nghiên cứu và áp dụng các công nghệ phát triển phần mềm hiện đại như Flutter, Node.js, PostgreSQL, Prisma ORM và Socket.io,

nhằm xây dựng một hệ thống có khả năng hoạt động ổn định, xử lý nhiều người dùng đồng thời và cung cấp trải nghiệm tốt cho người sử dụng.

2. Ý nghĩa của đề tài

2.1 Về mặt xã hội và thực tiễn

Hệ thống ứng dụng giao đồ ăn trực tuyến giúp tối ưu hóa quá trình kết nối và trao đổi thông tin giữa các bên tham gia trong hệ thống. Khách hàng có thể dễ dàng tìm kiếm cửa hàng, lựa chọn món ăn, đặt hàng và theo dõi trạng thái đơn hàng một cách nhanh chóng và minh bạch. Đối với các cửa hàng kinh doanh, hệ thống hỗ trợ quản lý danh mục sản phẩm, tiếp nhận và xử lý đơn hàng hiệu quả hơn. Trong khi đó, người giao hàng có thể nhận đơn, xác định vị trí giao hàng và cập nhật trạng thái đơn hàng một cách thuận tiện.

Nhờ đó, hệ thống góp phần nâng cao hiệu quả vận hành, giảm thiểu sai sót trong quá trình xử lý đơn hàng và mang lại trải nghiệm dịch vụ tiện lợi, nhanh chóng cho người tiêu dùng.

2.2 Về mặt công nghệ và học thuật

Đề tài là cơ hội để nghiên cứu và áp dụng các công nghệ phát triển phần mềm hiện đại trong việc xây dựng hệ thống ứng dụng đa nền tảng. Ứng dụng di động được phát triển bằng Flutter Framework, cho phép xây dựng giao diện thân thiện và hoạt động trên nhiều nền tảng khác nhau. Phía máy chủ sử dụng Node.js kết hợp với Express.js để xây dựng hệ thống API và xử lý các yêu cầu từ phía client.

Bên cạnh đó, việc sử dụng Prisma ORM giúp đơn giản hóa quá trình tương tác với cơ sở dữ liệu PostgreSQL, đồng thời đảm bảo tính nhất quán của dữ liệu. Ngoài ra, công nghệ Socket.io được sử dụng để triển khai cơ chế truyền dữ liệu theo thời gian thực, cho phép đồng bộ trạng thái đơn hàng giữa các ứng dụng của khách hàng, cửa hàng và người giao hàng. Thông qua việc áp dụng các công nghệ và kiến trúc phần mềm hiện đại, hệ thống có khả năng mở rộng

tốt, dễ dàng bảo trì và đáp ứng được các yêu cầu về hiệu năng cũng như khả năng xử lý nhiều kết nối đồng thời.

3. Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là phân tích, thiết kế và xây dựng một hệ thống ứng dụng giao đồ ăn trực tuyến có khả năng hoạt động theo thời gian thực, đáp ứng nhu cầu của các đối tượng tham gia trong hệ thống.

Cụ thể, đề tài hướng tới các mục tiêu sau:

- Phân tích và xây dựng hệ thống giao đồ ăn trực tuyến với ba nhóm người dùng chính gồm khách hàng, cửa hàng và người giao hàng.
- Thiết kế và phát triển backend server sử dụng Node.js và Express.js, cung cấp các API phục vụ cho việc quản lý người dùng, sản phẩm và đơn hàng.
- Xây dựng ứng dụng di động bằng Flutter cho các vai trò trong hệ thống, đảm bảo giao diện thân thiện và dễ sử dụng.
- Triển khai cơ chế đồng bộ dữ liệu thời gian thực bằng Socket.io để cập nhật trạng thái đơn hàng và vị trí giao hàng liên tục.
- Thiết kế hệ thống cơ sở dữ liệu sử dụng PostgreSQL và Prisma ORM, đảm bảo khả năng lưu trữ và truy xuất dữ liệu hiệu quả.

4. Đối tượng và phạm vi nghiên cứu

4.1 Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu và xây dựng hệ thống giao đồ ăn trực tuyến với sự tham gia của ba đối tượng chính:

- Khách hàng : Tìm kiếm cửa hàng, xem danh sách món ăn, đặt hàng, thanh toán và theo dõi trạng thái đơn hàng.
- Cửa hàng : Quản lý thông tin cửa hàng, danh mục sản phẩm, tiếp nhận và xử lý đơn hàng từ khách hàng.
- Người giao hàng : Nhận đơn giao hàng, xác định vị trí giao nhận và cập nhật trạng thái giao hàng theo thời gian thực.

4.2 Phạm vi nghiên cứu

Đề tài tập trung vào việc xây dựng hệ thống ứng dụng giao đồ ăn trực tuyến với các chức năng cơ bản như quản lý tài khoản người dùng, quản lý sản phẩm, đặt hàng và theo dõi trạng thái giao hàng. Hệ thống được thiết kế để hỗ trợ hoạt động giữa khách hàng, cửa hàng và người giao hàng trong phạm vi một nền tảng giao đồ ăn trực tuyến.

Các chức năng nâng cao như thanh toán điện tử tích hợp với cổng thanh toán bên thứ ba hoặc tối ưu hóa thuật toán điều phối giao hàng sẽ không được triển khai sâu trong phạm vi của đề tài này.

5. Kết quả mong muốn đạt được của đề tài

Sau khi hoàn thành đề tài, hệ thống dự kiến đạt được các kết quả sau:

- Xây dựng thành công một ứng dụng giao đồ ăn trực tuyến hoạt động ổn định, có khả năng phục vụ nhiều người dùng đồng thời.
- Hệ thống có khả năng cập nhật trạng thái đơn hàng theo thời gian thực, giúp người dùng theo dõi quá trình giao hàng một cách trực quan.
- Các thành phần của hệ thống được thiết kế theo kiến trúc rõ ràng, mã nguồn được tổ chức khoa học, dễ dàng bảo trì và phát triển mở rộng trong tương lai.
- Hoàn thiện tài liệu báo cáo chi tiết, bao gồm các phần phân tích yêu cầu, thiết kế hệ thống, xây dựng ứng dụng và kiểm thử đánh giá kết quả.

6. Cấu trúc báo cáo

Báo cáo đồ án tốt nghiệp được tổ chức thành các chương chính như sau:

- Chương I : Tổng quan về đề tài và cơ sở lý thuyết: Giới thiệu bối cảnh và lý do chọn đề tài, khảo sát thực trạng thị trường giao đồ ăn trực tuyến tại Việt Nam, trình bày phương pháp thực hiện theo mô hình Agile/Scrum, và giới thiệu các công nghệ được sử dụng trong hệ thống gồm Flutter, Node.js, Supabase và Firebase Authentication.
- Chương II: Phân tích và đặc tả yêu cầu ứng dụng: Phân tích bài toán, xác định các tác nhân trong hệ thống và đặc tả yêu cầu chức năng, phi chức

năng. Chương này bao gồm các biểu đồ Use Case, biểu đồ lớp, biểu đồ trình tự và biểu đồ trạng thái cho toàn bộ luồng nghiệp vụ của hệ thống.

- Chương III Thiết kế ứng dụng: Trình bày thiết kế cơ sở dữ liệu (ERD mức logic và vật lý), thiết kế kiến trúc backend, và thiết kế giao diện người dùng cho ba phân hệ: khách hàng, nhà hàng và tài xế.

- Chương IV — Cài đặt và kiểm thử: Mô tả quá trình triển khai hệ thống ở cả phía backend và frontend, trình bày kết quả đạt được, xây dựng kế hoạch và kịch bản kiểm thử, đánh giá kết quả kiểm thử, và nêu các hạn chế cùng hướng phát triển tiếp theo.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT

1.1. Giới thiệu chung về đề tài

Trong những năm gần đây, sự phát triển vượt bậc của công nghệ thông tin, Internet và đặc biệt là điện thoại thông minh đã thúc đẩy mạnh mẽ sự chuyển đổi số trong mọi lĩnh vực kinh tế - xã hội tại Việt Nam. Thương mại điện tử nổi lên như một trong những lĩnh vực tăng trưởng nhanh nhất, với tốc độ tăng trưởng kép hàng năm luôn ở mức cao. Theo các báo cáo ngành, doanh thu thương mại điện tử tại Việt Nam liên tục đạt mức tăng trưởng trên 20-25% mỗi năm, trong đó lĩnh vực thực phẩm và đồ uống trực tuyến là một trong những phân khúc tăng trưởng ấn tượng nhất.

Đặc biệt, dịch vụ giao đồ ăn trực tuyến đã trở thành một phần không thể thiếu trong nhịp sống hiện đại tại các đô thị lớn như Hà Nội, TP. Hồ Chí Minh, Đà Nẵng và các thành phố trực thuộc trung ương. Với lối sống bận rộn, quỹ thời gian hạn chế và nhu cầu tiện lợi ngày càng cao, người tiêu dùng ngày càng ưu tiên sử dụng các ứng dụng đặt món trực tuyến thay vì phải di chuyển đến cửa hàng hoặc quán ăn truyền thống. Sự bùng nổ của đại dịch COVID-19 trong giai đoạn 2020–2022 càng đẩy nhanh xu hướng này, khi mọi người hạn chế tiếp xúc trực tiếp và chuyển sang các giải pháp không tiếp xúc.

Bên cạnh đó, sự phổ biến rộng rãi của điện thoại thông minh, kết nối 4G/5G ổn định cùng với sự phát triển mạnh mẽ của các nền tảng thanh toán số (Mobile Banking, ví điện tử như Momo, ZaloPay, VNPay...) đã tạo điều kiện thuận lợi để các ứng dụng giao đồ ăn phát triển mạnh mẽ. Theo thống kê, tỷ lệ người dùng internet tại Việt Nam đã vượt trên 77% dân số (khoảng 78 triệu người), trong đó hơn 70% thường xuyên sử dụng các dịch vụ thương mại điện tử.

Đối với người tiêu dùng, ứng dụng giao đồ ăn mang lại nhiều lợi ích rõ rệt: tiết kiệm thời gian, đa dạng lựa chọn món ăn, theo dõi đơn hàng thời gian thực, đánh giá và phản hồi nhanh chóng, cùng nhiều chương trình khuyến mãi

hấp dẫn. Còn đối với các nhà hàng và quán ăn, đặc biệt là các cơ sở vừa và nhỏ, việc tham gia nền tảng giao đồ ăn giúp mở rộng đáng kể phạm vi tiếp cận khách hàng mà không cần đầu tư quá lớn vào mặt bằng hay nhân sự phục vụ tại chỗ. Đồng thời, các tài xế giao hàng cũng có thêm cơ hội kiếm thu nhập linh hoạt thông qua mô hình kinh tế chia sẻ.

Tuy nhiên, bên cạnh những lợi ích đó, thị trường giao đồ ăn trực tuyến tại Việt Nam vẫn tồn tại một số hạn chế như: phí dịch vụ trung gian cao (thường từ 20–30% giá trị đơn hàng), thời gian giao hàng chưa thực sự tối ưu, chất lượng dịch vụ chưa đồng đều, và sự phụ thuộc lớn vào một vài nền tảng lớn (ShopeeFood, GrabFood, Baemin...). Điều này mở ra nhu cầu về các giải pháp mới có thể giảm chi phí trung gian, tăng tốc độ giao hàng, tối ưu trải nghiệm người dùng và xây dựng mối quan hệ trực tiếp hơn giữa nhà hàng – khách hàng – tài xế.

Xuất phát từ thực tiễn trên, đề tài “Phát triển ứng dụng giao đồ ăn trực tuyến AppFoodDelivery” được lựa chọn. Đề tài nhằm xây dựng một hệ sinh thái số hoàn chỉnh, kết nối ba đối tượng chính: khách hàng, nhà hàng/quán ăn và tài xế giao hàng. Hệ thống không chỉ tập trung vào việc tối ưu hóa quy trình đặt món, thanh toán và giao nhận mà còn hướng đến việc giảm thiểu chi phí trung gian, nâng cao hiệu quả vận hành và mang lại trải nghiệm tốt nhất cho người dùng.

Ứng dụng được định hướng phát triển theo mô hình đa nền tảng hỗ trợ cả hai hệ điều hành di động phổ biến là IOS và Android. Hệ thống sẽ đảm bảo hoạt động ổn định, có khả năng xử lý đồng thời nhiều người dùng, hỗ trợ cập nhật dữ liệu theo thời gian thực tích hợp bản đồ định vị, hệ thống quản lý đơn hàng chặt chẽ, thông báo và các tính năng bảo mật cao.

Việc thực hiện đề tài này không chỉ góp phần áp dụng các công nghệ hiện đại vào giải quyết bài toán thực tiễn mà còn mang ý nghĩa kinh tế - xã hội rõ nét trong bối cảnh chuyển đổi số quốc gia đang được đẩy mạnh tại Việt Nam.

1.2 Khảo sát thị trường giao đồ ăn trực tuyến

1.2.1. Thực trạng thị trường

Thị trường giao đồ ăn trực tuyến tại Việt Nam đang phát triển mạnh mẽ nhờ sự gia tăng nhu cầu tiện lợi của người tiêu dùng tại các đô thị lớn và xu hướng chuyển đổi số trong lĩnh vực dịch vụ ăn uống. Theo báo cáo của Momentum Works năm 2026, tổng giá trị giao dịch của thị trường giao đồ ăn trực tuyến năm 2025 đạt khoảng 2,1 tỷ USD, tăng 19% so với năm 2024. Dự báo đến năm 2030, quy mô thị trường kết hợp giao đồ ăn và gọi xe có thể đạt khoảng 9 tỷ USD^[11].

Các nền tảng lớn như GrabFood, ShopeeFood và beFood đã tạo dựng được hệ sinh thái khá hoàn chỉnh, kết nối chặt chẽ giữa người dùng, nhà hàng và đơn vị vận chuyển. Hai ông lớn GrabFood và ShopeeFood hiện thống trị thị trường với mỗi nền tảng chiếm khoảng 48% thị phần, tổng cộng kiểm soát gần 96% thị trường, trong khi beFood chỉ chiếm khoảng 4%. Các nền tảng này cung cấp đa dạng dịch vụ từ đặt món, giao hàng nhanh đến chương trình khuyến mãi hấp dẫn^{[12], [13], [14]}.

Một trong những yếu tố quan trọng thúc đẩy sự phát triển của thị trường là đa dạng phương thức thanh toán, đặc biệt là thanh toán không dùng tiền mặt. Người dùng ngày càng có xu hướng sử dụng ví điện tử và cổng thanh toán trực tuyến như MoMo, ZaloPay và VNPAY nhờ tính tiện lợi, nhanh chóng và an toàn. Sự phổ biến của smartphone cùng hạ tầng thanh toán số đã tạo điều kiện thuận lợi để các ứng dụng giao đồ ăn phát triển, góp phần nâng cao trải nghiệm người dùng và mở rộng cơ hội cho nhà hàng.

Tuy nhiên, thị trường vẫn tồn tại một số hạn chế chung như phí dịch vụ trung gian cao (thường 20–30%), trải nghiệm người dùng chưa đồng đều ở một số khâu, và khả năng cá nhân hóa còn hạn chế.

1.2.2. Đánh giá hệ thống hiện tại

Hiện tại, hệ thống ứng dụng giao đồ ăn trực tuyến đang được xây dựng mới chủ yếu hỗ trợ các chức năng cơ bản, bao gồm:

- Xem danh sách món ăn từ các nhà hàng đối tác.
- Thêm món vào giỏ hàng và chỉnh sửa đơn hàng.
- Tạo và xác nhận đơn hàng.
- Theo dõi trạng thái đơn hàng cơ bản.

Tuy nhiên, hệ thống chưa tích hợp các phương thức thanh toán trực tuyến thông qua bên thứ ba, mà chủ yếu dựa vào hình thức thanh toán khi nhận hàng (COD - Cash on Delivery). Việc thiếu tích hợp thanh toán số này dẫn đến một số hạn chế đáng kể:

- Trải nghiệm người dùng chưa tối ưu so với các nền tảng lớn như GrabFood hay ShopeeFood, khiến quy trình đặt hàng chưa thực sự tiện lợi và nhanh chóng.
- Không tận dụng được xu hướng thanh toán không tiền mặt đang phát triển mạnh mẽ tại Việt Nam, dẫn đến bỏ lỡ cơ hội tăng sự hài lòng và lòng trung thành của khách hàng.

Những hạn chế này làm giảm khả năng cạnh tranh của ứng dụng và chưa đáp ứng đầy đủ mục tiêu tối ưu hóa quy trình đặt món, giảm chi phí trung gian và nâng cao trải nghiệm người dùng như định hướng của đề tài “Phát triển ứng dụng giao đồ ăn trực tuyến AppFoodDelivery”.

1.2.3. Định hướng phát triển

Để khắc phục những hạn chế trên và phù hợp với xu hướng công nghệ thông tin, Internet cùng thanh toán số đã thúc đẩy sự bùng nổ thương mại điện tử tại Việt Nam, hệ thống sẽ được nâng cấp mạnh mẽ trong tương lai. Cụ thể, ưu tiên tích hợp các cổng thanh toán bên thứ ba như MoMo, ZaloPay hoặc VNPAY thông qua backend server.

Việc tích hợp này nhằm đạt được các mục tiêu sau:

- Tăng tính tiện lợi và tốc độ thanh toán cho người dùng, hỗ trợ thanh toán một chạm hoặc quét QR code.
- Rút ngắn thời gian xử lý đơn hàng, giảm rủi ro liên quan đến tiền mặt (mất mát, tranh chấp).
- Nâng cao khả năng cạnh tranh của AppFoodDelivery so với các nền tảng hiện có trên thị trường.
- Hỗ trợ xây dựng hệ sinh thái hoàn chỉnh, kết nối trực tiếp giữa khách hàng, nhà hàng và tài xế, đồng thời dễ dàng mở rộng các tính năng tài chính trong tương lai.

1.3. Giới thiệu hệ thống phần mềm

Xuất phát từ thực tiễn phát triển mạnh mẽ của thị trường giao đồ ăn trực tuyến tại Việt Nam cùng những hạn chế của các nền tảng hiện tại đã được khảo sát, đề tài “Phát triển ứng dụng giao đồ ăn trực tuyến AppFoodDelivery” được thực hiện nhằm xây dựng một hệ sinh thái số hoàn chỉnh, kết nối trực tiếp ba đối tượng chính: khách hàng, nhà hàng và tài xế giao hàng. Hệ thống tập trung vào việc tối ưu hóa quy trình đặt món, thanh toán (hiện tại hỗ trợ COD và định hướng tích hợp thanh toán trực tuyến trong tương lai) và giao nhận, từ đó giảm thiểu chi phí trung gian, nâng cao trải nghiệm người dùng và tăng hiệu quả vận hành cho các bên tham gia.

Hệ thống hướng đến việc phát triển một ứng dụng di động đa nền tảng, hoạt động ổn định trên cả hai hệ điều hành phổ biến là iOS và Android, đồng thời hỗ trợ cập nhật dữ liệu theo thời gian thực và đảm bảo tính bảo mật cao.

1.3.1. Phạm vi của đề tài

Phạm vi hệ thống tập trung vào việc xây dựng ba ứng dụng di động riêng biệt dành cho ba nhóm đối tượng người dùng chính:

- Ứng dụng dành cho Khách hàng (Customer App): Cho phép tìm kiếm nhà hàng, đặt món, thanh toán và theo dõi đơn hàng.
- Ứng dụng dành cho Nhà hàng (Restaurant App): Hỗ trợ quản lý menu, xử lý đơn hàng và tương tác với khách hàng.
- Ứng dụng dành cho Tài xế (Driver App): Hỗ trợ nhận đơn hàng, cập nhật vị trí và hoàn thành giao hàng.

Ứng dụng được thiết kế theo mô hình cross-platform, nhằm đảm bảo giao diện và trải nghiệm người dùng nhất quán giữa các nền tảng. Phạm vi hiện tại tập trung vào các chức năng cốt lõi của hệ thống, chưa bao gồm module quản trị backend dành cho quản trị viên hệ thống (có thể triển khai ở các giai đoạn sau).

1.3.2. Các chức năng chính của hệ thống

Hệ thống AppFoodDelivery cung cấp các chức năng chính sau:

- Quản lý món ăn và danh mục:
- Quản lý danh mục món ăn và thông tin chi tiết món ăn (tên, giá, hình ảnh, mô tả).
- Hỗ trợ các tùy chọn bổ sung cho món ăn (topping, kích cỡ/size, mức độ cay, cách chế biến...).
- Cho phép nhà hàng bật/tắt tình trạng món ăn (còn hàng hoặc hết hàng) một cách linh hoạt.

Giỏ hàng và quy trình đặt hàng linh hoạt:

- Thêm, chỉnh sửa và xóa món trong giỏ hàng.
- Hỗ trợ đặt hàng ngay hoặc đặt trước theo khung giờ.
- Tính toán tự động tổng tiền, phí giao hàng dựa trên khoảng cách.

Hệ thống thông báo và cập nhật trạng thái đơn hàng thời gian thực:

- Cập nhật trạng thái đơn hàng theo các giai đoạn (đang chuẩn bị, đang giao, đã giao thành công...).
- Gửi thông báo đẩy (push notification) cho tất cả các bên liên quan.
- Theo dõi vị trí tài xế trên bản đồ thời gian thực.

Quản lý địa chỉ và vị trí dựa trên bản đồ:

- Lưu trữ và quản lý nhiều địa chỉ giao hàng của khách hàng.
- Tích hợp bản đồ để xác định vị trí chính xác và tính toán khoảng cách.
- Hỗ trợ tìm kiếm nhà hàng theo vị trí hiện tại của người dùng.

Ngoài ra, hệ thống còn hỗ trợ các chức năng bổ trợ quan trọng như đăng ký/đăng nhập, quản lý tài khoản, đánh giá phản hồi và lịch sử đơn hàng.

1.3.3. Đặc điểm kỹ thuật nổi bật

Hệ thống được thiết kế theo kiến trúc client-server với trọng tâm là tính linh hoạt, khả năng mở rộng và hiệu suất cao. Các đặc điểm kỹ thuật chính bao gồm:

- Sử dụng Supabase làm nền tảng Backend-as-a-Service (BaaS), cung cấp cơ sở dữ liệu PostgreSQL, cơ chế xác thực người dùng và dịch vụ Realtime mạnh mẽ, giúp đồng bộ trạng thái đơn hàng nhanh chóng giữa khách hàng, nhà hàng và tài xế mà không cần gọi API liên tục.
- Áp dụng Realtime subscription để cập nhật trạng thái đơn hàng và vị trí tài xế theo thời gian thực, giảm đáng kể độ trễ và tiết kiệm tài nguyên.
- Thiết kế cơ sở dữ liệu được chuẩn hóa với các mối quan hệ phức giữa các thực thể Order, Order_Items, Food, Option Group và User.
- Hỗ trợ mở rộng dễ dàng việc tích hợp API bên thứ ba (thanh toán trực tuyến, bản đồ, thông báo đẩy).
- Sử dụng công nghệ cross-platform cho lớp giao diện người dùng, đảm bảo tính nhất quán và giảm chi phí bảo trì.

Điểm khác biệt của AppFoodDelivery nằm ở việc tập trung vào khả năng mở rộng linh hoạt và tối ưu chi phí triển khai thông qua việc sử dụng nền tảng Backend-as-a-Service hiện đại. Giải pháp này đặc biệt phù hợp với các mô hình startup hoặc hệ thống quy mô nhỏ và trung bình tại Việt Nam, giúp rút ngắn thời gian phát triển mà vẫn đảm bảo hiệu suất và tính chuyên nghiệp cao.

1.4. Phương pháp thực hiện

Để đảm bảo tính khoa học, linh hoạt và chất lượng sản phẩm trong quá trình thực hiện đề tài “Phát triển ứng dụng giao đồ ăn trực tuyến AppFoodDelivery”, nhóm tác giả đã áp dụng mô hình Agile kết hợp với phương pháp Scrum. Đây là phương pháp phát triển phần mềm hiện đại, phù hợp với quy mô đồ án tốt nghiệp có thời gian hạn chế và yêu cầu có thể điều chỉnh trong quá trình triển khai.

1.4.1. Lý do lựa chọn mô hình Agile/Scrum

Việc lựa chọn mô hình Agile/Scrum xuất phát từ những đặc điểm cụ thể của hệ thống:

- Yêu cầu chức năng có thể thay đổi hoặc bổ sung trong quá trình phát triển (ví dụ: tối ưu hóa giao diện người dùng, điều chỉnh logic đơn hàng, hoặc chuẩn bị lộ trình tích hợp thanh toán trực tuyến).
- Cần thực hiện kiểm thử liên tục để phát hiện và sửa lỗi sớm, đặc biệt với các tính năng thời gian thực.
- Hệ thống có sự tương tác chặt chẽ giữa các phân hệ (ứng dụng cho khách hàng, ứng dụng cho nhà hàng, ứng dụng cho tài xế) và yêu cầu đồng bộ dữ liệu cao.
- Quy mô đồ án phù hợp với cách làm việc theo từng chu kỳ ngắn (sprint), giúp dễ dàng đánh giá tiến độ và điều chỉnh kịp thời.

So với mô hình thác nước (Waterfall) truyền thống, Agile/Scrum mang lại tính linh hoạt cao hơn, giảm rủi ro và tăng khả năng đáp ứng yêu cầu thực tế của đề tài.

1.4.2. Các giai đoạn thực hiện chính

Quá trình phát triển hệ thống được tổ chức theo phương pháp phát triển (iterative), bao gồm các giai đoạn chính sau:

- Phân tích yêu cầu: Thu thập, phân tích và làm rõ yêu cầu chức năng cũng như phi chức năng từ ba đối tượng người dùng (khách hàng, nhà hàng và tài xế). Sử dụng User Story và Use Case để mô tả chi tiết các chức năng chính của hệ thống.
- Thiết kế hệ thống: Thiết kế kiến trúc tổng thể, thiết kế cơ sở dữ liệu và thiết kế giao diện người dùng cho ba ứng dụng di động.
- Phát triển Backend: Xây dựng API và các dịch vụ backend sử dụng Supabase (PostgreSQL + Realtime), triển khai logic kinh doanh, hệ thống phân quyền và xử lý dữ liệu.
- Phát triển Frontend: Triển khai ứng dụng di động đa nền tảng bằng Flutter cho cả ba phân hệ: Ứng dụng cho khách hàng, ứng dụng cho nhà hàng, ứng dụng cho tài xế.
- Tích hợp và kiểm thử: Tích hợp giữa Frontend và Backend, thực hiện kiểm thử đơn vị, kiểm thử tích hợp và kiểm thử hệ thống. Đặc biệt chú trọng kiểm tra tính năng cập nhật trạng thái đơn hàng thời gian thực và hiệu năng hệ thống.
- Hoàn thiện và đánh giá: Hoàn thiện sản phẩm, viết tài liệu báo cáo, chuẩn bị demo và đánh giá kết quả đạt được so với mục tiêu đề ra.

Bảng 1. Bảng product backlog

ID	User story	Vai trò	Ưu tiên	Story point
US01	Là khách hàng, tôi muốn đăng ký tài khoản với xác thực OTP để đảm bảo tài khoản hợp lệ trước khi sử dụng dịch vụ.	Khách hàng	Cao	5
US02	Là khách hàng, tôi muốn đăng nhập bằng email/password để truy cập hệ thống nhanh chóng.	Khách hàng	Cao	2
US03	Là nhà hàng, tôi muốn đăng ký và đăng nhập tài khoản nhà hàng để quản lý cửa hàng trên nền tảng.	Nhà hàng	Cao	3
US04	Là shipper, tôi muốn đăng ký và đăng nhập tài khoản shipper để có thể nhận đơn giao hàng.	Shipper	Cao	3
US05	Là hệ thống, cần thiết kế và cài đặt database schema PostgreSQL (ERD mức logic & vật lý) để lưu trữ toàn bộ dữ liệu.	Hệ thống	Cao	5
US06	Là hệ thống, cần xây dựng API Authentication tích hợp Firebase để phân quyền an toàn cho 3 vai trò.	Hệ thống	Cao	5
US07	Là khách hàng, tôi muốn tìm kiếm nhà hàng theo vị trí hiện tại để nhanh chóng chọn nơi đặt món gần nhất.	Khách hàng	Cao	5
US08	Là khách hàng, tôi muốn xem thực đơn và chi tiết món ăn (hình ảnh, giá, topping) để lựa chọn phù hợp.	Khách hàng	Cao	3

US09	Là khách hàng, tôi muốn quản lý giỏ hàng (thêm, sửa số lượng, xóa món) để chuẩn bị đặt hàng linh hoạt.	Khách hàng	Cao	5
US10	Là khách hàng, tôi muốn đặt đơn hàng và thanh toán COD để hoàn tất giao dịch mua món ăn.	Khách hàng	Cao	5
US11	Là khách hàng, tôi muốn quản lý tài khoản và nhiều địa chỉ giao hàng để thuận tiện trong các lần đặt hàng sau.	Khách hàng	Trung	3
US12	Là khách hàng, tôi muốn lưu danh sách món ăn yêu thích để tái đặt hàng nhanh hơn.	Khách hàng	Thấp	2
US13	Là khách hàng, tôi muốn xem lịch sử đơn hàng để tra cứu các giao dịch trước đây.	Khách hàng	Trung	2
US14	Là hệ thống, cần xây dựng API quản lý đơn hàng (tạo, cập nhật trạng thái, truy vấn lịch sử) phục vụ 3 phân hệ.	Hệ thống	Cao	8
US15	Là nhà hàng, tôi muốn quản lý thực đơn (thêm, sửa, xóa, bật/tắt tình trạng món) để cập nhật menu linh hoạt.	Nhà hàng	Cao	5
US16	Là nhà hàng, tôi muốn tiếp nhận và xác nhận đơn hàng từ khách để bắt đầu chuẩn bị món ăn kịp thời.	Nhà hàng	Cao	5
US17	Là nhà hàng, tôi muốn xác nhận món ăn đã nấu xong để thông báo cho shipper đến lấy hàng.	Nhà hàng	Cao	3

US18	Là hệ thống, cần tích hợp Supabase Realtimedể đồng bộ trạng thái đơn hàng tức thời giữa khách hàng, nhà hàng và shipper.	Hệ thống	Cao	8
US19	Là nhà hàng, tôi muốn xem thống kê doanh thu theo ngày/tháng để đánh giá hiệu quả kinh doanh.	Nhà hàng	Trung	5
US20	Là shipper, tôi muốn bật/tắt chế độ bắt đơn để chủ động kiểm soát việc nhận đơn hàng theo lịch làm việc của mình.	Shipper	Cao	3
US21	Là shipper, tôi muốn xem danh sách và nhận đơn hàng gần vị trí hiện tại để tối ưu quãng đường di chuyển.	Shipper	Cao	5
US22	Là shipper, tôi muốn cập nhật trạng thái giao hàng và vị trí real-time để các bên liên quan theo dõi được tiến độ.	Shipper	Cao	8
US24	Là khách hàng, tôi muốn theo dõi đơn hàng và vị trí shipper real-time trên bản đồ để biết tiến độ giao hàng.	Khách hàng	Cao	8
US25	Là hệ thống, cần tích hợp Push Notification để thông báo đẩy cho các bên khi trạng thái đơn hàng thay đổi.	Hệ thống	Trung	5
US26	Là shipper, tôi muốn xem thống kê thu nhập theo ngày/tháng để theo dõi hiệu quả làm việc của mình.	Shipper	Trung	3
US27	Là nhà hàng, tôi muốn quản lý hồ sơ cửa hàng(tên, ảnh, địa chỉ, giờ mở cửa) để khách hàng nhận biết đúng thông tin.	Nhà hàng	Trung	3

US28	Là hệ thống, cần kiểm thử toàn hệ thống (unit test, integration test, system test) để đảm bảo chất lượng sản phẩm.	Hệ thống	Cao	5
------	--	----------	-----	---

Product backlog được xây dựng nhằm tổng hợp toàn bộ các User Story của hệ thống đặt đồ ăn trực tuyến, phản ánh đầy đủ yêu cầu chức năng từ ba nhóm người dùng chính: khách hàng, nhà hàng và tài xế giao hàng (shipper). Để đảm bảo tính khoa học và hỗ trợ hiệu quả cho quá trình lập kế hoạch Sprint, nhóm tiến hành phân loại và sắp xếp các User Story dựa trên ba tiêu chí sau:

- Vai trò (Role): Mỗi User Story được gán cho một vai trò người dùng cụ thể, giúp xác định rõ phạm vi và đối tượng thụ hưởng của từng chức năng, đồng thời đảm bảo hệ thống đáp ứng đúng nhu cầu của từng phân hệ.

- Mức độ ưu tiên (Priority): Các User Story được phân loại theo ba mức độ ưu tiên như sau:

+ Cao: Các chức năng cốt lõi, thiết yếu cần được triển khai trong giai đoạn đầu để hệ thống có thể vận hành cơ bản.

+ Trung bình: Các chức năng hỗ trợ, góp phần nâng cao trải nghiệm người dùng và hoàn thiện luồng nghiệp vụ.

+ Thấp: Các chức năng mở rộng, cải thiện tiện ích sử dụng và có thể được phát triển ở các giai đoạn sau.

- Story Point: Được sử dụng để ước lượng độ phức tạp và khối lượng công việc tương đối của từng User Story theo thang điểm tương đối (thường dựa trên dãy Fibonacci). Đây là cơ sở quan trọng để lập kế hoạch và cân đối năng lực thực hiện trong từng Sprint.

Bảng 2. Bảng chia sprint

Sprint 1: 06/01/2026 – 24/01/2026 (3 tuần) Mục tiêu: Xây dựng nền tảng hệ thống và module xác thực				
ID	Task	Loại	Story point	Kết quả
US05	Thiết kế ERD và cài đặt database PostgreSQL trên Supabase (Prisma schema)	Backend	5	Schema đầy đủ 12 bảng, migration chạy thành công
US06	Xây dựng API Authentication tích hợp Firebase, phân quyền 3 vai trò (Customer / Restaurant / Shipper)	Backend	5	JWT token hợp lệ, middleware phân quyền hoạt động đúng
US01	Màn hình đăng ký khách hàng với luồng xác thực OTP trước khi tạo tài khoản	Frontend	5	OTP gửi và xác thực đúng, tài khoản được tạo trong DB
US02	Màn hình đăng nhập khách hàng (email/password) với xử lý lỗi và phiên đăng nhập	Frontend	2	Đăng nhập thành công, token lưu local, điều hướng đúng
US03	Màn hình đăng ký/đăng nhập nhà hàng với xác thực tài khoản	Frontend	3	Nhà hàng đăng nhập thành công, chuyển đúng màn hình Restaurant
US04	Màn hình đăng ký/đăng nhập shipper với xác thực tài khoản	Frontend	3	Shipper đăng nhập thành công, chuyển đúng màn hình Shipper
Tổng Sprint 1			23	Hệ thống auth hoàn chỉnh, 3 vai trò có thể đăng nhập

Sprint 2 26/01/2026 – 13/02/2026 (3 tuần)				
Mục tiêu: Hoàn thiện toàn bộ Customer App				
US07	Màn hình trang chủ: tìm kiếm nhà hàng theo vị trí GPS, hiển thị danh sách kèm khoảng cách	Frontend	5	Danh sách nhà hàng hiển thị đúng, sắp xếp theo khoảng cách
US08	Màn hình chi tiết nhà hàng: xem thực đơn, phân loại, chi tiết món ăn kèm topping/option	Frontend	3	Hiển thị đầy đủ menu, option group render đúng
US09	Màn hình giỏ hàng: thêm/sửa số lượng/xóa món, tính tổng tiền tự động kèm phí vận chuyển	Frontend	5	Giỏ hàng cập nhật real-time, tổng tiền tính đúng
US10	Màn hình đặt hàng và thanh toán COD: chọn địa chỉ, xác nhận đơn, gửi lên server	Frontend	5	Đơn hàng tạo thành công trong DB, trạng thái "Chờ xác nhận"
US11	Màn hình hồ sơ: cập nhật thông tin cá nhân, thêm/sửa/xóa nhiều địa chỉ giao hàng	Frontend	3	Thông tin lưu đúng DB, chọn địa chỉ mặc định hoạt động
US12	Tính năng yêu thích: thêm/xóa nhà hàng/món ăn vào danh sách yêu thích	Frontend	2	Favorite toggle hoạt động, dữ liệu persist sau khi restart app
US13	Màn hình lịch sử đơn hàng: xem danh sách các đơn cũ với trạng thái và tổng tiền	Frontend	2	Lịch sử hiển thị đúng, có thể xem chi tiết từng đơn
Tổng Sprint 2			25	Customer App hoàn chỉnh, luồng đặt hàng hoạt động

Sprint 3 16/02/2026 – 06/03/2026 (3 tuần) Mục tiêu: Restaurant App + Realtime backend				
US14	Xây dựng API quản lý đơn hàng đầy đủ (CRUD + cập nhật trạng thái theo luồng nghiệp vụ)	Backend	8	endpoint đúng logic, trả về đúng status
US18	Tích hợp Supabase Realtime: subscribe bảng orders, broadcast trạng thái tức thời tới 3 phân hệ	Backend	8	Cập nhật trạng thái xuất hiện trên thiết bị khách/ nhà hàng/ shipper
US15	Màn hình quản lý thực đơn: thêm/sửa/xóa món, upload ảnh, thiết lập tùy chọn (option group)	Frontend	5	CRUD món ăn hoạt động, ảnh upload lên Supabase Storage thành công
US16	Màn hình quản lý đơn hàng (nhà hàng): xem đơn mới, xác nhận/từ chối đơn hàng	Frontend	5	Đơn hàng realtime cập nhật, trạng thái đổi đúng khi nhà hàng thao tác
US17	Nút xác nhận "Món ăn đã sẵn sàng" để thông báo shipper đến lấy hàng	Frontend	3	Trạng thái đơn chuyển "Sẵn sàng", shipper nhận thông báo tức thì
US19	Màn hình thống kê doanh thu: biểu đồ doanh thu theo ngày/tháng, tổng đơn hàng	Frontend	5	Biểu đồ render đúng dữ liệu từ DB, filter theo ngày hoạt động
US27	Màn hình hồ sơ cửa hàng: cập nhật tên, ảnh đại diện, địa chỉ, giờ hoạt động	Frontend	3	Thông tin nhà hàng lưu và hiển thị đúng trên Customer App

Tổng Sprint 3			37 SP	Restaurant App hoàn chỉnh + Realtime hoạt động ổn định
Sprint 4 09/03/2026 – 27/03/2026 (3 tuần) Mục tiêu: Shipper App + Real-time tracking + Kiểm thử và hoàn thiện				
US20	Màn hình bật/tắt chế độ online: shipper chủ động nhận/ngừng nhận đơn hàng	Frontend	3	Toggle trạng thái hoạt động, đơn hàng chỉ gửi khi shipper online
US21	Màn hình danh sách đơn hàng chờ: xem đơn gần vị trí, xem chi tiết và nhận đơn	Frontend	3	Đơn hàng hiển thị đúng, nhận đơn thành công và chuyển trạng thái
US22	Cập nhật vị trí GPS shipper real-time và trạng thái đơn (đang lấy hàng / đang giao)	Frontend	8	Vị trí cập nhật <5s, khách hàng thấy shipper di chuyển trên bản đồ
US23	Nút xác nhận "Giao hàng thành công": hoàn tất đơn hàng và cập nhật trạng thái cuối	Frontend	3	Đơn hàng chuyển trạng thái "Hoàn thành", KH nhận thông báo giao thành công
US24	Màn hình theo dõi đơn hàng (Customer): hiển thị bản đồ, vị trí shipper, trạng thái real-time	Frontend	8	Bản đồ hiển thị đúng, shipper marker di chuyển mượt, trạng thái đồng bộ
US25	Gửi thông báo đẩy cho KH, nhà hàng, shipper khi đơn thay đổi	Backend	5	Thông báo gửi đúng thiết bị đích, nội dung rõ ràng và kịp thời
US26	Màn hình thống kê thu nhập shipper: doanh thu theo ngày/tuần/tháng, số đơn hoàn thành	Frontend	3	Số liệu thống kê chính xác khớp với DB, biểu đồ hiển thị đúng

US28	Kiểm thử hệ thống: viết kịch bản test, thực hiện kiểm thử tích hợp và hệ thống, sửa lỗi	Testing	5	Toàn bộ test case pass, báo cáo kiểm thử hoàn chỉnh
Tổng Sprint 4			38	Shipper App hoàn chỉnh, hệ thống đã kiểm thử, sẵn sàng demo
Tổng kết: 4 Sprints $\times$ 3 tuần = 12 tuần (06/01/2026 – 27/03/2026) Tổng: 123 Story Points Sprint 1: 23 SP Sprint 2: 25 SP Sprint 3: 37 SP Sprint 4: 38 SP				

Kế hoạch Sprint được xây dựng theo hướng phát triển tăng dần chức năng (Incremental Delivery), đảm bảo các thành phần quan trọng của hệ thống được triển khai theo đúng thứ tự phụ thuộc kỹ thuật và nghiệp vụ.

- Sprint 1 tập trung xây dựng nền tảng hệ thống, bao gồm thiết kế cơ sở dữ liệu, triển khai cơ chế xác thực người dùng và phân quyền cho ba vai trò chính. Đây là giai đoạn đặt nền móng kiến trúc cho toàn bộ hệ thống.
- Sprint 2 hướng đến hoàn thiện ứng dụng dành cho khách hàng (Customer App), triển khai đầy đủ luồng nghiệp vụ đặt món từ tìm kiếm nhà hàng, xem thực đơn, quản lý giỏ hàng đến tạo đơn hàng.
- Sprint 3 phát triển phân hệ nhà hàng (Restaurant App) kết hợp xây dựng các API quản lý đơn hàng và tích hợp cơ chế đồng bộ dữ liệu thời gian thực, giúp các bên liên quan cập nhật trạng thái đơn hàng tức thời.
- Sprint 4 hoàn thiện ứng dụng dành cho tài xế giao hàng (Shipper App), triển khai chức năng theo dõi vị trí thời gian thực, thông báo đẩy và thực hiện kiểm thử tổng thể nhằm đảm bảo tính ổn định của hệ thống trước khi bàn giao.

1.4.3. Công cụ hỗ trợ phát triển

Trong quá trình thực hiện, đã sử dụng một số công cụ sau để hỗ trợ phát triển:

- Quản lý phiên bản mã nguồn: Git và GitHub.
- Phát triển ứng dụng: Flutter (Frontend) và Supabase (Backend).
- Kiểm thử API: Postman.
- Thiết kế cơ sở dữ liệu: Prisma ORM và Supabase Studio.

Việc áp dụng mô hình Agile/Scrum cùng với các công cụ phù hợp đã giúp quá trình phát triển được kiểm soát tốt, đảm bảo tiến độ và chất lượng sản phẩm trong khuôn khổ đồ án tốt nghiệp.

1.5. Đối tượng sử dụng

Ứng dụng AppFoodDelivery được thiết kế để phục vụ đồng thời ba nhóm đối tượng chính, mỗi nhóm có vai trò và nhu cầu sử dụng riêng biệt. Việc xác định rõ ràng các đối tượng sử dụng giúp hệ thống định hướng chức năng một cách chính xác, đảm bảo trải nghiệm tối ưu và xây dựng cơ chế phân quyền phù hợp.

1.5.1. Khách hàng (Customer)

Đây là nhóm đối tượng chính của ứng dụng. Khách hàng sử dụng AppFoodDelivery để:

- Tìm kiếm và khám phá các nhà hàng, quán ăn gần vị trí hiện tại.
- Xem chi tiết menu, chọn món ăn kèm theo các tùy chọn bổ sung (size, topping...).
- Đặt hàng, quản lý giỏ hàng và thanh toán (hiện tại hỗ trợ hình thức COD).
- Theo dõi vị trí tài xế và trạng thái đơn hàng theo thời gian thực.
- Đánh giá chất lượng món ăn, dịch vụ giao hàng và nhà hàng sau khi nhận hàng.

1.5.2. Nhà hàng (Restaurant)

Nhà hàng là đối tác quan trọng, đóng vai trò cung cấp sản phẩm và

dịch vụ cho khách hàng. Nhóm này sử dụng ứng dụng để:

- Quản lý thực đơn (thêm, sửa, xóa món ăn, cập nhật giá, hình ảnh và mô tả).
- Bật/tắt tình trạng món ăn (còn hàng hoặc hết hàng) một cách linh hoạt.
- Tiếp nhận, xác nhận và xử lý đơn hàng từ khách hàng.
- Theo dõi danh sách đơn hàng đang chờ xử lý và đã hoàn thành.
- Xem báo cáo doanh thu cơ bản theo ngày hoặc tháng.

1.5.3. Tài xế giao hàng (Shipper)

Tài xế là yếu tố then chốt trong chuỗi giao nhận đồ ăn. Nhóm này sử dụng ứng dụng để:

- Xem danh sách đơn hàng khả dụng gần vị trí hiện tại.
- Nhận đơn hàng và cập nhật trạng thái đơn hàng (đang lấy hàng, đang giao hàng...).
- Sử dụng bản đồ để định vị và dẫn đường tối ưu từ nhà hàng đến khách hàng.
- Cập nhật vị trí thời gian thực trong quá trình giao hàng.
- Xác nhận giao hàng thành công và nhận đánh giá từ khách hàng.

Việc phân chia rõ ràng ba nhóm đối tượng sử dụng giúp hệ thống AppFoodDelivery xây dựng được cơ chế phân quyền dựa trên vai trò (Role-Based Access Control) chặt chẽ. Mỗi nhóm người dùng chỉ được truy cập và thực hiện những chức năng phù hợp với vai trò của mình, từ đó nâng cao tính bảo mật, tính chuyên nghiệp và hiệu quả vận hành tổng thể của hệ thống.

1.6. Kiến trúc và mô hình phát triển ứng dụng

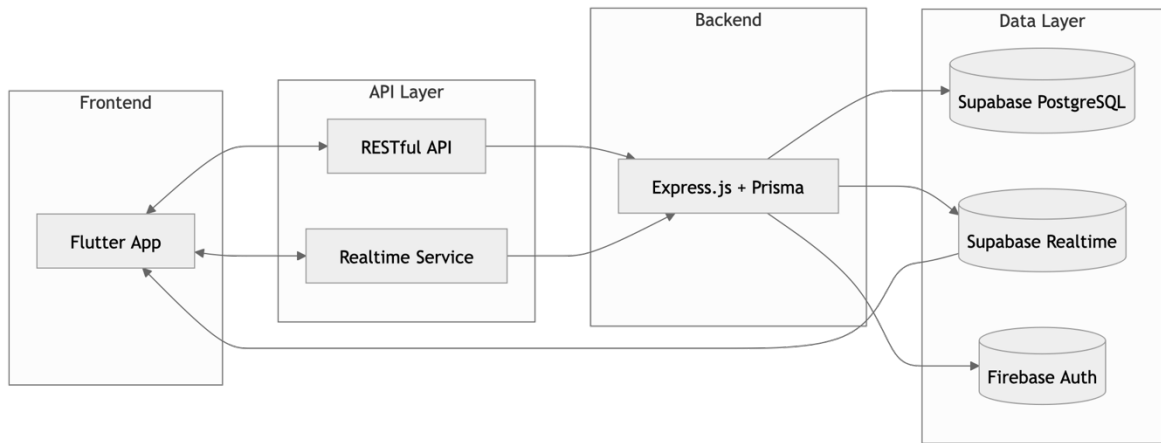
Để đảm bảo hệ thống hoạt động ổn định, dễ bảo trì và có khả năng mở rộng trong tương lai, AppFoodDelivery được thiết kế với kiến trúc rõ ràng và các mô hình phát triển hiện đại. Phần này trình bày tổng quan về kiến trúc hệ thống cũng như mô hình phát triển được áp dụng ở cả hai phía Frontend và Backend.

1.6.1. Kiến trúc hệ thống

Hệ thống AppFoodDelivery được xây dựng theo mô hình Client-Server truyền thống, trong đó:

- Frontend (lớp giao diện người dùng) được phát triển bằng Flutter, chịu trách nhiệm xây dựng ba ứng dụng di động (ứng dụng cho khách hàng, ứng dụng cho nhà hàng, ứng dụng cho Tài xế).
- Backend xử lý logic kinh doanh, lưu trữ dữ liệu và cung cấp dịch vụ cho Frontend thông qua RESTful API.
- Để cân bằng giữa hiệu suất, chi phí và khả năng mở rộng, hệ thống áp dụng kiến trúc Micro-services lite (kiến trúc vi dịch vụ nhẹ). Cụ thể, hệ thống kết hợp các dịch vụ chuyên biệt sau:
 - Firebase Authentication: Quản lý xác thực người dùng (Google Sign-In, Email/Password) và sinh token bảo mật.
 - Supabase: Làm nền tảng chính cho cơ sở dữ liệu, dịch vụ Realtime (đồng bộ trạng thái đơn hàng và vị trí tài xế), và một số chức năng backend-as-a-service khác.
 - Backend tự triển khai (Node.js + Express.js): Xử lý các logic kinh doanh phức tạp, tích hợp Prisma ORM để giao tiếp với cơ sở dữ liệu.

Kiến trúc này giúp tách biệt rõ ràng các thành phần, giảm sự phụ thuộc lẫn nhau, đồng thời tận dụng tối đa sức mạnh của các dịch vụ đám mây sẵn có, từ đó giảm thời gian phát triển và chi phí vận hành cho một đề án quy mô sinh viên. Sơ đồ kiến trúc tổng thể của hệ thống được mô tả như sau:



Hình 1. Sơ đồ kiến trúc hệ thống

Sơ đồ mô tả kiến trúc tổng thể của hệ thống AppFoodDelivery theo mô hình nhiều lớp. Ứng dụng Flutter (Frontend) giao tiếp với backend thông qua RESTful API và kênh Realtime. Backend được xây dựng bằng Express.js kết hợp Prisma ORM để xử lý nghiệp vụ và truy xuất dữ liệu từ Supabase PostgreSQL. Hệ thống sử dụng Supabase Realtime để đồng bộ dữ liệu theo thời gian thực và Firebase Authentication để xác thực người dùng.

1.6.2. Mô hình phát triển ứng dụng

◆ Phía Frontend (Mobile Development)

Ứng dụng di động được xây dựng theo Clean Architecture kết hợp với mô hình quản lý trạng thái Provider^{[15], [17]}. Cụ thể:

- Clean Architecture: Phân tầng rõ ràng thành các lớp (Core, Features, Data, Domain), giúp tách biệt logic kinh doanh khỏi giao diện người dùng và nguồn dữ liệu. Điều này làm cho mã nguồn dễ đọc, dễ kiểm thử và dễ mở rộng khi thêm tính năng mới.
- State Management với Provider: Sử dụng Provider để quản lý trạng thái ứng dụng (giỏ hàng, trạng thái đơn hàng, vị trí người dùng...) một cách hiệu quả và nhẹ nhàng, tránh tình trạng rebuild toàn bộ widget không cần thiết.

◆ Phía Backend

Backend được tổ chức theo mô hình MVC (Model - View - Controller) cổ điển nhưng được tối ưu cho môi trường Node.js:

- Model: Sử dụng Prisma ORM để định nghĩa schema dữ liệu và thực hiện các truy vấn phức tạp với cơ sở dữ liệu PostgreSQL.
- Controller: Xử lý logic kinh doanh, nhận request từ Frontend và trả về response phù hợp.
- Route: Định nghĩa các endpoint RESTful API một cách rõ ràng và có cấu trúc.

Mô hình này giúp mã nguồn backend trở nên gọn gàng, dễ bảo trì và dễ dàng tích hợp thêm các tính năng mới như thanh toán trực tuyến hoặc thông báo đẩy nâng cao trong tương lai.

Việc kết hợp Clean Architecture ở Frontend và MVC ở Backend đã tạo nên một hệ thống có cấu trúc rõ ràng, dễ phát triển theo nhóm và phù hợp với mục tiêu của đề tài.

1.7. Công cụ và công nghệ sử dụng phát triển phần mềm.

Để phát triển ứng dụng AppFoodDelivery một cách hiệu quả, cá nhân thực hiện đã lựa chọn bộ công cụ và công nghệ hiện đại, mã nguồn mở, ổn định và phù hợp với quy mô đề án. Các lựa chọn này đảm bảo tính cross-platform, khả năng mở rộng, chi phí thấp và hỗ trợ mạnh mẽ cho tính năng thời gian thực.

1.7.1. Công cụ phát triển phần mềm.

1.7.1.1. Visual Studio Code (VS Code)

Visual Studio Code (VS Code) được lựa chọn làm môi trường phát triển chính (Primary IDE). Đây không chỉ là một trình soạn thảo mã nguồn nhẹ mà còn đóng vai trò là một "Trung tâm điều phối phát triển" (Development Hub) toàn diện, kết nối chặt chẽ giữa Frontend và Backend. Tối ưu hóa hiệu năng và trải nghiệm lập trình (Developer Experience - DX):

- Hiệu suất hệ thống: Được xây dựng trên nền tảng Electron, VS Code duy trì sự cân bằng giữa tính năng mạnh mẽ và việc tiêu tốn ít tài nguyên (RAM/CPU), đảm bảo môi trường làm việc mượt mà ngay cả khi chạy đồng thời nhiều trình giả lập và server backend.
- Giao diện tùy biến: Hệ thống phím tắt linh hoạt kết hợp với Command Palette giúp tối ưu hóa tốc độ thao tác, giảm thiểu thời gian rời tay khỏi bàn phím.

Bộ công cụ phân tích và Debug chuyên sâu (Flutter DevTools) vượt xa khả năng viết code thông thường, hệ thống tận dụng bộ Flutter DevTools được tích hợp sẵn trong VS Code để tối ưu hóa ứng dụng:

- Widget Inspector: Cho phép kiểm tra cấu trúc phân cấp của Widget Tree, hỗ trợ debug lỗi layout và căn chỉnh giao diện một cách trực quan.
- Performance và memory profiling: Theo dõi biểu đồ render frame thời gian thực để phát hiện các tình trạng giật lag. Đồng thời, công cụ này giúp kiểm soát bộ nhớ, phát hiện rò rỉ dữ liệu trong suốt vòng đời ứng dụng.
- CPU Profiler: Phân tích các hàm tốn nhiều tài nguyên xử lý, từ đó tối ưu hóa logic code để ứng dụng vận hành mượt mà nhất.

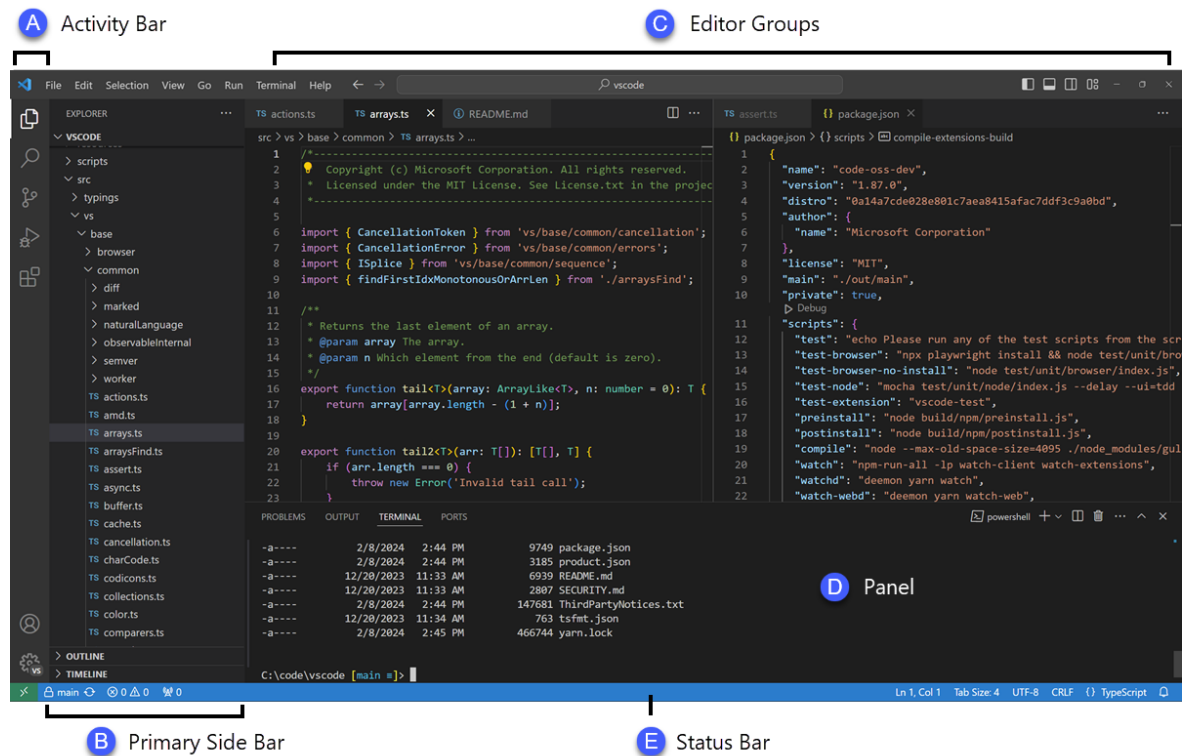
Quản lý hệ thống toàn diện với multi-root workspace Thay vì quản lý các phần của hệ thống rời rạc, sử dụng tính năng workspace đa hệ thống:

- Hợp nhất Frontend & Backend: Cho phép mở đồng thời thư mục Flutter và Express/Node.js (Backend) trong cùng một cửa sổ làm việc.
- Đồng bộ quy trình: Giúp dễ dàng đối chiếu mã nguồn, quản lý cấu hình chung và chuyển đổi nhanh chóng giữa việc viết API và tích hợp giao diện mà không làm gián đoạn luồng suy nghĩ.

Quy trình Debug API và Database trực tiếp Để đẩy nhanh tốc độ phát triển và giảm sự phụ thuộc vào các công cụ bên thứ ba (như Postman), hệ thống tích hợp các công cụ hỗ trợ ngay tại IDE:

- REST Client / Thunder Client: Cho phép viết và gửi các request HTTP, kiểm tra kết quả trả về từ API ngay trong giao diện code. Điều này giúp quá trình kiểm thử các luồng dữ liệu Backend trở nên trực quan và nhanh chóng hơn.
- Prisma Extension: Hỗ trợ định dạng và kiểm tra lỗi cú pháp cho file schema. Đồng thời kết hợp với Prisma Studio để quản lý dữ liệu trực tiếp, đảm bảo tính nhất quán giữa code và cơ sở dữ liệu thực tế.
- Quản lý phiên bản và Source Control Timeline Hệ thống kiểm soát phiên bản Git được tích hợp sâu với các tính năng nâng cao:
- Source Control Timeline: Cung cấp cái nhìn chi tiết về lịch sử thay đổi của từng file theo thời gian. Tính năng này cực kỳ hữu ích trong việc truy vết các thay đổi gây lỗi và hỗ trợ phục hồi các phiên bản ổn định trước đó một cách chính xác.
- Visual Diff: So sánh trực quan các thay đổi trong mã nguồn trước khi thực hiện Commit, giúp kiểm soát chất lượng code chặt chẽ.

Việc lựa chọn VS Code làm công cụ chủ đạo không chỉ giúp tối ưu hiệu suất hệ thống mà còn nâng cao đáng kể trải nghiệm lập trình (Developer Experience). Nhờ khả năng tích hợp toàn diện từ Frontend (Flutter) đến Backend (Node.js/Prisma), VS Code đóng vai trò như một trung tâm phát triển hợp nhất, giúp giảm độ phức tạp trong quá trình xây dựng, tối ưu hóa hiệu năng và vận hành hệ thống Fullstack một cách chuyên nghiệp.



Hình 2. Giao diện minh họa của Visual Studio Code

1.7.1.3. Công cụ quản lý cơ sở dữ liệu

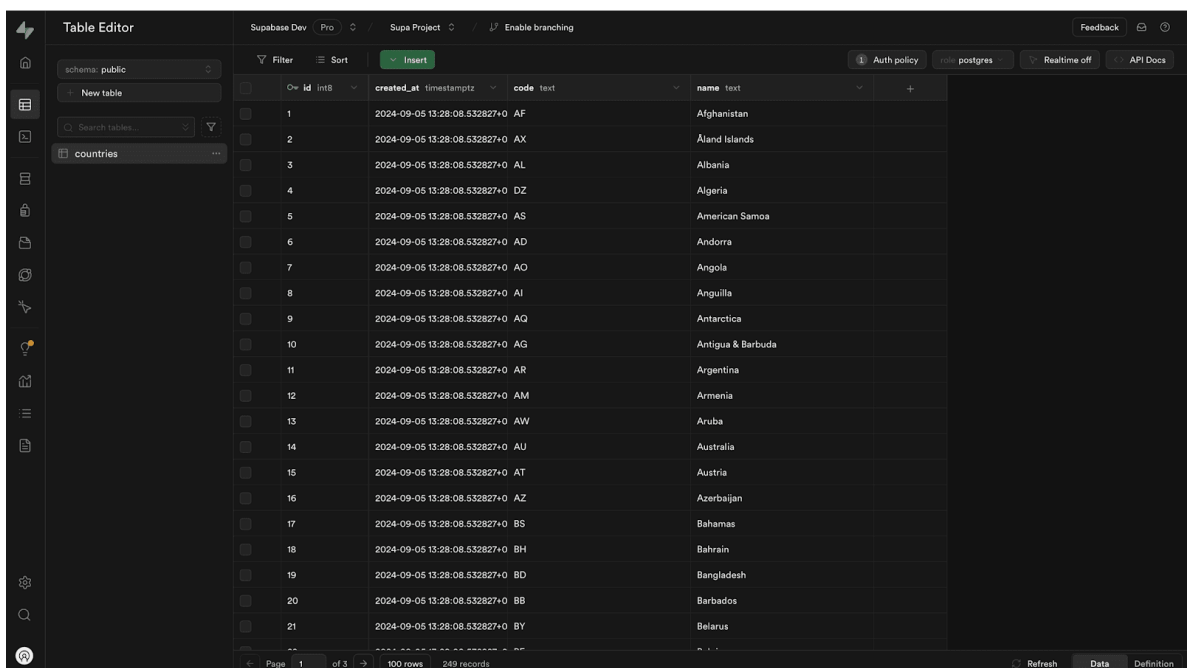
Để đảm bảo tính toàn vẹn của dữ liệu và tối ưu hóa quy trình vận hành Backend, hệ thống kết hợp sử dụng hai công cụ quản trị mạnh mẽ là Supabase Dashboard và Prisma Studio. Sự kết hợp này giúp kiểm soát dữ liệu từ mức hạ tầng hệ thống đến mức thực thi mã nguồn.

Supabase Dashboard là một nền tảng mã nguồn mở thay thế cho Firebase, dựa trên hệ quản trị cơ sở dữ liệu PostgreSQL^{[21], [23]}. Supabase Dashboard cung cấp giao diện quản lý trực quan cho các dịch vụ cốt lõi:

- Table Editor (Quản trị bảng dữ liệu): Cung cấp giao diện dạng bảng (spreadsheet-like) giúp thao tác thêm, xóa, sửa dữ liệu và cấu hình các ràng buộc (Constraints), khóa ngoại (Foreign Keys) một cách dễ dàng mà không cần viết các câu lệnh SQL phức tạp.
- Authentication Management (Quản lý người dùng): Cho phép theo dõi danh sách người dùng, cấu hình các phương thức đăng nhập (Email/Password, Social

Login) và thiết lập chính sách bảo mật Row Level Security (RLS) để bảo vệ dữ liệu ở mức dòng.

- Realtime Engine: Công cụ giám sát các thay đổi dữ liệu theo thời gian thực. Dashboard cho phép kiểm tra việc gửi/nhận các sự kiện (Insert, Update, Delete) giúp đảm bảo tính năng Realtime trên ứng dụng di động hoạt động chính xác.
- SQL Editor: Tích hợp môi trường thực thi các câu lệnh SQL nâng cao, cho phép tạo các Store Procedures, Functions và Triggers để xử lý các logic nghiệp vụ phức tạp trực tiếp dưới database.



	id	created_at	code	name
	1	2024-09-05 13:28:08.532827+0	AF	Afghanistan
	2	2024-09-05 13:28:08.532827+0	AX	Åland Islands
	3	2024-09-05 13:28:08.532827+0	AL	Albania
	4	2024-09-05 13:28:08.532827+0	DZ	Algeria
	5	2024-09-05 13:28:08.532827+0	AS	American Samoa
	6	2024-09-05 13:28:08.532827+0	AD	Andorra
	7	2024-09-05 13:28:08.532827+0	AO	Angola
	8	2024-09-05 13:28:08.532827+0	AI	Anguilla
	9	2024-09-05 13:28:08.532827+0	AQ	Antarctica
	10	2024-09-05 13:28:08.532827+0	AG	Antigua & Barbuda
	11	2024-09-05 13:28:08.532827+0	AR	Argentina
	12	2024-09-05 13:28:08.532827+0	AM	Armenia
	13	2024-09-05 13:28:08.532827+0	AW	Aruba
	14	2024-09-05 13:28:08.532827+0	AU	Australia
	15	2024-09-05 13:28:08.532827+0	AT	Austria
	16	2024-09-05 13:28:08.532827+0	AZ	Azerbaijan
	17	2024-09-05 13:28:08.532827+0	BS	Bahamas
	18	2024-09-05 13:28:08.532827+0	BH	Bahrain
	19	2024-09-05 13:28:08.532827+0	BD	Bangladesh
	20	2024-09-05 13:28:08.532827+0	BB	Barbados
	21	2024-09-05 13:28:08.532827+0	BY	Belarus

Hình 3. Hình ảnh minh họa Dashboard Supabase

Prisma Studio (Trình quản trị dữ liệu phía Client) Prisma Studio là một giao diện đồ họa đi kèm với Prisma ORM, đóng vai trò là cầu nối giữa mã nguồn và cơ sở dữ liệu trong quá trình phát triển cục bộ:

- Visual CRUD Operations: Hỗ trợ xem, lọc, sắp xếp và chỉnh sửa dữ liệu cực kỳ nhanh chóng. Mọi thay đổi trên Prisma Studio được đồng bộ hóa ngay lập tức với cơ sở dữ liệu, giúp tiết kiệm thời gian kiểm thử API.
- Relationship Visualization (Trực quan hóa quan hệ): Điểm mạnh nhất của Prisma Studio là khả năng hiển thị các mối quan hệ (Relation) giữa các bảng.

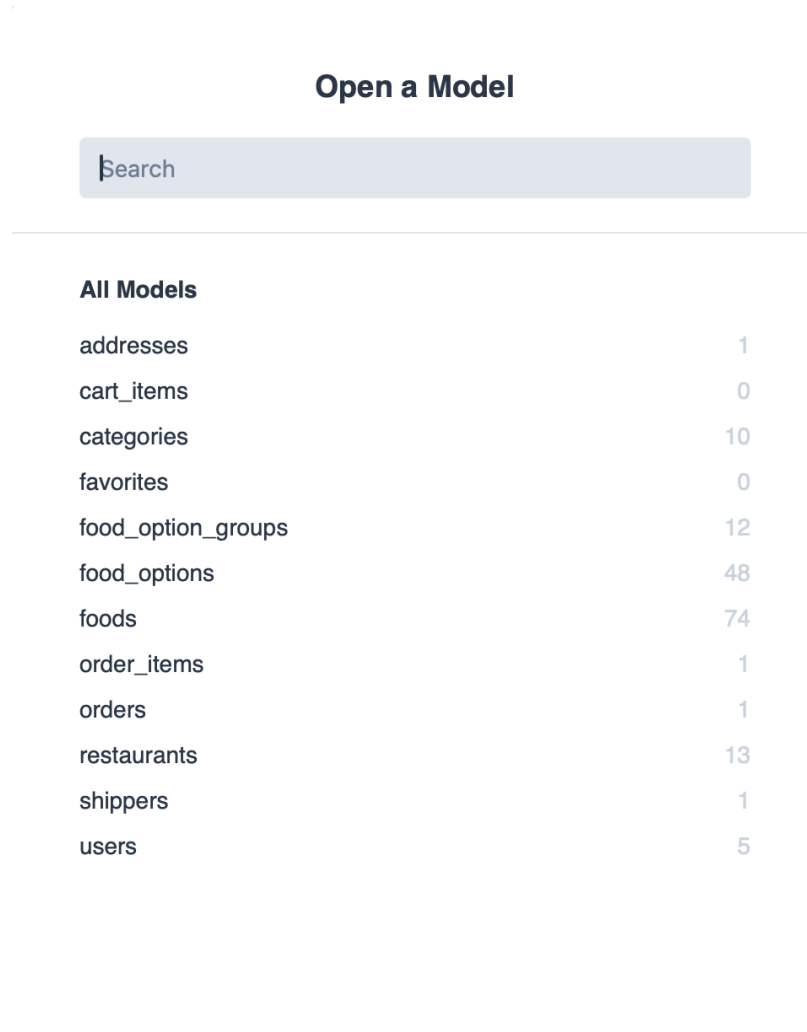
Có thể "nhấn" để chuyển hướng giữa các bản ghi có liên quan (ví dụ: từ một Đơn hàng nhảy sang thông tin Chi tiết Người dùng), giúp kiểm tra tính logic của lược đồ dữ liệu.

- Zero-Configuration: Khác với các công cụ quản lý DB truyền thống, Prisma Studio tự động nhận diện cấu hình từ file `schema.prisma`, đảm bảo giao diện quản trị luôn khớp 100% với cấu trúc code mà không cần cấu hình thủ công.

Việc sử dụng song song hai công cụ này mang lại quy trình làm việc khép kín:

- Supabase đóng vai trò là "Trung tâm vận hành": Quản lý hạ tầng, bảo mật lưu trữ và giám sát hiệu năng của Database PostgreSQL trên Cloud.
- Prisma Studio giúp nhanh chóng kiểm tra dữ liệu trong quá trình code và debug Backend mà không cần rời khỏi môi trường phát triển cục bộ.

Sự kết hợp này không chỉ giúp tăng tốc độ phát triển mà còn đảm bảo hệ thống có khả năng mở rộng cao. Nhờ khả năng quản trị trực quan, có thể dễ dàng phát hiện các sai sót về logic dữ liệu ngay từ giai đoạn đầu, từ đó giảm thiểu rủi ro lỗi hệ thống khi triển khai thực tế.



Hình 4. Hình ảnh minh họa Dashboard Prisma Studio

1.7.1.4. Công cụ kiểm thử API (Postman)

Để đảm bảo các điểm cuối dịch vụ hoạt động chính xác trước khi tiến hành tích hợp với ứng dụng Flutter, hệ thống sử dụng Postman làm công cụ chủ đạo trong quá trình kiểm thử, gỡ lỗi và đối soát dữ liệu giữa phía Client và Server.

Việc tích hợp Postman Extension trực tiếp vào môi trường phát triển Visual Studio Code giúp thực hiện kiểm thử API ngay trong quá trình phát triển Backend, từ đó rút ngắn thời gian chuyển đổi ngữ cảnh làm việc và nâng cao hiệu quả phát triển hệ thống.

♦ Kiểm thử và xác thực phản hồi

Postman cung cấp môi trường mô phỏng các yêu cầu từ ứng dụng di động gửi đến máy chủ, cho phép kiểm tra toàn diện phản hồi của hệ thống:

- Xác thực mã trạng thái HTTP (Status Code): Kiểm tra tính chính xác của các mã phản hồi tiêu chuẩn như 200 OK, 401 Unauthorized hoặc 500 Internal Server Error, qua đó đánh giá mức độ xử lý đúng đắn của Backend.
- Kiểm tra cấu trúc dữ liệu JSON: Phân tích nội dung dữ liệu trả về nhằm đảm bảo các trường dữ liệu và kiểu dữ liệu phù hợp với thiết kế cơ sở dữ liệu và mô hình dữ liệu phía Flutter. Việc này giúp hạn chế lỗi phát sinh khi ứng dụng thực hiện quá trình phân tích dữ liệu.

♦ Tự động hóa kiểm thử bằng Test Scripts

Bên cạnh kiểm thử thủ công, Postman hỗ trợ viết các đoạn mã kiểm thử bằng JavaScript nhằm tự động hóa quá trình xác minh API:

- Tự động kiểm tra sự tồn tại của các trường dữ liệu bắt buộc trong phản hồi.
- Xác nhận điều kiện logic như mã trạng thái hợp lệ hoặc dữ liệu trả về đúng định dạng.
- Giảm thiểu sai sót do thao tác thủ công và hỗ trợ kiểm thử lại toàn bộ hệ thống API sau mỗi lần cập nhật mã nguồn Backend.

Cách tiếp cận này góp phần xây dựng quy trình kiểm thử có tính lặp lại và ổn định trong vòng đời phát triển phần mềm.

♦ Quản lý danh mục API bằng Collections và Variables

- Toàn bộ API của hệ thống được tổ chức thành các Collections theo từng nhóm chức năng như: Xác thực người dùng, Quản lý sản phẩm và Giỏ hàng.
- Việc sử dụng biến môi trường (Environment Variables) như `base_url` hoặc `access_token` mang lại các lợi ích:
- Tránh việc gán cứng địa chỉ máy chủ trong từng yêu cầu API.

- Dễ dàng chuyển đổi giữa môi trường phát triển nội bộ (Local) và môi trường triển khai thực tế (Production).

- Đảm bảo tính nhất quán trong quá trình kiểm thử và triển khai hệ thống.

- ♦ Hỗ trợ đối soát và tích hợp Frontend – Backend

Postman đóng vai trò trung gian trong quá trình tích hợp hệ thống:

- Các phản hồi mẫu từ Postman được sử dụng làm cơ sở xây dựng các lớp mô hình dữ liệu (Model) trong ứng dụng Flutter.

- Quá trình đối soát API giúp cô lập lỗi hiệu quả: nếu API hoạt động chính xác trên Postman nhưng xảy ra lỗi trên ứng dụng, nguyên nhân có thể được xác định thuộc phía Frontend.

- Nhờ đó, có thể tối ưu hóa quy trình sửa lỗi và rút ngắn thời gian tích hợp hệ thống.

Việc chuẩn hóa quy trình kiểm thử API bằng Postman ngay từ giai đoạn phát triển Backend giúp đảm bảo tính chính xác của luồng dữ liệu, nâng cao độ ổn định hệ thống và giảm thiểu rủi ro khi tích hợp với ứng dụng Flutter. Cách tiếp cận này góp phần xây dựng quy trình phát triển phần mềm có kiểm soát, hướng tới khả năng mở rộng và bảo trì lâu dài.

1.7.1.5. Công cụ Quản lý mã nguồn (Git và GitHub)



Hình 5. Công cụ quản lý mã nguồn Git và GitHub

Trong quá trình phát triển hệ thống AppFoodDelivery, việc quản lý phiên bản mã nguồn đóng vai trò quan trọng nhằm đảm bảo tính ổn định của hệ thống, theo dõi lịch sử thay đổi và hỗ trợ khôi phục khi xảy ra lỗi. Do đó, hệ thống đã sử dụng Git – hệ thống quản lý phiên bản phân tán kết hợp với GitHub – nền tảng lưu trữ mã nguồn trực tuyến.

Git cho phép theo dõi chi tiết mọi thay đổi của mã nguồn thông qua cơ chế commit. Mỗi lần cập nhật đều được lưu lại như một phiên bản độc lập kèm theo thông điệp mô tả rõ ràng, giúp ghi nhận tiến trình phát triển theo từng giai đoạn, dễ dàng khôi phục về trạng thái ổn định trước đó khi phát sinh lỗi, đồng thời tránh mất dữ liệu trong quá trình thử nghiệm hoặc chỉnh sửa chức năng mới.

Điểm mạnh của việc áp dụng Git và GitHub trong hệ thống:

- Khả năng truy vết và khôi phục cao: Mọi thay đổi đều được ghi nhận đầy đủ, giúp dễ dàng xác định thời điểm xuất hiện lỗi và rollback nhanh chóng.

- An toàn và sao lưu dữ liệu: GitHub cung cấp kho lưu trữ đám mây, đảm bảo mã nguồn không bị mất do sự cố phần cứng hoặc lỗi thao tác.
- Tính nhất quán và chuyên nghiệp: Hệ thống áp dụng quy ước commit rõ ràng với các tiền tố như feat, fix, refactor, chore, giúp lịch sử code trở nên dễ đọc và dễ quản lý.
- Dễ dàng mở rộng sau này: Khi hệ thống cần phát triển thêm hoặc chuyển sang làm việc nhóm, chỉ cần áp dụng branching model mà không cần thay đổi lớn cấu trúc hiện tại.

Điểm yếu và hạn chế khi sử dụng:

- Chi phí quản lý phát sinh: Vì hệ thống được thực hiện bởi một cá nhân và quy mô không lớn, việc commit thường xuyên đôi khi làm tăng thời gian quản lý so với cách làm truyền thống.
- Phụ thuộc vào kết nối mạng: Mọi thao tác push/pull đều yêu cầu kết nối internet, dẫn đến một số bất tiện khi làm việc ngoại tuyến.
- Khúc mắc ban đầu: Người thực hiện cần thời gian làm quen với các lệnh Git và quy ước commit, đặc biệt ở giai đoạn đầu của hệ thống.

Trong phạm vi hệ thống cá nhân, quá trình phát triển được thực hiện chủ yếu trên nhánh chính. Các chức năng được xây dựng và cập nhật thông qua các commit riêng biệt, đảm bảo mỗi thay đổi đều được ghi nhận rõ ràng và có thể truy vết. Việc commit được thực hiện thường xuyên theo từng chức năng nhỏ giúp dễ dàng kiểm soát tiến độ phát triển, phát hiện lỗi sớm và khôi phục dữ liệu khi cần thiết.

Tổng thể, dù vẫn tồn tại một số hạn chế, việc áp dụng Git kết hợp GitHub đã mang lại giá trị lớn cho hệ thống. Công cụ này không chỉ giúp quản lý mã nguồn một cách khoa học mà còn chuẩn hóa quy trình phát triển theo hướng chuyên nghiệp, giảm thiểu rủi ro và tạo nền tảng vững chắc để dễ dàng bảo trì, cập nhật và mở rộng hệ thống trong tương lai.

1.7.2. Công nghệ phát triển phần mềm

1.7.2.1. Flutter SDK và Dart.



Hình 6. Các nền tảng phổ biến với Flutter

Flutter là framework phát triển ứng dụng di động mã nguồn mở do Google phát triển, sử dụng ngôn ngữ lập trình Dart^{[8], [15]}. Đây là công nghệ cốt lõi được lựa chọn để xây dựng lớp giao diện người dùng (Frontend) cho toàn bộ hệ thống AppFoodDelivery, bao gồm ba ứng dụng riêng biệt: Ứng dụng cho khách hàng, ứng dụng cho nhà hàng, ứng dụng cho tài xế.

Flutter hoạt động dựa trên kiến trúc nhiều lớp rõ ràng. Framework Layer cung cấp hệ thống Widget phong phú, animation, gesture detection và các API cấp cao. Engine Layer sử dụng thư viện đồ họa Skia để render giao diện trực tiếp lên canvas, không phụ thuộc vào thành phần UI mặc định của hệ điều hành. Embedder Layer chịu trách nhiệm kết nối Flutter Engine với nền tảng Android và iOS, xử lý input, lifecycle ứng dụng và tích hợp các dịch vụ hệ thống. Nhờ kiến trúc này, Flutter mang lại giao diện nhất quán tuyệt đối trên mọi thiết bị, hiệu năng render cao và khả năng kiểm soát UI toàn diện.

Flutter được lựa chọn vì đáp ứng xuất sắc các yêu cầu chính của đề tài. Framework này cho phép phát triển đa nền tảng với chỉ một codebase duy nhất nhưng vẫn biên dịch ra ứng dụng native cho cả iOS và Android. Cơ chế Hot Reload giúp xem kết quả thay đổi ngay lập tức mà không cần rebuild, từ đó nâng cao đáng kể năng suất phát triển. Rendering engine Skia riêng biệt mang lại hiệu năng cao và animation mượt mà, vượt trội hơn so với nhiều framework khác.

So sánh với các công nghệ khác, Flutter có nhiều lợi thế rõ rệt. So với React Native, Flutter không phải đi qua Bridge nên hiệu năng cao hơn, giao diện nhất quán hơn và ít gặp vấn đề về tính tương thích giữa hai nền tảng. So với phát triển native riêng biệt (Swift cho iOS và Kotlin cho Android), Flutter giúp giảm mạnh thời gian và chi phí phát triển, đồng thời dễ bảo trì hơn vì chỉ cần duy trì một codebase. Tuy nhiên, Flutter cũng có điểm yếu là kích thước ứng dụng thường lớn hơn native thuần và một số tính năng yêu cầu truy cập sâu vào hệ điều hành vẫn cần sử dụng Platform Channel.

Bảng 3. Bảng tiêu chí so sánh các công nghệ

Tiêu chí	Flutter	React Native	Swift/Kotlin
Cách tiếp cận	Một codebase cho iOS & Android	Một codebase (JavaScript)	Hai codebase riêng biệt
Hiệu năng	Cao (render trực tiếp bằng Skia)	Trung bình (qua Bridge)	Rất cao
Giao diện	Nhất quán cao, đẹp và mượt	Có sự khác biệt giữa nền tảng	Tốt nhất, tuân thủ hoàn toàn native
Chi phí phát triển	Thấp	Trung bình	Cao

Bảo trì	Dễ (chỉ 1 codebase)	Trung bình	Khó (phải bảo trì 2 codebase)
Kích thước ứng dụng	Trung bình đến lớn	Trung bình	Nhỏ nhất
Tùy chỉnh sâu	Cần Platform Channel	Cần Native Module	Dễ dàng nhất

Trong hệ thống AppFoodDelivery, Flutter đóng vai trò then chốt trong việc xây dựng toàn bộ tầng client. Với Customer App, Flutter xử lý tìm kiếm nhà hàng theo vị trí thời gian thực, hiển thị menu chi tiết với các tùy chọn size và topping qua BottomSheet, giỏ hàng động tự động tính phí giao hàng dựa trên khoảng cách, và màn hình theo dõi đơn hàng realtime. Trong Restaurant App, Flutter hỗ trợ quản lý thực đơn, bật/tắt tình trạng món ăn, xử lý luồng trạng thái đơn hàng (Pending → Confirmed → Preparing → Delivering) và cập nhật realtime. Với Driver App, Flutter cung cấp giao diện nhận đơn hàng gần vị trí, bản đồ dẫn đường, cập nhật vị trí tài xế thời gian thực và xác nhận giao hàng thành công.

Hệ thống sử dụng Provider làm giải pháp quản lý trạng thái chính, giúp tách biệt rõ ràng giữa giao diện và logic kinh doanh, quản lý tập trung các dữ liệu như giỏ hàng, thông tin người dùng và trạng thái đơn hàng, đồng thời tự động rebuild chỉ những widget cần thiết khi dữ liệu thay đổi. Việc giao tiếp với Backend được thực hiện thông qua RESTful API bằng thư viện dio, kết hợp async/await và interceptor để tự động gắn token xác thực, xử lý lỗi tập trung và ghi log request/response trong quá trình phát triển.

Flutter còn hỗ trợ xuất sắc các tính năng thời gian thực và định vị. Thông qua Supabase Realtime, ứng dụng có thể nhận cập nhật ngay lập tức về trạng

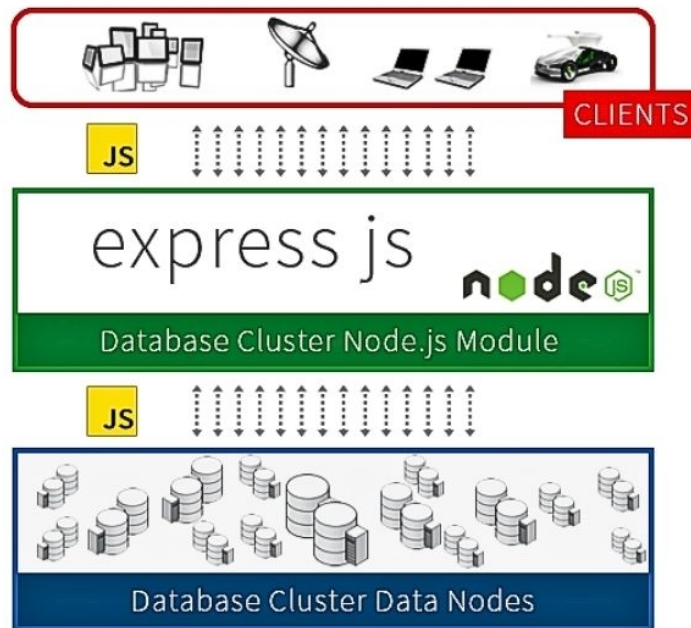
thái đơn hàng và vị trí tài xế mà không cần polling liên tục. Bản đồ được tích hợp bằng package flutter_map kết hợp OpenStreetMap để hiển thị vị trí người dùng, nhà hàng và tài xế, đồng thời tính toán khoảng cách giao hàng^[25]. Package geolocator được sử dụng để lấy tọa độ GPS chính xác với quyền truy cập phù hợp.

Để đảm bảo hiệu năng tối ưu, hệ thống áp dụng nhiều kỹ thuật như sử dụng const widget, ListView.builder cho lazy loading, hạn chế rebuild widget không cần thiết thông qua Provider, và tận dụng khả năng render trực tiếp của Skia, giúp animation mượt mà và ứng dụng phản hồi nhanh ngay cả trên thiết bị trung cấp.

Ưu điểm lớn nhất của Flutter trong hệ thống là tốc độ phát triển cao nhờ Hot Reload, giao diện đẹp mắt và nhất quán trên cả iOS và Android, hiệu năng gần với ứng dụng native, hệ sinh thái package phong phú và chi phí bảo trì thấp. Tuy nhiên, công nghệ này cũng tồn tại một số hạn chế như kích thước ứng dụng tương đối lớn, một số tính năng native sâu đòi hỏi Platform Channel, sự phụ thuộc vào chất lượng của package bên thứ ba, và cần thời gian làm quen với Dart cùng tư duy lập trình declarative (Widget tree).

Tóm lại, việc lựa chọn Flutter SDK kết hợp với ngôn ngữ Dart là quyết định phù hợp và hiệu quả nhất cho hệ thống AppFoodDelivery. Công nghệ này không chỉ giúp hoàn thành nhanh chóng ba ứng dụng di động với chất lượng cao, giao diện chuyên nghiệp và hiệu năng ổn định, mà còn đảm bảo tính nhất quán, khả năng mở rộng dễ dàng và chi phí bảo trì thấp trong tương lai.

1.7.1.2. Backend (Node.js và Express.js)



Hình 7. Kiến trúc Backend sử dụng Node.js và Express.js

Node.js là môi trường runtime mã nguồn mở dựa trên engine JavaScript V8 của Google Chrome, cho phép chạy JavaScript trên phía server^[18]. Express.js là một framework web nhẹ, linh hoạt và tối giản được xây dựng trên nền tảng Node.js^[19]. Sự kết hợp giữa Node.js và Express.js được sử dụng làm nền tảng chính để xây dựng phần Backend cho hệ thống AppFoodDelivery.

Node.js nổi bật với mô hình lập trình non-blocking I/O (không chặn luồng) và kiến trúc event-driven (dựa trên sự kiện). Thay vì chờ một tác vụ hoàn thành trước khi xử lý tác vụ tiếp theo, Node.js sử dụng một event loop duy nhất để xử lý đồng thời rất nhiều yêu cầu từ client. Điều này mang lại khả năng xử lý concurrency cao, rất phù hợp với các ứng dụng cần xử lý nhiều kết nối thời gian thực như hệ thống giao đồ ăn trực tuyến (cập nhật vị trí tài xế, trạng thái đơn hàng, thông báo đẩy...).

Express.js đóng vai trò là lớp framework mỏng trên Node.js, cung cấp hệ thống routing rõ ràng, middleware để xử lý request và response, cùng cơ chế dễ dàng mở rộng. Việc sử dụng cú pháp JavaScript/ESM (ECMAScript

Modules) hiện đại giúp code trở nên sạch sẽ, dễ import/export và phù hợp với tiêu chuẩn lập trình mới.

♦ Lý do lựa chọn Node.js + Express.js trong hệ thống:

Công nghệ này được lựa chọn vì đáp ứng tốt các yêu cầu chính của đề tài: hiệu năng cao khi xử lý nhiều request đồng thời, tốc độ phát triển nhanh, dễ dàng tích hợp với Supabase và Prisma ORM, cũng như khả năng mở rộng sau này. So với các framework nặng hơn như Spring Boot (Java) hay Django (Python), Node.js + Express.js có lợi thế về dung lượng nhẹ, tốc độ phát triển và chi phí học tập thấp. Hơn nữa, việc sử dụng JavaScript ở cả phía Backend và có thể kết nối logic với Flutter (dù Flutter dùng Dart) giúp đội ngũ dễ dàng hiểu tổng thể hệ thống.

♦ Vai trò của Node.js + Express.js trong hệ thống

Backend được xây dựng bằng Node.js và Express.js đóng vai trò là trung tâm xử lý của toàn bộ hệ thống. Cụ thể, nó chịu trách nhiệm:

- Xác thực và phân quyền người dùng thông qua việc kiểm tra token từ Firebase Auth.
- Xử lý toàn bộ logic kinh doanh liên quan đến quản lý món ăn, danh mục, topping, tùy chọn và khuyến mãi.
- Quản lý quy trình đặt hàng phức tạp, bao gồm tạo đơn hàng, tính toán tổng tiền, phí giao hàng dựa trên khoảng cách và áp dụng voucher.
- Quản lý và cập nhật trạng thái đơn hàng theo các giai đoạn (Pending → Confirmed → Preparing → Delivering → Completed).
- Xử lý các truy vấn vị trí, tính khoảng cách giữa nhà hàng, tài xế và khách hàng.
- Cung cấp dữ liệu cho các báo cáo doanh thu cơ bản của nhà hàng.
- Làm trung gian giao tiếp giữa ba ứng dụng Flutter và cơ sở dữ liệu Supabase.

Toàn bộ logic được tổ chức theo mô hình MVC (Model - View - Controller), trong đó Prisma ORM đóng vai trò Model, các Route của Express.js định nghĩa endpoint, và Controller xử lý business logic chính.

♦ Điểm mạnh nổi bật:

- Tốc độ phát triển nhanh nhờ cú pháp JavaScript đơn giản và hệ sinh thái npm cực kỳ phong phú.
- Khả năng xử lý concurrency cao, phù hợp với yêu cầu realtime của ứng dụng giao đồ ăn.
- Dễ dàng tích hợp với các dịch vụ bên thứ ba như Supabase Realtime và Firebase.
- Code nhẹ, dễ đọc và dễ bảo trì khi áp dụng cấu trúc rõ ràng.
- Hỗ trợ JavaScript/ESM hiện đại, giúp code trở nên sạch sẽ và dễ quản lý module.

♦ Hạn chế:

- JavaScript là ngôn ngữ động nên dễ xảy ra lỗi runtime nếu không có cơ chế kiểm tra nghiêm ngặt (hệ thống hiện tại chưa áp dụng TypeScript).
- Quản lý lỗi và cấu trúc code có thể trở nên phức tạp nếu không tổ chức tốt từ đầu.
- Hiệu năng kém hơn so với các ngôn ngữ compiled (như Go hoặc Java) khi phải xử lý các tác vụ tính toán nặng hoặc logic phức tạp.

1.7.2.3. Lớp dữ liệu Prisma ORM và PostgreSQL (Supabase)



Hình 8. Prisma ORM và cơ sở dữ liệu PostgreSQL

Prisma ORM là công cụ giúp ứng dụng Node.js tương tác với cơ sở dữ liệu thông qua mã JavaScript thay vì viết trực tiếp các câu lệnh SQL. Trong hệ thống AppFoodDelivery, Prisma đóng vai trò là lớp trung gian giữa backend Express.js và cơ sở dữ liệu PostgreSQL được triển khai trên Supabase.

Quy trình làm việc với Prisma gồm ba bước chính:

- Định nghĩa cấu trúc toàn bộ cơ sở dữ liệu trong file `schema.prisma`, bao gồm các bảng, kiểu dữ liệu, khóa ngoại và mối quan hệ giữa các thực thể.
- Prisma tự động chuyển schema này thành các câu lệnh SQL và áp dụng lên Supabase thông qua lệnh `migrate`.
- Prisma sinh ra một client JavaScript có đầy đủ thông tin về cấu trúc dữ liệu, giúp phát hiện lỗi kiểu ngay tại thời điểm viết code thay vì chờ đến lúc chạy chương trình.

Ứng dụng trong hệ thống:

- Khi khách hàng đặt đơn hàng, hệ thống cần tạo đồng thời một bản ghi trong bảng `orders` và nhiều bản ghi trong bảng `order_items`. Prisma cho phép thực hiện cả hai thao tác trong cùng một transaction, đảm bảo tính toàn vẹn dữ liệu

nếu một bước thất bại, toàn bộ sẽ được hủy và không để lại dữ liệu không nhất quán.

- Tính năng include cho phép lấy dữ liệu từ nhiều bảng liên quan chỉ trong một lần truy vấn duy nhất, tránh tình trạng gọi database lặp lại nhiều lần gây ảnh hưởng đến hiệu năng ứng dụng.

- Prisma Studio được sử dụng như công cụ hỗ trợ debug, cho phép xem và chỉnh sửa dữ liệu trực tiếp qua giao diện web mà không cần cài đặt thêm công cụ quản trị riêng.

Hạn chế:

- Với một số truy vấn thống kê phức tạp chẳng hạn tổng hợp doanh thu theo nhiều điều kiện lọc Prisma không hỗ trợ đủ linh hoạt. Trong những trường hợp đó, hệ thống sử dụng \$queryRaw để thực thi SQL trực tiếp, đảm bảo đáp ứng được yêu cầu nghiệp vụ mà không bị giới hạn bởi API của Prisma.

1.7.2.4. Các dịch vụ hỗ trợ và Tích hợp bên thứ ba

Để tối ưu hóa quy trình phát triển và đảm bảo tính ổn định cho các tính năng đặc thù của một ứng dụng giao đồ ăn, hệ thống đã lựa chọn tích hợp các dịch vụ hàng đầu trong lĩnh vực xác thực và hệ thống tin địa lý.

◆ Hệ thống quản lý định danh và Bảo mật: Firebase Authentication

Trong kiến trúc phần mềm hiện đại, việc tách biệt lớp xác thực khỏi cơ sở dữ liệu nghiệp vụ là một chiến lược quan trọng để tăng cường bảo mật. Hệ thống sử dụng Firebase Authentication làm trung tâm quản lý định danh.

Cơ chế xác thực đa phương thức:

- Email/Password: Cung cấp luồng đăng ký/đăng nhập truyền thống với các tiêu chuẩn mã hóa mật khẩu an toàn, tích hợp sẵn tính năng xác thực email để loại bỏ tài khoản ảo.

- Google Sign-In: Tận dụng hệ sinh thái Google để rút ngắn quy trình đăng ký, giúp tăng trải nghiệm người dùng (UX) và giảm tỷ lệ bỏ cuộc ở bước tạo tài khoản.
- Xác thực đa yếu tố (Multi-Factor Authentication - MFA): Để bảo vệ các tác vụ nhạy cảm như rút tiền từ ví tài xế hoặc thay đổi thông tin nhà hàng, hệ thống triển khai lớp bảo mật thứ hai qua mã OTP. Điều này đảm bảo tính toàn vẹn của tài khoản ngay cả khi mật khẩu bị lộ.
- Kiến trúc liên kết dữ liệu Hybrid: Hệ thống triển khai mô hình lưu trữ song song: thông tin đăng nhập được quản lý bởi Google, trong khi thông tin nghiệp vụ (Lịch sử đơn hàng, vai trò người dùng) được lưu trữ tại PostgreSQL. Hai hệ thống này được liên kết chặt chẽ thông qua Firebase UID – đóng vai trò là khóa ngoại (Foreign Key) xuyên suốt các bảng dữ liệu trong Prisma.

◆ Hệ thống tin địa lý (GIS) và Định vị thời gian thực

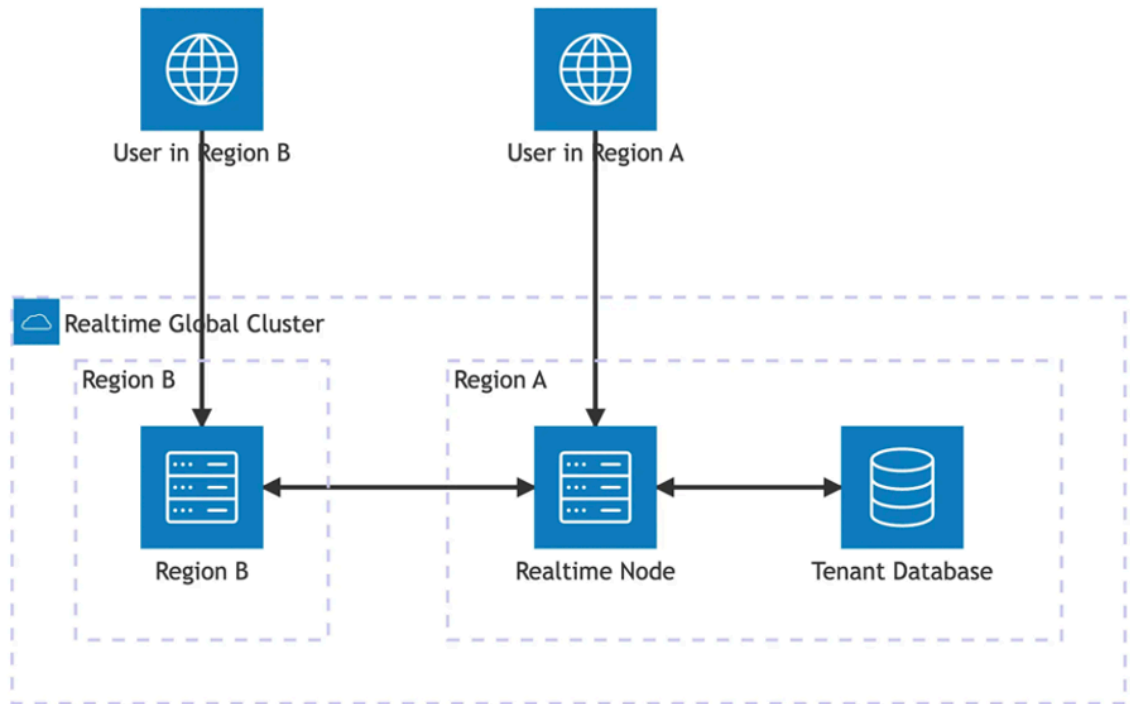


Hình 9. Tích hợp OpenStreetMap và định vị GPS trong ứng dụng Flutter

Hệ thống bản đồ là thành phần quyết định đến khả năng vận hành của ứng dụng giao đồ ăn, từ việc tìm kiếm nhà hàng đến theo dõi hành trình tài xế.

- Nền tảng bản đồ mã nguồn mở OpenStreetMap: Thay vì sử dụng các dịch vụ trả phí theo lượt truy cập, hệ thống tích hợp OSM thông qua thư viện flutter_map. Giải pháp này mang lại:

- + Tính kinh tế: Hoàn toàn miễn phí, không giới hạn số lượng request API, phù hợp với quy mô đồ án và khởi nghiệp.
 - + Khả năng tùy biến: Dễ dàng can thiệp vào các lớp bản đồ, hỗ trợ hiển thị các điểm tùy chỉnh như vị trí nhà hàng, shipper và khách hàng một cách trực quan.
 - Xử lý định vị với Geolocator: Ứng dụng tương tác với phần cứng GPS của thiết bị để lấy tọa độ thực tế với độ chính xác cao. Hệ thống xử lý thông minh các trạng thái quyền truy cập và hoạt động ổn định ở cả chế độ chạy ngầm để theo dõi đơn hàng liên tục.
- Cơ chế cập nhật thời gian thực với Supabase Realtime: Để tối ưu hóa trải nghiệm theo dõi đơn hàng, hệ thống sử dụng tính năng realtime của Supabase. Thay vì ứng dụng phải liên tục gửi yêu cầu lên server để kiểm tra trạng thái Supabase Realtime sẽ chủ động đẩy dữ liệu mới từ PostgreSQL xuống ứng dụng Flutter ngay khi có sự thay đổi trong bảng Orders hoặc Driver_Location. Điều này giúp:
- Cập nhật trạng thái đơn hàng: Khách hàng nhận thông báo tức thì khi nhà hàng xác nhận hoặc shipper đã lấy hàng.
 - Tiết kiệm tài nguyên: Giảm thiểu băng thông và dung lượng pin cho thiết bị di động do không phải duy trì các truy vấn lặp lại liên tục.



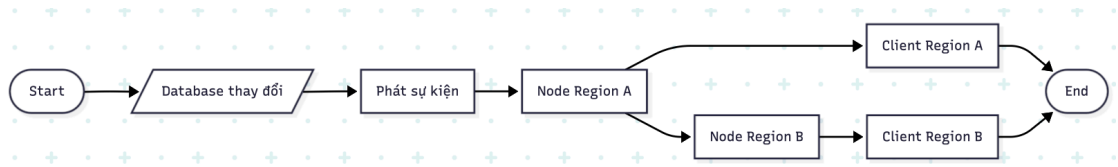
Hình 10. Kiến trúc của Supabase Realtime

Kiến trúc này không chỉ đơn thuần là kết nối giữa App và Database, mà là một hệ thống Cluster (Cụm máy chủ) đa vùng, giúp tối ưu hóa tốc độ truyền tải dữ liệu.

Thành phần chính trong sơ đồ:

- User in Region A/B: Đại diện cho người dùng ứng dụng ở các khu vực địa lý khác nhau (ví dụ: Region A là Singapore, Region B là Mỹ).
- Realtime Node: Các máy chủ trung gian (được viết bằng Elixir/Phoenix) chịu trách nhiệm duy trì kết nối WebSocket với người dùng.
- Cơ sở dữ liệu biệt lập : Cơ sở dữ liệu PostgreSQL duy nhất của hệ thống (nơi lưu trữ dữ liệu AppFoodDelivery).
- Realtime Global Cluster: Một mạng lưới các node kết nối với nhau, cho phép dữ liệu đồng bộ xuyên biên giới.

Luồng vận hành của hệ thống :



Hình 11. Luồng vận hành của hệ thống

Quy trình truyền tin diễn ra theo các bước:

- Gửi yêu cầu: Khi có một sự thay đổi dữ liệu tại cơ sở dữ liệu biệt lập (ví dụ: Tài xế cập nhật vị trí), Database sẽ phát ra một sự kiện.
- Phát tín hiệu đến Node gần nhất: Node tại Region A (nơi đặt Database) sẽ nhận tín hiệu đầu tiên.
- Đồng bộ xuyên vùng: Nhờ kiến trúc Cluster, Realtime Node ở Region A sẽ ngay lập tức chuyển tiếp sự kiện đó sang Region B thông qua đường truyền nội bộ của Supabase.
- Phân phối tới Client:
 - + Người dùng ở Region A nhận dữ liệu từ Node A.
 - + Người dùng ở Region B nhận dữ liệu từ Node B.

=> Cả hai đều nhận được thông tin gần như cùng lúc mà không bị ảnh hưởng bởi khoảng cách địa lý xa xôi với máy chủ chứa Database.

Bảng 4. Bảng tóm tắt các dịch vụ tích hợp

Thành phần	Giải pháp tích hợp	Vai trò
Identity Provider	Firebase Auth	Quản lý User Session, JWT, và bảo mật MFA.
Map Engine	OpenStreetMap	Hiển thị bản đồ, quản lý Tile Layer và tối ưu chi phí.

Location API	Geolocator	Lấy tọa độ GPS thời gian thực, xử lý quyền vị trí.
Cloud Storage	Supabase Storage	Lưu trữ hình ảnh món ăn, nhà hàng và ảnh đại diện.
Realtime Service	Supabase Realtime	Cập nhật trạng thái đơn hàng và vị trí shipper tức thì.

CHƯƠNG II. PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU ỨNG DỤNG

2.1. Cơ sở lý thuyết phân tích và đặc tả yêu cầu ứng dụng

Hệ thống AppFoodDelivery phục vụ đồng thời ba nhóm người dùng với vai trò và luồng thao tác khác nhau — khách hàng, nhà hàng và tài xế. Mỗi hành động của một bên đều kéo theo phản ứng từ hai bên còn lại theo thời gian thực. Đặc điểm này đòi hỏi yêu cầu hệ thống phải được phân tích và mô tả rõ ràng ngay từ đầu, tránh tình trạng hiểu sai luồng nghiệp vụ dẫn đến sai sót khó sửa khi đã vào giai đoạn lập trình.

Áp dụng phương pháp phân tích và thiết kế hướng đối tượng để mô hình hóa hệ thống thông qua các lớp đối tượng và mối quan hệ giữa chúng. Phương pháp này phù hợp với hệ thống có nhiều thực thể tương tác phức tạp như Order, User, Restaurant, Shipper, giúp làm rõ ranh giới trách nhiệm của từng thành phần trước khi viết code.

Để trực quan hóa kết quả phân tích, ngôn ngữ mô hình hóa UML được sử dụng với các loại biểu đồ sau:

- Biểu đồ Use Case xác định các chức năng hệ thống cần xây dựng từ góc nhìn của từng tác nhân, làm cơ sở để đặc tả yêu cầu chức năng cho cả ba phân hệ.
- Biểu đồ Lớp (Class Diagram) mô tả cấu trúc dữ liệu của hệ thống — các lớp thực thể, thuộc tính và mối quan hệ — phục vụ trực tiếp cho thiết kế cơ sở dữ liệu.
- Biểu đồ Trình tự (Sequence Diagram) mô tả chi tiết luồng trao đổi dữ liệu giữa Frontend, Backend và cơ sở dữ liệu theo từng use case cụ thể.
- Biểu đồ Trạng thái (State Diagram) mô hình hóa vòng đời của đơn hàng qua các trạng thái từ lúc tạo đến khi hoàn thành hoặc hủy, giúp xác định rõ điều kiện chuyển trạng thái và các trường hợp ngoại lệ cần xử lý.

- Client: Ứng dụng di động đa nền tảng được phát triển bằng Flutter, chịu trách nhiệm xử lý giao diện người dùng (UI), trải nghiệm người dùng (UX) và các tương tác trực tiếp phía frontend.
- Server (Backend): Xây dựng bằng Node.js kết hợp với Express.js, đảm nhận cốt lõi logic nghiệp vụ, quản lý luồng dữ liệu và xử lý toàn bộ các API endpoints.
- Database: Sử dụng PostgreSQL – hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mạnh mẽ, hỗ trợ ACID transactions, đảm bảo tính toàn vẹn và nhất quán cao của dữ liệu.
- ORM: Prisma được lựa chọn làm lớp Object-Relational Mapping, cung cấp truy vấn type-safe, migration an toàn và nâng cao hiệu quả phát triển.

Ngoài ra, hệ thống còn áp dụng mạnh mẽ các công nghệ và thực hành hiện đại sau:

- Xây dựng giao tiếp chuẩn hóa qua RESTful API giữa Client và Server.
- Tích hợp Firebase Authentication kết hợp JWT Token để thực hiện xác thực và quản lý quyền cho người dùng, nhà hàng và shipper.
- Áp dụng công nghệ Real-time (WebSockets/Realtime Database thông qua Firebase) để đồng bộ tức thì trạng thái đơn hàng và theo dõi vị trí GPS của shipper với độ trễ mili-giây.
- Triển khai validation, error handling chặt chẽ cùng hệ thống logging và monitoring tại tầng backend.

Việc kết hợp chặt chẽ OOAD + UML cùng với kiến trúc công nghệ hiện đại giúp hệ thống không chỉ được thiết kế rõ ràng, logic mà còn tạo nền tảng vững chắc cho việc bảo trì, mở rộng quy mô, chịu tải lớn và dễ dàng nâng cấp các tính năng trong tương lai.

2.2. Mô tả bài toán

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ, nhu cầu sử dụng dịch vụ giao nhận thức ăn trực tuyến ngày càng gia tăng, đặc biệt tại các khu vực đô thị. Tuy nhiên, các giải pháp truyền thống hiện nay vẫn tồn tại nhiều hạn chế, điển hình là việc thiếu cơ chế đồng bộ dữ liệu hiệu quả giữa các bên tham gia. Điều này dẫn đến các vấn đề như sai lệch trạng thái đơn hàng, chậm trễ trong cập nhật thông tin, cũng như làm suy giảm trải nghiệm tổng thể của người dùng.

Bài toán đặt ra cho hệ thống là xây dựng một nền tảng trung gian tập trung, có khả năng kết nối và điều phối hoạt động giữa ba thực thể chính, bao gồm:

- Người tiêu dùng (Khách hàng): Thực hiện tìm kiếm, lựa chọn món ăn và đặt hàng.
- Cửa hàng cung cấp dịch vụ (Nhà hàng): Tiếp nhận, xử lý và chuẩn bị đơn hàng.
- Đối tác vận chuyển (Shipper): Nhận đơn và thực hiện giao hàng đến khách hàng.

Hệ thống cần đảm bảo luồng thông tin được truyền tải một cách liên tục, chính xác và theo thời gian thực, từ đó giảm thiểu sai sót và tối ưu hiệu suất vận hành. Bên cạnh đó, hệ thống phải giải quyết đồng thời nhiều bài toán kỹ thuật và nghiệp vụ phức tạp, bao gồm:

- Cá nhân hóa trải nghiệm người dùng: Cho phép khách hàng tùy chỉnh món ăn theo nhu cầu (ví dụ: thêm topping, thay đổi khẩu phần), đồng thời lưu trữ và gợi ý dựa trên hành vi sử dụng.
- Quản lý vòng đời đơn hàng đa trạng thái: Theo dõi và cập nhật liên tục các trạng thái của đơn hàng như: xác nhận, đang chuẩn bị, đang giao, hoàn tất hoặc hủy, đảm bảo tính nhất quán giữa các bên liên quan.

- Tính toán chi phí vận chuyển động: Ứng dụng dữ liệu vị trí theo thời gian thực để xác định khoảng cách, từ đó tính toán phí giao hàng một cách tự động và chính xác.
- Đồng bộ dữ liệu thời gian thực : Đảm bảo mọi thay đổi (trạng thái đơn hàng, vị trí shipper,...) được cập nhật tức thời đến tất cả các bên, giảm thiểu độ trễ và sai lệch thông tin.
- Phân quyền và bảo mật dữ liệu: Áp dụng cơ chế phân quyền chặt chẽ ngay từ tầng cơ sở dữ liệu nhằm đảm bảo mỗi đối tượng chỉ có thể truy cập và thao tác trên dữ liệu được phép.

Bài toán không chỉ dừng lại ở việc xây dựng một ứng dụng đặt đồ ăn, mà còn là thiết kế một hệ thống phân tán có khả năng xử lý dữ liệu thời gian thực, đảm bảo tính nhất quán, hiệu năng cao và khả năng mở rộng trong tương lai.

2.3. Xác định tác nhân hệ thống

Thông qua quá trình khảo sát yêu cầu và thiết kế hệ thống, có thể xác định hệ thống bao gồm ba tác nhân chính tương tác trực tiếp với ứng dụng. Mỗi tác nhân được phân quyền rõ ràng dựa trên thuộc tính role trong cơ sở dữ liệu, nhằm đảm bảo tính bảo mật và kiểm soát truy cập theo từng chức năng cụ thể. Các tác nhân này đại diện cho các nhóm người dùng khác nhau trong hệ sinh thái của hệ thống, mỗi nhóm đảm nhận một vai trò và chức năng riêng biệt trong quy trình vận hành tổng thể.

2.3.1. Khách hàng

Khách hàng là người sử dụng cuối của hệ thống, có nhu cầu đặt món và sử dụng dịch vụ giao nhận thực phẩm. Đây là tác nhân có tần suất tương tác cao nhất với hệ thống. Các chức năng chính của khách hàng bao gồm:

- Tìm kiếm và duyệt danh sách món ăn, nhà hàng.
- Tùy chỉnh cấu hình món ăn (thêm topping, lựa chọn kích thước,...).
- Thêm sản phẩm vào giỏ hàng .

- Khởi tạo và thanh toán đơn hàng
- Theo dõi trạng thái đơn hàng và định vị Shipper theo thời gian thực.
- Lưu trữ danh sách món ăn yêu thích.
- Quản lý thông tin địa chỉ giao hàng cá nhân.

Tác nhân này đóng vai trò khởi tạo luồng nghiệp vụ chính của hệ thống.

2.3.2. Nhà hàng

Nhà hàng là đối tác cung cấp sản phẩm (món ăn) trên nền tảng, chịu trách nhiệm quản lý nội dung và xử lý tiếp nhận đơn đặt hàng. Các chức năng chính bao gồm:

- Quản lý thực đơn, bao gồm danh sách món ăn và danh mục các món ăn.
- Thiết lập các nhóm tùy chọn cho món ăn.
- Cập nhật trạng thái hoạt động của cửa hàng.
- Tiếp nhận, xử lý và cập nhật trạng thái các đơn hàng từ hệ thống.

Nhà hàng đóng vai trò trung gian xử lý đơn hàng giữa khách hàng và shipper.

2.3.3. Shipper

Shipper là đối tác vận chuyển, đảm nhiệm việc giao hàng từ nhà hàng đến người tiêu dùng. Đây là tác nhân chịu trách nhiệm trong giai đoạn cuối của quy trình. Các chức năng chính bao gồm:

- Quản lý thông tin phương tiện vận chuyển.
- Cập nhật trạng thái sẵn sàng hoạt động.
- Truyền tải liên tục vị trí địa lý theo thời gian thực.
- Nhận phân bổ đơn hàng thông minh từ hệ thống.
- Cập nhật các mốc trạng thái giao hàng (Đã lấy món, Đang giao, Hoàn thành).

2.4. Phân tích yêu cầu

Dựa trên các tác nhân đã xác định ở mục trước, hệ thống được đặc tả thông qua hai nhóm yêu cầu chính: yêu cầu chức năng và yêu cầu phi chức năng. Các yêu cầu này nhằm đảm bảo hệ thống đáp ứng đầy đủ nhu cầu nghiệp vụ, đồng thời duy trì hiệu năng, bảo mật và khả năng mở rộng.

2.4.1. Yêu cầu chức năng

2.4.1.1. Về phía khách hàng

Khách hàng là tác nhân chính khởi tạo các luồng nghiệp vụ, do đó hệ thống cần cung cấp các chức năng toàn diện, bao gồm:

- Quản lý tài khoản và địa chỉ: Hỗ trợ đăng ký và đăng nhập an toàn. Người dùng có thể lưu trữ và quản lý nhiều địa chỉ giao hàng khác nhau, cho phép chỉnh sửa, thêm mới hoặc xóa linh hoạt.
- Tìm kiếm và mua sắm: Cho phép tìm kiếm nhà hàng hoặc món ăn theo danh mục, từ khóa hoặc các tiêu chí lọc khác. Giao diện hiển thị danh sách món ăn trực quan, hỗ trợ người dùng dễ dàng lựa chọn.
- Tùy biến món ăn: Hệ thống cho phép cá nhân hóa món ăn thông qua các nhóm tùy chọn bao gồm cơ chế chọn một hoặc chọn nhiều. Ví dụ: lựa chọn kích cỡ, thêm topping hoặc thay đổi thành phần.
- Lưu trữ yêu thích: Cho phép lưu lại các nhà hàng hoặc món ăn yêu thích để truy cập nhanh cho những lần đặt hàng sau.
- Quản lý giỏ hàng và thanh toán: Người dùng có thể thêm sản phẩm vào giỏ hàng, chỉnh sửa số lượng và cấu hình. Hệ thống hiển thị minh bạch tổng giá trị món ăn và phí vận chuyển.
- Theo dõi đơn hàng thời gian thực: Cho phép người dùng theo dõi trạng thái đơn hàng (đã xác nhận, đang chuẩn bị, đang giao,...) và định vị vị trí của shipper trên bản đồ theo thời gian thực.

2.4.1.2. Về phía nhà hàng

Nhà hàng đóng vai trò quản lý nội dung và xử lý đơn hàng, với các chức năng chính:

- Quản lý thông tin và thực đơn: Cập nhật thông tin cửa hàng (tên, địa chỉ, vị trí GPS, thời gian hoạt động). Thực hiện CRUD đối với danh mục và món ăn
- Tùy chỉnh nhóm tùy chọn: Thiết lập các tùy chọn đi kèm với mức giá tương ứng cho từng món ăn.
- Xử lý đơn hàng: Nhận thông báo tự động khi có đơn hàng mới. Có quyền xác nhận hoặc từ chối đơn hàng tùy theo trạng thái tồn kho thực tế.
- Cập nhật tiến độ: Thay đổi trạng thái xử lý đơn (đang chuẩn bị, sẵn sàng giao,...).

2.4.1.3. Về phía shipper

Shipper đảm nhiệm khâu vận chuyển, với các chức năng chính:

- Quản lý trạng thái hoạt động: Cho phép bật/tắt chế độ nhận đơn để hệ thống điều phối và phân bổ đơn hàng phù hợp.
- Nhận đơn và điều hướng: Tiếp nhận thông tin đơn hàng, lấy lộ trình di chuyển và hỗ trợ điều hướng đến điểm lấy/giao hàng thông qua bản đồ.
- Cập nhật trạng thái giao hàng: Liên tục cập nhật các cột mốc trạng thái quan trọng như: Đã đến điểm lấy hàng, Đang giao hàng, Giao hàng thành công.
- Cập nhật vị trí thời gian thực: Truyền tải liên tục tọa độ GPS lên hệ thống để phục vụ việc theo dõi hành trình luân chuyển đơn hàng.

2.4.2. Yêu cầu phi chức năng

2.4.2.1. Giao diện người dùng

- Ứng dụng được phát triển bằng framework Flutter, đảm bảo tính nhất quán cao trên cả hai nền tảng iOS và Android.

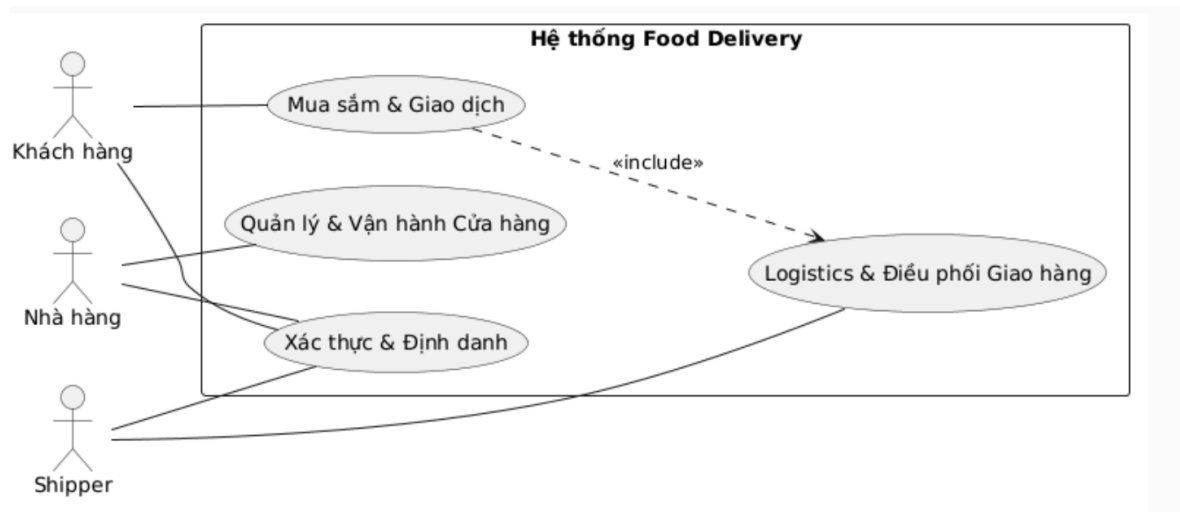
- Giao diện cần trực quan, dễ sử dụng, bám sát các tiêu chuẩn thiết kế hiện đại nhằm tối ưu trải nghiệm người dùng.
- Các thao tác điều hướng phải diễn ra mượt mà, hạn chế tối đa độ trễ.
- Hình ảnh món ăn và dữ liệu bản đồ cần được tải nhanh, tối ưu băng thông và thời gian phản hồi máy chủ.

2.4.2.2. Tính bảo mật

- Xác thực và phân quyền: Hệ thống sử dụng cơ chế xác thực dựa trên Token (JWT), đảm bảo phân quyền chặt chẽ để mỗi người dùng chỉ truy cập được vào tài nguyên được cấp phép.
- Toàn vẹn dữ liệu: Áp dụng cơ chế ràng buộc tham chiếu và Cascade trong cơ sở dữ liệu. Đảm bảo khi xóa một thực thể lỗi (ví dụ: tài khoản User), các dữ liệu liên quan cũng được dọn dẹp đồng bộ, tránh dư thừa và rác toàn vẹn dữ liệu.
- Hiệu năng hệ thống: Server backend cần đáp ứng khả năng xử lý nhiều kết nối đồng thời với tần suất cao.
- Tính khả dụng Realtime: Áp dụng giao tiếp hai chiều thông qua WebSocket hoặc cơ chế Realtime Database, đảm bảo mọi luồng dữ liệu (tọa độ, quá trình chuyển đổi trạng thái đơn hàng) đều được đồng bộ liên tục đến toàn network mà không gây nghẽn cho máy chủ.

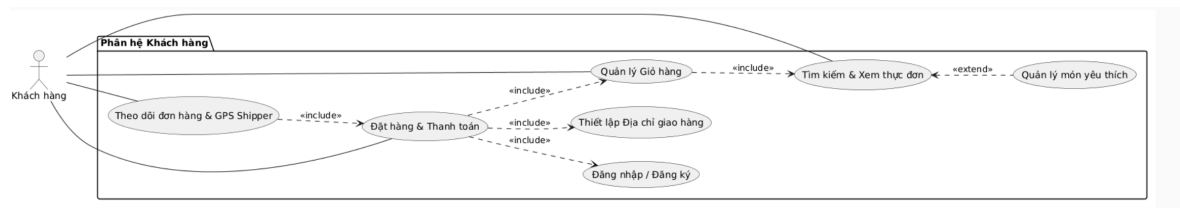
2.5. Mô hình hóa Use case

2.5.1. Mô hình Use case tổng quát



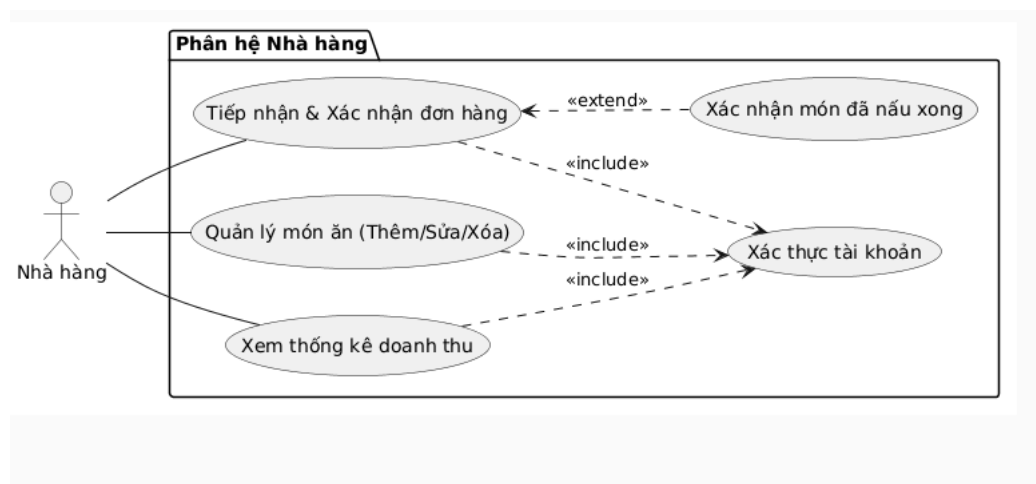
Hình 12. Mô hình use case tổng quát

2.5.2. Use case phía khách hàng



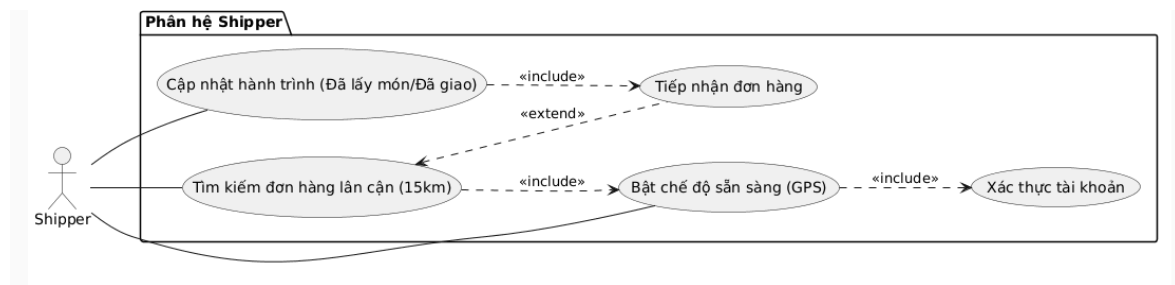
Hình 13. Mô hình use case phía khách hàng

2.5.3. Use case phía nhà hàng



Hình 14. Mô hình use case phía nhà hàng

2.5.4. Use case phía shipper



Hình 15. Mô hình use case phía shipper

2.6. Đặc tả Use case

2.6.1. Use case “Xác thực người dùng”

Bảng 5. Mô tả chi tiết use case đăng ký tài khoản người dùng

Tên use case	Đăng ký tài khoản người dùng
Mô tả	Use case này cho phép người sử dụng (Khách hàng, Nhà hàng, Shipper) tạo tài khoản mới trong hệ thống bằng Email và Mật khẩu tự thiết lập.
Người thực hiện	Khách hàng
Luồng sự kiện chính	- Use case bắt đầu khi người dùng chọn "Đăng ký", nhập đầy đủ thông tin: Tên hiển thị (name), Số điện thoại (phone), Email và Mật khẩu. - Người dùng click nút “Đăng ký”. - Hệ thống sẽ gửi Email và Mật khẩu lên máy chủ Firebase Authentication để kiểm tra định dạng và xét tính duy nhất của Email.

	4. Firebase Auth tạo tài khoản thành công và trả về một mã định danh duy nhất. 5. Ứng dụng ở Client tiếp tục lấy uid này cùng các thông tin cá nhân gõ lúc đầu để gửi một yêu cầu API sang Node.js Server. 6. Server Node.js chèn tự động một bản ghi mới vào bảng users trong cơ sở dữ liệu với các trường tương ứng. Use case kết thúc.
Luồng rẽ nhánh	- Tại bước 3, nếu Email đã tồn tại trong Firebase hoặc sai định dạng, hệ thống Firebase trả lỗi. UI hiển thị thông báo: "Email đã được sử dụng hoặc không hợp lệ". - Tại bước 5, nếu API phía Server bị gián đoạn thiết lập, tài khoản Firebase sẽ được tạo nhưng Database báo lỗi. Hiển thị "Lỗi đăng ký dữ liệu máy chủ". - Tại bước 1, nếu mật khẩu nhập quá yếu (dưới 6 ký tự), hệ thống báo "Mật khẩu quá yếu".
Các yêu cầu đặc biệt	- Mật khẩu phải lưu trữ trên Firebase, tuyệt đối máy chủ Node.js không lưu mật khẩu. - Trường uid là duy nhất bắt buộc ánh xạ giữa Firebase và PostgreSQL.
Tiền điều kiện	- Người dùng chưa từng đăng ký tài khoản với Email này.
Hậu điều kiện	- Tài khoản đăng ký thành công trên Firebase Authentication.

	- Bản ghi tài khoản được lưu trữ an toàn trong bảng users với phân quyền
Điểm mở rộng	- Hệ thống cho phép mở rộng tính năng đăng ký bằng google .

2.6.2. Use case “Đặt đơn hàng”

Bảng 6. Mô tả chi tiết use case đặt đơn hàng

Tên use case	Đặt đơn hàng
Mô tả	- Use case này cho phép Khách hàng tiến hành gửi yêu cầu đặt các món ăn đã chọn. Hệ thống sẽ tự động tính toán tổng hóa đơn bao gồm cả giá tùy chọn kèm theo (Topping) và Phí giao hàng dựa trên thuật toán tọa độ GPS thực tế.
Người thực hiện	Khách hàng
Luồng sự kiện chính	1. Use case bắt đầu khi Khách hàng ở màn hình Giỏ hàng và ấn nút "Đi đến Thanh toán". 2. Hệ thống trích xuất dữ liệu món ăn, các tùy chọn khách đã tích và tọa độ địa chỉ nhận hàng gửi lên Server thông qua API. 3. Server tiến hành tra cứu giá gốc từ bảng foods. Đồng thời sử dụng thuật toán Haversine để tính khoảng cách

	(dựa trên km) giữa tọa độ khách hàng và tọa độ nhà hàng. 4. Hệ thống gán phụ phí vận chuyển theo công thức tự động: 1 USD / 1 km (phí tối thiểu là 1\$). Hệ thống cộng dồn tất cả và xuất ra giá trị tổng hóa đơn . 5. Người dùng xem bảng chốt giá, chọn phương thức thanh toán và click nút “Xác nhận Đặt hàng”. 6. Hệ thống tạo lập một bản ghi giao dịch trên bảng orders với trạng thái ban đầu là PENDING, đồng thời lưu chi tiết các món vào bảng phụ order_items. Hệ thống kích hoạt cơ chế Realtime đẩy thông báo Pop-up đến ứng dụng của Nhà hàng đối tác. Use case kết thúc thành công.
Luồng rẽ nhánh	- Tại bước 2: Nếu Khách hàng chưa cấp quyền sử dụng định vị không gian API sẽ khước từ phục vụ. Giao diện báo lỗi yêu cầu "Thiết lập định vị bắt buộc". - Tại bước 3: Nếu có bất kỳ đối tượng Món ăn gốc nào do nhà hàng xóa đi và không còn tồn tại trong database, hệ thống sẽ cảnh báo không thể thanh toán món ăn đó. - Tại bước 6: Nếu có sự cố lỗi truy xuất cơ sở dữ liệu hệ thống hủy bỏ quá trình tạo document, trả về lỗi máy chủ và từ chối trừ tiền của người dùng.
Các yêu cầu đặc biệt	- Quá trình tính toán total_price tuyệt đối không được tin tưởng gửi từ Client (Mobile) lên, mà bắt buộc phải cho Server (Node.js API) tự query đối chiếu từng mức

	giá price của Option và Food một lần nữa để chống hack lệch giá. - Thuật toán Haversine đo khoảng cách cần được làm tròn (Round) đến 1 chữ số thập phân trước khi tính tiền.
Tiền điều kiện	- Người dùng đã Đăng nhập và trong giỏ hàng cần có tối thiểu 1 mặt hàng hợp lệ thuộc về một nhà hàng.
Hậu điều kiện	- Một bản ghi Đơn hàng mới chính thức được ghi vào CSDL hệ thống với trạng thái PENDING. - Giỏ hàng tạm thời sẽ được thực thi lệnh rỗng (xóa sạch). - Giao diện Khách hàng sẽ bị điều hướng sang màn hình lắng nghe lộ trình của Shipper.
Điểm mở rộng	Không có

2.6.3. Use case “Xác nhận đơn hàng”

Bảng 7. Mô tả chi tiết use case xác nhận đơn hàng

Tên use case	Xác nhận đơn hàng
Mô tả	Use case này cho phép tài khoản cấp bậc Đối tác (Nhà hàng) tiếp nhận yêu cầu đặt món đầy về từ Khách hàng, kiểm tra tính khả dụng của nguyên liệu và ra quyết định xác nhận nấu để chuyển đơn hàng sang luồng điều phối Shipper.
Người thực hiện	Nhà hàng

Luồng sự kiện chính	1. Giao diện của Nhà hàng duy trì kết nối lắng nghe dữ liệu thời gian thực từ máy chủ. 2. Hệ thống báo có đơn đặt hàng mới mang trạng thái PENDING. 3. Chủ nhà hàng bấm vào đơn hàng, giao diện hiển thị thông tin trích xuất: Danh sách món ăn, tùy chọn đi kèm và doanh thu dự kiến. 4. Nhà hàng click vào nút "Xác nhận nhận đơn". 5. Client request lên Server Node.js. 6. Server đọc token và đối chiếu với bảng restaurants để chứng thực quyền sở hữu của hàng. 7. Server cấp lệnh update database, thay đổi trường status của đơn hàng sang CONFIRMED. Màn hình Nhà hàng chuyển đơn vào tab "Đang nấu". Use case hoàn thành.
Luồng rẽ nhánh	- Tại bước 4: Nhà hàng nhận thấy hết nguyên liệu nên đã click nút "Từ chối đơn". Hệ thống gọi API nhánh rẽ để ép trạng của đơn hàng thành chuỗi CANCELLED. Lúc này giao diện ứng dụng của Khách hàng sẽ báo "Đơn bị hủy bởi cửa hàng". - Tại bước 6: Nếu Server không tìm thấy hồ sơ của cửa hàng được liên kết, hoặc tài khoản dùng Token giả, Server sẽ lập tức từ chối thao tác, ném ra mã HTTP-404. - Tại bất kỳ bước nào: Lỗi kết nối CSDL ném lỗi HTTP-500. Báo "Lỗi đường truyền hệ thống".

Các yêu cầu đặc biệt	- Hành động xác nhận thành CONFIRMED đóng vai trò cực kỳ quan trọng để khối API của Shipper có thể quét và phân phối đơn hàng cho tài xế lân cận. - Chỉ cập nhật duy nhất trường status. Mọi dữ liệu giá trị của đơn hàng phải được đóng băng hoàn toàn.
Tiền điều kiện	- Người thao tác phải có quyền RESTAURANT, cửa hàng đang trực tuyến và đang tồn tại một (hoặc nhiều) Đơn hàng ở trạng thái PENDING trở đến restaurant_id của họ.
Hậu điều kiện	- Trạng thái đơn hàng trong Database thay đổi thành CONFIRMED. Hệ thống tự động đẩy đơn hàng ra "Chợ đơn" để các Shipper chuẩn bị bắt cuộc.
Điểm mở rộng	Không có.

2.6.4. Use case “Giao hàng”

Bảng 8. Mô tả chi tiết use case giao hàng

Tên use case	Giao hàng
Mô tả	Use case này mô tả quy trình thực thi nghiệp vụ của Shipper: từ việc tìm kiếm đơn hàng khả dụng quanh vị trí hiện tại, tiếp nhận đơn, lấy hàng tại quán và giao đến tận tay khách hàng.
Người thực hiện	Shipper
Luồng sự kiện chính	1. Shipper bật chế độ làm việc. Hệ thống tự động gửi tọa độ GPS hiện tại lên Server qua API.

	2. Server sử dụng thuật toán Haversine để lọc danh sách các đơn hàng ở trạng thái CONFIRMED trong bán kính 15km so với Shipper. 3. Shipper xem danh sách, chọn một đơn hàng và nhấn "Nhận đơn". 4. Hệ thống gọi hàm acceptOrder, kiểm tra nếu đơn chưa có ai nhận, sẽ gán shipper_id của tài xế vào bảng orders. 5. Shipper di chuyển đến nhà hàng. Sau khi nhận túi hàng, Shipper nhấn "Đã lấy hàng". 6. Hệ thống gọi API, cập nhật trạng thái đơn sang DELIVERING. 7. Trong suốt quá trình di chuyển, ứng dụng Shipper liên tục gọi API để cập nhật tọa độ vào database. 8. Khi đến nơi và bàn giao cho khách, Shipper nhấn "Hoàn thành". 9. Hệ thống cập nhật trạng thái đơn thành COMPLETED. Use case kết thúc.
Luồng rẽ nhánh	- Tại bước 4: Nếu cùng lúc có shipper khác vừa nhấn nhận đơn trước, Server trả về lỗi 409 ("Đơn hàng này đã được nhận bởi shipper khác"). Hệ thống yêu cầu Shipper chọn đơn khác. - Tại bước 7: Nếu shipper di chuyển vào vùng mất sóng, dữ liệu vị trí không thể cập nhật. Client hiển thị thông báo "Đang kết nối lại...".

	- Tại bất kỳ bước nào: Nếu xảy ra lỗi hệ thống (500), Shipper không thể cập nhật trạng thái đơn hàng.
Các yêu cầu đặc biệt	- Xử lý tranh chấp: Phải kiểm tra điều kiện trước khi cho phép nhận đơn để tránh tình trạng một đơn có hai người giao. - Độ chính xác: Tọa độ GPS của shipper phải được cập nhật định kỳ (ví dụ mỗi 10-20 giây) để khách hàng có thể theo dõi vị trí chính xác trên bản đồ.
Tiền điều kiện	- Người dùng có role là SHIPPER, đơn hàng đang ở trạng thái CONFIRMED và chưa có ai tiếp nhận.
Hậu điều kiện	- Đơn hàng được giao thành công (Trạng thái COMPLETED). - Hệ thống ghi nhận hoàn tất lộ trình và sẵn s
Điểm mở rộng	Không có.

2.6.5. Use case “Quản lý tài khoản”

Bảng 9. Mô tả chi tiết use case quản lý tài khoản

Tên use case	Quản lý tài khoản và địa chỉ giao hàng
Mô tả	- Use case này cho phép người dùng cập nhật thông tin cá nhân, thay đổi mật khẩu và quản lý danh sách các địa chỉ giao hàng (thêm mới, chỉnh sửa, xóa và thiết lập địa chỉ mặc định).
Người thực hiện	Khách hàng

Luồng sự kiện chính	1. Người dùng truy cập vào mục "Hồ sơ cá nhân". 2. Cập nhật hồ sơ: Người dùng chỉnh sửa họ tên hoặc số điện thoại và nhấn Lưu. Hệ thống gọi API <code>updateUser</code> để cập nhật bảng <code>users</code>. 3. Quản lý địa chỉ: Người dùng chọn thêm địa chỉ mới. Hệ thống yêu cầu nhập chuỗi địa chỉ và tọa độ. 4. Thiết lập mặc định: Nếu là địa chỉ đầu tiên hoặc người dùng tích chọn "Mặc định", hệ thống sẽ tự động hủy trạng thái của các địa chỉ cũ và gán mặc định cho địa chỉ mới. 5. Thay đổi mật khẩu: Người dùng nhập mật khẩu hiện tại và mật khẩu mới. Hệ thống thực hiện tái xác thực qua Firebase và cập nhật mật khẩu mới. Use case kết thúc.
Luồng rẽ nhánh	- Xóa địa chỉ mặc định: Nếu người dùng xóa địa chỉ đang là mặc định, hệ thống sẽ tự động tìm địa chỉ được tạo gần nhất để thiết lập lại làm mặc định mới. - Lỗi cập nhật: Nếu email cập nhật trùng với người dùng khác, hệ thống trả về lỗi "Email already exists". - Lỗi mật khẩu: Nếu mật khẩu hiện tại không chính xác, Firebase trả lỗi "wrong-password", hệ thống hiển thị thông báo yêu cầu nhập lại.
Các yêu cầu đặc biệt	- Tính toàn vẹn: Khi xóa địa chỉ, phải đảm bảo không còn địa chỉ nào khác đang là mặc định trước khi chỉ định cái mới.

	- Bảo mật: Thao tác đổi mật khẩu bắt buộc phải thông qua dịch vụ FirebaseAuthService để đảm bảo tính an toàn cực cao.
Tiền điều kiện	- Người dùng đã đăng nhập thành công vào ứng dụng.
Hậu điều kiện	- Thông tin người dùng và danh sách địa chỉ được cập nhật đồng bộ trong cả Firebase Auth và PostgreSQL.
Điểm mở rộng	Không có.

2.6.6. Use case “Tìm kiếm và xem thực đơn”

Bảng 10. Bảng mô tả chi tiết use case tìm kiếm và xem thực đơn

Tên use case	Tìm kiếm và xem thực đơn
Mô tả	Cho phép khách hàng tìm kiếm món ăn hoặc nhà hàng theo từ khóa hoặc danh mục, từ đó xem được thông tin chi tiết về giá cả, hình ảnh, đánh giá và thực đơn của quán.
Người thực hiện	Khách hàng
Luồng sự kiện chính	1. Người dùng nhập từ khóa (tên món, tên quán) vào thanh tìm kiếm hoặc chọn một danh mục trên trang chủ. 2. Hệ thống gọi API truy vấn dữ liệu đồng thời từ các bảng food, restaurant và categories. 3. Hệ thống trả về danh sách kết quả bao gồm: Hình

	ảnh đại diện, tên, giá bán và đánh giá trung bình. 4. Người dùng nhấn vào một nhà hàng hoặc món ăn cụ thể để xem chi tiết. 5. Hệ thống hiển thị toàn bộ thực đơn của quán, các nhóm tùy chọn (Size, Topping) và thông tin liên hệ của nhà hàng.
Luồng rẽ nhánh	- Tại bước 2, nếu không tìm thấy dữ liệu phù hợp, hệ thống hiển thị thông báo: "Rất tiếc, không tìm thấy kết quả phù hợp với từ khóa của bạn" và gợi ý các món ăn phổ biến khác. - Nếu nhà hàng đang trong trạng thái đóng cửa (<code>is_open = false</code>), hệ thống sẽ hiển thị nhãn "Quán đã đóng cửa" trên hình ảnh.
Các yêu cầu đặc biệt	Tìm kiếm phải hỗ trợ cả tiếng Việt có dấu và không dấu. Tốc độ hiển thị hình ảnh phải được tối ưu để tránh giật lag.
Tiền điều kiện	Người dùng đã mở ứng dụng và điện thoại có kết nối mạng.
Hậu điều kiện	Người dùng nắm bắt được thông tin chi tiết để sẵn sàng cho hành vi "Thêm vào giỏ hàng".
Điểm mở rộng	Tích hợp bộ lọc nâng cao: Lọc theo khoảng cách gần nhất, Lọc theo giá từ thấp đến cao, Lọc theo đánh giá 5 sao.

2.6.7. Use case “Quản lý món yêu thích”

Bảng 11. Bảng mô tả chi tiết use case quản lý món yêu thích

Tên use case	Quản lý món yêu thích
Mô tả	Cho phép khách hàng lưu lại các món ăn ưu thích bằng cách nhấn vào biểu tượng "Trái tim" để dễ dàng tìm kiếm và đặt lại trong những lần truy cập sau.
Người thực hiện	Khách hàng
Luồng sự kiện chính	1. Người dùng xem danh sách món ăn hoặc chi tiết một món ăn cụ thể. 2. Người dùng nhấn vào biểu tượng "Trái tim" trên thẻ món ăn. 3. Hệ thống Client gửi yêu cầu API kèm theo mã món ăn và mã nhà hàng lên Server. 4. Hệ thống Server kiểm tra và thêm bản ghi mới vào bảng favorites trong cơ sở dữ liệu. 5. Hệ thống phản hồi thành công, biểu tượng "Trái tim" trên ứng dụng chuyển sang trạng thái đã chọn.
Luồng rẽ nhánh	- Tại bước 4, nếu món ăn đã tồn tại trong danh sách yêu thích, hệ thống sẽ thực hiện lệnh xóa bản ghi đó. - Nếu người dùng chưa đăng nhập, hệ thống sẽ hiển thị thông báo yêu cầu đăng nhập trước khi thực hiện hành động này.
Các yêu cầu đặc biệt	Việc cập nhật trạng thái "Yêu thích" phải diễn ra mượt mà, không làm tải lại toàn bộ trang.

Tiền điều kiện	Người dùng đã đăng nhập thành công vào ứng dụng.
Hậu điều kiện	Dữ liệu yêu thích được đồng bộ chính xác giữa máy chủ và thiết bị người dùng.
Điểm mở rộng	Gợi ý các món ăn tương tự dựa trên danh sách món yêu thích của người dùng.

2.6.8. Use case “Quản lý giỏ hàng”

Bảng 12. Bảng mô tả chi tiết use case quản lý giỏ hàng

Tên use case	Quản lý giỏ hàng
Mô tả	Cho phép khách hàng thêm các món ăn mong muốn vào danh sách chờ đơn, điều chỉnh số lượng hoặc loại bỏ món ăn trước khi tiến hành thanh toán chính thức.
Người thực hiện	Khách hàng
Luồng sự kiện chính	1. Người dùng chọn món ăn từ thực đơn và nhấn nút "Thêm vào giỏ hàng". 2. Hệ thống kiểm tra tính hợp lệ (kiểm tra xem món có thuộc cùng một nhà hàng đang có trong giỏ hay không). 3. Hệ thống tạo hoặc cập nhật bản ghi trong bảng <code>cart_items</code> của cơ sở dữ liệu PostgreSQL.

	4. Người dùng mở màn hình "Giỏ hàng" để kiểm tra danh sách món, số lượng và các tùy chọn đi kèm. 5. Người dùng thực hiện tăng/giảm số lượng hoặc xóa bỏ món ăn khỏi danh sách. 6. Hệ thống tự động tính toán lại tổng giá trị đơn hàng dựa trên đơn giá và số lượng món hiện tại.
Luồng rẽ nhánh	- Tại bước 2, nếu người dùng thêm món từ nhà hàng khác, hệ thống hiển thị thông báo hỏi: "Bạn có muốn xóa giỏ hàng hiện tại để bắt đầu đặt đơn từ nhà hàng mới này không?". - Nếu số lượng món giảm về 0, hệ thống tự động xóa món ăn đó khỏi giỏ hàng.
Các yêu cầu đặc biệt	Dữ liệu giỏ hàng phải được đồng bộ liên tục với Server để đảm bảo tính nhất quán khi người dùng chuyển đổi thiết bị hoặc đăng nhập lại.
Tiền điều kiện	Người dùng đã đăng nhập tài khoản thành công.
Hậu điều kiện	Danh sách món ăn trong giỏ hàng được cập nhật chính xác, sẵn sàng cho bước "Đặt hàng & Thanh toán".
Điểm mở rộng	Tự động gợi ý thêm các món nước uống kèm theo khi khách hàng đang xem giỏ hàng để tăng giá trị đơn hàng.

2.6.9. Use case “Theo dõi đơn hàng”

Bảng 13. Bảng mô tả chi tiết use case theo dõi đơn hàng

Tên use case	Theo dõi đơn hàng
--------------	-------------------

Mô tả	Cho phép khách hàng theo dõi tiến trình xử lý đơn hàng theo thời gian thực và xem vị trí di chuyển trực tiếp của Shipper trên bản đồ khi đơn hàng đang trong quá trình giao.
Người thực hiện	Khách hàng
Luồng sự kiện chính	1. Người dùng truy cập vào mục "Hoạt động" hoặc "Lịch sử đơn hàng" trên ứng dụng. 2. Người dùng nhấn chọn vào một đơn hàng đang được xử lý (Trạng thái: CONFIRMED hoặc DELIVERING). 3. Hệ thống hiển thị dòng thời gian trạng thái (Đã nhận đơn, Đang nấu, Đang giao). 4. Nếu đơn hàng đang giao, hệ thống hiển thị màn hình bản đồ trực tuyến. 5. Hệ thống liên tục truy vấn tọa độ shipper_lat và shipper_lng của tài xế từ máy chủ. 6. Hệ thống vẽ biểu tượng Shipper di chuyển trên bản đồ dựa trên tọa độ thực tế nhận được.
Luồng rẽ nhánh	- Nếu đơn hàng bị hủy: Hệ thống dừng hiển thị bản đồ và thông báo lý do hủy cho khách hàng. - Nếu thiết bị của Shipper mất kết nối GPS: Hệ thống hiển thị vị trí cuối cùng được ghi nhận và thông báo "Vị trí Shipper đang cập nhật".

Các yêu cầu đặc biệt	Tọa độ GPS phải được cập nhật định kỳ (mỗi 10 - 20 giây) để đảm bảo marker trên bản đồ di chuyển mượt mà.
Tiền điều kiện	Đơn hàng đã được nhà hàng xác nhận và đã được chỉ định Shipper giao hàng.
Hậu điều kiện	Khách hàng an tâm về lộ trình đơn hàng và biết được thời gian dự kiến nhận hàng.
Điểm mở rộng	Tích hợp tính năng gọi điện hoặc nhắn tin trực tiếp cho Shipper từ màn hình theo dõi nếu đơn hàng gặp sự cố chậm trễ.

2.6.10. Use case “Xác thực tài khoản nhà hàng”

Bảng 14. Bảng mô tả chi tiết use case xác thực tài khoản nhà hàng

Tên use case	Xác thực tài khoản nhà hàng
Mô tả	Cho phép chủ nhà hàng đăng nhập vào hệ thống dành riêng cho đối tác để quản lý thực đơn, tiếp nhận đơn hàng và theo dõi doanh thu.
Người thực hiện	Nhà hàng
Luồng sự kiện chính	1. Người dùng chọn chế độ "Đối tác nhà hàng" trên màn hình khởi động của ứng dụng. 2. Người dùng nhập Email và Mật khẩu đã đăng ký. 3. Hệ thống Client gửi thông tin xác thực lên Firebase Authentication để kiểm tra tính hợp lệ.

	4. Sau khi Firebase trả về mã UID thành công, Client gửi UID này sang Node.js Server. 5. Server kiểm tra trường role trong cơ sở dữ liệu. 6. Nếu vai trò là RESTAURANT, hệ thống cấp quyền truy cập vào giao diện Quản trị nhà hàng (Dashboard).
Luồng rẽ nhánh	- Tại bước 3, nếu thông tin đăng nhập sai, hệ thống hiển thị thông báo: "Email hoặc mật khẩu không chính xác". - Tại bước 6, nếu tài khoản đăng nhập thành công nhưng có role là CUSTOMER hoặc SHIPPER, hệ thống sẽ từ chối truy cập và báo lỗi: "Tài khoản không có quyền quản trị nhà hàng".
Các yêu cầu đặc biệt	Thông tin mật khẩu phải được mã hóa và truyền tải qua giao thức bảo mật (HTTPS). Hệ thống phải duy trì phiên đăng nhập (Token) để người dùng không phải đăng nhập lại nhiều lần.
Tiền điều kiện	Nhà hàng đã được tạo tài khoản và phân quyền RESTAURANT trong cơ sở dữ liệu.
Hậu điều kiện	Chủ nhà hàng truy cập được vào màn hình quản lý đơn hàng và món ăn.
Điểm mở rộng	Khôi phục mật khẩu thông qua mã OTP gửi về Email nếu người dùng nhấn vào "Quên mật khẩu".

2.6.10. Use case “Quản lý món ăn”

Bảng 15. Bảng mô tả chi tiết use case quản lý món ăn

Tên use case	Quản lý món ăn
Mô tả	Cho phép nhà hàng chủ động thêm mới món ăn, chỉnh sửa thông tin (tên, giá, ảnh) hoặc xóa bỏ các món không còn kinh doanh để cập nhật thực đơn cho khách hàng.
Người thực hiện	Nhà hàng.
Luồng sự kiện chính	1. Nhà hàng truy cập vào giao diện "Quản lý thực đơn". 2. Hệ thống tải và hiển thị danh sách toàn bộ món ăn hiện có từ bảng foods thuộc nhà hàng đó. 3. Nhà hàng chọn hành động: "Thêm món mới". 4. Nhà hàng nhập đầy đủ thông tin: Tên món, Mô tả, Giá tiền, Hình ảnh minh họa và chọn Danh mục món. 5. Nhà hàng xác nhận lưu thông tin. 6. Hệ thống thực hiện ghi dữ liệu mới vào bảng foods trong Database.
Luồng rẽ nhánh	- Chỉnh sửa: Tại bước 3, nếu chọn món có sẵn, nhà hàng thay đổi thông tin và nhấn "Cập nhật". Hệ thống thực hiện lệnh Update. - Cập nhật trạng thái: Nhà hàng có thể bật/tắt công tắc "Còn món" hoặc "Hết món" để tạm ẩn món ăn khỏi ứng dụng khách hàng mà không cần xóa.
Các yêu cầu đặc biệt	Việc thay đổi giá tiền phải đảm bảo tính nhất quán (không làm thay đổi giá của các đơn hàng đã đặt trong quá khứ).

Tiền điều kiện	Nhà hàng đã đăng nhập và được xác thực quyền quản trị cửa hàng.
Hậu điều kiện	Thông tin thực đơn mới được cập nhật và hiển thị ngay lập tức trên giao diện tìm kiếm của Khách hàng.
Điểm mở rộng	Thiết lập các tùy chọn bổ sung (Options Group) như: Thêm topping, chọn mức độ cay, chọn size.

2.6.11. Use case “Xác nhận món đã nấu xong”

Bảng 16. Bảng mô tả chi tiết use case xác nhận món đã nấu xong

Tên use case	Xác nhận món đã nấu xong
Mô tả	Nhà hàng thông báo cho hệ thống rằng các món ăn trong đơn đã được chuẩn bị xong, sẵn sàng để Shipper đến lấy và thực hiện quá trình giao hàng.
Người thực hiện	Nhà hàng
Luồng sự kiện chính	1. Nhà hàng truy cập danh sách đơn hàng đang thực hiện (Trạng thái: CONFIRMED). 2. Nhà hàng hoàn tất việc chế biến món ăn theo phiếu điều phối. 3. Nhà hàng nhấn nút "Hoàn thành nấu" (Ready for Pickup) trên ứng dụng. 4. Hệ thống cập nhật trạng thái phụ cho đơn hàng trên cơ sở dữ liệu.

	5. Hệ thống gửi thông báo đến Shipper chịu trách nhiệm đơn hàng đó. 6. Thông báo trạng thái: "Đồ ăn của bạn đã chuẩn bị xong" cũng được gửi đồng thời đến ứng dụng của Khách hàng.
Luồng rẽ nhánh	- Nếu tại thời điểm nhấn nút vẫn chưa có Shipper nào nhận đơn, hệ thống sẽ lưu trạng thái "Chờ Shipper" và tự động gửi thông báo lấy hàng ngay khi có Shipper chấp nhận đơn sau đó.
Các yêu cầu đặc biệt	Tín hiệu thông báo phải được gửi đi theo thời gian thực để tối ưu hóa thời gian lấy hàng của Shipper.
Tiền điều kiện	Đơn hàng phải ở trạng thái CONFIRMED (Đã được nhà hàng xác nhận trước đó).
Hậu điều kiện	Đơn hàng sẵn sàng để chuyển sang trạng thái DELIVERING ngay khi Shipper quét mã hoặc xác nhận lấy hàng.
Điểm mở rộng	Tự động in hóa đơn tại quầy ngay khi nhà hàng nhấn xác nhận nấu xong.

2.6.12. Use case “Xem thống kê doanh thu”

Bảng 17. Bảng mô tả chi tiết use case xem thống kê doanh thu

Tên use case	Xem thống kê doanh thu
Mô tả	Cho phép nhà hàng theo dõi tổng kết quả kinh doanh, số lượng đơn hàng hoàn thành và tổng số tiền thu được theo từng mốc thời gian (ngày, tuần, tháng)

Người thực hiện	Nhà hàng
Luồng sự kiện chính	1. Nhà hàng chọn mục "Thống kê doanh thu" trên Dashboard quản trị. 2. Người dùng chọn khoảng thời gian cần xem báo cáo (mặc định là ngày hiện tại). 3. Hệ thống thực hiện truy vấn các đơn hàng có trạng thái COMPLETED trong bảng orders gắn với restaurant_id đó. 4. Hệ thống tính tổng số tiền thu được sau khi đã trừ đi phí dịch vụ (nếu có). 5. Hệ thống hiển thị biểu đồ tăng trưởng doanh thu và bảng chi tiết các đơn hàng đã thực hiện thành công.
Luồng rẽ nhánh	- Tại bước 3, nếu không có đơn hàng nào hoàn thành trong kỳ, hệ thống hiển thị thông báo: "Chưa có dữ liệu doanh thu cho khoảng thời gian này". - Người dùng có thể lọc xem danh sách các món ăn "Bán chạy nhất" để phân tích xu hướng khách hàng.
Các yêu cầu đặc biệt	Các con số thống kê phải được tính toán chính xác tuyệt đối từ lịch sử giao dịch. Giao diện biểu đồ phải trực quan và hỗ trợ cảm ứng (zoom/pan) trên thiết bị di động.
Tiền điều kiện	Nhà hàng đã đăng nhập thành công và có ít nhất một đơn hàng ở trạng thái COMPLETED.

Hậu điều kiện	Nhà hàng nắm bắt được doanh số thực tế để đưa ra các quyết định điều chỉnh thực đơn hoặc kế hoạch kinh doanh.
Điểm mở rộng	Tính năng xuất báo cáo chi tiết ra định dạng file Excel hoặc PDF để lưu trữ hoặc đối soát thủ công.

2.6.13. Use case “Xác thực tài khoản shipper”

Bảng 18. Bảng mô tả chi tiết use case xác thực tài khoản shipper

Tên use case	Xác thực tài khoản shipper
Mô tả	Cho phép nhân viên giao hàng (Shipper) đăng nhập vào hệ thống để bắt đầu nhận đơn, cập nhật lộ trình giao hàng và theo dõi thu nhập.
Người thực hiện	Shipper
Luồng sự kiện chính	1. Người dùng chọn chế độ "Đăng nhập Shipper" trên ứng dụng di động. 2. Người dùng nhập Email và Mật khẩu đã đăng ký. 3. Hệ thống Client gửi yêu cầu xác thực qua Firebase Authentication để kiểm tra danh tính. 4. Sau khi Firebase trả về mã UID hợp lệ, hệ thống gửi yêu cầu kiểm tra vai trò lên Node.js Server.

	5. Server truy vấn cơ sở dữ liệu để xác nhận trường role có giá trị là SHIPPER. 6. Nếu đúng, hệ thống chuyển giao diện đến màn hình "Thị trường đơn hàng" (Order Market).
Luồng rẽ nhánh	- Tại bước 3, nếu sai thông tin: Hệ thống báo "Email hoặc mật khẩu không chính xác". - Tại bước 5, nếu tài khoản không có vai trò SHIPPER (ví dụ: Khách hàng thường): Hệ thống thông báo "Tài khoản của bạn không có đặc quyền dành cho Shipper" và từ chối truy cập.
Các yêu cầu đặc biệt	Sau khi đăng nhập thành công, ứng dụng phải yêu cầu Shipper cấp quyền "Luôn truy cập vị trí" (Always access location) để phục vụ tính năng tracking.
Tiền điều kiện	Shipper đã có tài khoản và đã được quản trị viên duyệt thông tin cá nhân (biển số xe, số điện thoại) trong bảng shippers.
Hậu điều kiện	Shipper đăng nhập thành công và sẵn sàng để chuyển sang chế độ "Bắt đơn".
Điểm mở rộng	Tích hợp tính năng "Nhận diện khuôn mặt" hoặc "Cảm biến vân tay" cho các lần đăng nhập sau để tăng tính bảo mật và thuận tiện.

2.6.14. Use case “Bật chế độ bắt đơn”

Bảng 19. Bảng mô tả chi tiết use case bật chế độ bắt đơn

Tên use case	Bật chế độ bắt đơn
--------------	--------------------

Mô tả	Cho phép Shipper chuyển trạng thái tài khoản sang "Sẵn sàng hoạt động" để hệ thống bắt đầu quét và hiển thị các đơn hàng mới trong khu vực lân cận.
Người thực hiện	Shipper
Luồng sự kiện chính	1. Shipper truy cập vào màn hình Trang chủ của ứng dụng giao hàng. 2. Shipper nhấn vào nút gạt "Bật chế độ bắt đơn". 3. Hệ thống kiểm tra quyền truy cập vị trí (Location Permission) của thiết bị. 4. Hệ thống gửi yêu cầu cập nhật trạng thái <code>is_active = true</code> lên Node.js Server. 5. Server cập nhật tọa độ GPS cuối cùng của Shipper vào bảng shippers trong Database. 6. Hệ thống bắt đầu đẩy danh sách các đơn hàng ở gần (Bán kính < 15km) có trạng thái CONFIRMED lên màn hình của Shipper.
Luồng rẽ nhánh	- Tại bước 3, nếu Shipper từ chối quyền vị trí: Hệ thống hiển thị thông báo "Yêu cầu bật định vị để nhận đơn" và không kích hoạt trạng thái hoạt động. - Nếu tài khoản Shipper đang bị khóa (Banned): Hệ thống báo lỗi và yêu cầu liên hệ quản trị viên.
Các yêu cầu đặc biệt	Trạng thái bắt đơn phải được đồng bộ liên tục. Nếu Shipper mất kết nối Internet quá 5 phút, hệ thống sẽ tự động chuyển trạng thái về "Ngoại tuyến".

Tiền điều kiện	Shipper đã đăng nhập thành công và không đang trong quá trình thực hiện một đơn hàng khác.
Hậu điều kiện	Shipper bắt đầu nhận được thông báo về các đơn hàng mới và có thể chọn "Chấp nhận đơn".
Điểm mở rộng	Tích hợp tính năng "Bắt đơn tự động" dựa trên tiêu chí về quãng đường và doanh thu tối thiểu do Shipper thiết lập.

2.6.15. Use case “Giao thành công”

Bảng 20. Bảng mô tả chi tiết use case giao thành công

Tên use case	Giao thành công
Mô tả	Shipper xác nhận đã giao đồ ăn tận tay khách hàng thành công, hệ thống ghi nhận hoàn tất đơn hàng và thực hiện quyết toán doanh thu.
Người thực hiện	Shipper
Luồng sự kiện chính	1. Shipper di chuyển tới vị trí của Khách hàng. 2. Shipper thực hiện giao đồ ăn và nhận thanh toán (nếu đơn hàng là tiền mặt - COD). 3. Shipper nhấn nút "Hoàn thành giao hàng" trên ứng dụng. 4. Hệ thống gọi API cập nhật trạng thái đơn hàng thành COMPLETED trong bảng orders.

	5. Server ghi nhận thời gian hoàn thành (updated_at) và tính toán phí ship cộng vào ví của Shipper. 6. Hệ thống gửi thông báo "Đơn hàng của bạn đã giao thành công!" đến ứng dụng của Khách hàng và Nhà hàng.
Luồng rẽ nhánh	- Khách hàng không nhận hàng: Shipper nhấn "Báo cáo sự cố", nhập lý do (vd: Không liên lạc được). Đơn hàng sẽ được chuyển trạng thái "Hủy" hoặc "Hoàn trả" tùy theo quy định hệ thống. - Lỗi vị trí: Nếu Shipper ở quá xa tọa độ khách hàng (>500m), hệ thống cảnh báo yêu cầu xác nhận lại vị trí.
Các yêu cầu đặc biệt	Việc cập nhật trạng thái phải diễn ra tức thời để giải phóng Shipper cho các đơn hàng tiếp theo. Dữ liệu tài chính (doanh thu) phải được cập nhật chính xác tuyệt đối ngay sau khi nhấn xác nhận.
Tiền điều kiện	Đơn hàng phải đang ở trạng thái DELIVERING (Đang giao hàng).
Hậu điều kiện	Đơn hàng kết thúc vòng đời.
Điểm mở rộng	Yêu cầu Shipper chụp ảnh món ăn đã giao tại vị trí của khách hàng để làm bằng chứng.

2.6.16. Use case “Xem thống kê thu nhập”

Bảng 21. Bảng mô tả chi tiết use case Xem thống kê thu nhập

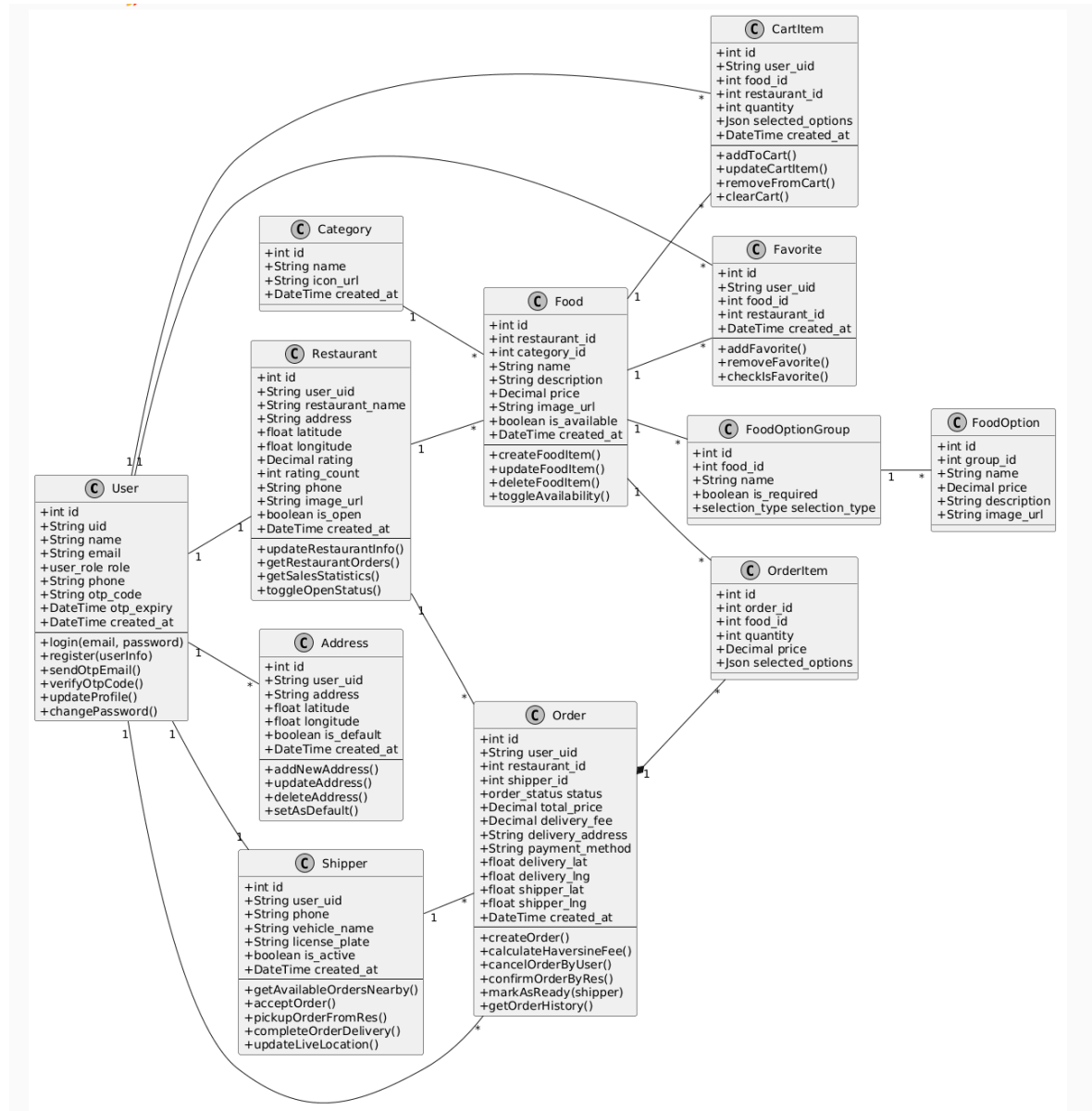
Tên use case	Xem thống kê thu nhập
--------------	-----------------------

Mô tả	Cho phép Shipper xem tổng số tiền hoa hồng nhận được từ các đơn hàng đã giao thành công, giúp họ quản lý tài chính cá nhân.
Người thực hiện	Shipper
Luồng sự kiện chính	1. Shipper truy cập vào mục "Ví của tôi" hoặc "Thu nhập" trong ứng dụng. 2. Hệ thống hiển thị tổng số dư hiện tại và danh sách các giao dịch gần đây. 3. Shipper chọn xem theo bộ lọc: Hôm nay, Tuần này hoặc Tháng này. 4. Hệ thống truy vấn các đơn hàng COMPLETED gắn với shipper_id đó để tính tổng phí vận chuyển (delivery_fee). 5. Hệ thống hiển thị các thông số: Tổng số đơn đã giao, Tổng quãng đường đã đi, và Tổng thu nhập thực nhận.
Luồng rẽ nhánh	- Nếu Shipper mới tham gia và chưa có đơn hàng: Hệ thống hiển thị số dư bằng 0 và gợi ý "Hãy bắt đầu chuyến xe đầu tiên của bạn".
Các yêu cầu đặc biệt	Thu nhập phải được cập nhật ngay lập tức (Real-time) sau khi đơn hàng chuyển sang trạng thái COMPLETED.
Tiền điều kiện	Shipper đã đăng nhập thành công.
Hậu điều kiện	Shipper nắm được hiệu suất làm việc và số tiền tích lũy được trên hệ thống.

Điểm mở rộng	Tính năng "Rút tiền" (Withdraw) về tài khoản ngân hàng liên kết khi số dư đạt mức tối thiểu.
--------------	--

2.7. Thiết kế biểu đồ lớp (Class Diagram)

2.7.1. Biểu đồ lớp tổng quan



Hình 16. Biểu đồ lớp tổng quan

2.7.2. Các lớp chính của hệ thống

◆ Lớp User:

Lớp User đóng vai trò là thực thể trung tâm và khởi nguồn cho hầu hết các luồng dữ liệu trong hệ thống. Nhiệm vụ cốt lõi của lớp này là định danh sinh trắc học và phân quyền thông qua hệ thống role. Ngoài việc quản lý thông tin tĩnh như họ tên, số điện thoại, lớp User còn đảm nhận vòng đời tự bảo mật thông qua việc gọi hàm tạo và xác minh mã OTP `verifyOtpCode()`.

Bảng 22. Lớp user

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK, Auto Increment) + uid: String (Unique) + name: String + email: String (Unique) + role: Enum + phone:String (Nullable) + otp_code: String (Nullable) + otp_expiry: DateTime (Nullable) + created_at: DateTime	login() register() sendOtpEmail() verifyOtpCode() updateProfile() changePassword()	1 User – 0..1 Shipper 1 User – 0..1 Restaurant 1 User – N Address 1 User – N Order 1 User – N CartItem 1 User – N Favorite

◆ Lớp Address:

Lớp Address giải quyết bài toán tính linh hoạt cho người sử dụng. Thay vì giới hạn mỗi người dùng chỉ có một tọa độ, lớp này áp dụng quan hệ 1-Nhiều với user, cho phép người dùng tự tổ chức danh bạ tọa độ vật lý của riêng mình (Nhà riêng, Cơ quan, v.v.). Thuộc tính `is_default` kết hợp cùng phương thức `setAsDefault()` giúp hệ thống sở hữu cơ chế ưu tiên, hỗ trợ tối đa hóa tốc độ thao tác chọn điểm đến tại màn hình thanh toán mà không bắt buộc khách hàng phải quét bật GPS mỗi lần sử dụng ứng dụng.

Bảng 23. Lớp address

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + user_uid: String (FK → User.uid) + address: String + latitude: Float + longitude: Float + is_default: Boolean + created_at: DateTime	addNewAddress() updateAddress() deleteAddress() setAsDefault()	N Address – 1 User

◆ **Lớp Restaurant:**

Lớp Restaurant đại diện cho hạt nhân cung cấp dịch vụ ẩm thực trong mô hình kinh doanh của hệ thống. Nhiệm vụ của lớp này không chỉ dừng lại ở việc lưu trữ (tên quán, địa chỉ, kinh độ, vĩ độ) mà còn đóng vai trò như một bảng điều khiển động. Thông qua các phương thức điều khiển trạng thái như `toggleOpenStatus()`, lớp này có thể tự quyết định trạng thái hiển thị của chính mình trên ứng dụng Khách hàng tùy theo giờ giấc hoạt động.

Bảng 24. Lớp restaurant

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + user_uid: String (FK → User.uid) + restaurant_name: String + address: String + latitude: Float + longitude: Float + rating: Decimal + rating_count: Int + phone: String + image_url: String + is_open: Boolean + created_at: DateTime	updateRestaurantInfo() getRestaurantOrders() getSalesStatistics() toggleOpenStatus()	N Restaurant – 1 User 1 Restaurant – N Food 1 Restaurant – N Order

◆ Lớp Shipper:

Lớp Shipper là đại diện kỹ thuật số cho lực lượng đối tác vận chuyển ngoài thực địa. Trọng trách cốt lõi của lớp này là quản lý thông tin phương tiện (vehicle_name, license_plate) và quan trọng nhất là duy trì tọa độ di chuyển

theo thời gian thực (Realtime location). Vòng đời của lớp Shipper có tính chất "chấp nhận rủi ro", nghĩa là liên tục chuyển đổi trạng thái sẵn sàng (Active) để lắng nghe sự kiện `acceptOrder()` từ mạng lưới. Khi một tương tác nhận đơn xảy ra, lớp Shipper sẽ tạm thời bị "khóa" với một Order duy nhất cho đến khi hàm `completeOrderDelivery()` được kích hoạt, đảm bảo tính toàn vẹn của chuỗi Logistics đồ ăn.

Bảng 25. Lớp shipper

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + user_uid: String (FK → User.uid, Unique) + phone: String + vehicle_name: String + license_plate: String (Unique) + is_active: Boolean + created_at: DateTime	<code>getAvailableOrdersNearby()</code> <code>acceptOrder()</code> <code>pickupOrderFromRes()</code> <code>completeOrderDelivery()</code> <code>updateLiveLocation()</code>	1 Shipper – 1 User 1 Shipper – N Order

◆ Lớp Food và Category:

Lớp Food đóng vai trò là "Sản phẩm cốt lõi" mang lại giá trị thương mại cho toàn hệ thống. Đây là thực thể có tần suất được truy vấn (Query) lớn nhất từ phía App Khách hàng. Lớp này chứa các logic nghiệp vụ quan trọng về giá trị kinh tế (`price`, `is_available`) và hình ảnh trực quan (`image_url`). Để tối ưu

hóa thuật toán tìm kiếm và gạch nối giao diện, lớp Food được tổ chức phụ thuộc mạnh mẽ vào lớp Category (Danh mục). Mô hình này cho phép thuật toán dễ dàng lọc và phân nhóm sản phẩm, đồng thời mở ra khả năng mở rộng để gộp thêm các lớp tùy chọn món phụ (FoodOptionGroup) mà không phá vỡ cấu trúc định giá gốc.

Bảng 26. Lớp food

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + restaurant_id: Int (FK → Restaurant.id) + category_id: Int (FK → Category.id) + name: String + description: String + price: Decimal + image_url: String + is_available: Boolean + created_at: DateTime	createFoodItem() updateFoodItem() deleteFoodItem() toggleAvailability()	N Food – 1 Restaurant N Food – 1 Category 1 Food – N OrderItem 1 Food – N CartItem 1 Food – N Favorite 1 Food – N FoodOptionGroup

Bảng 27. Lớp category

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + name: String (Unique) + icon_url: String + created_at: DateTime	Không có	1 Category – N Food

◆ Lớp Food Option Group và Food Option:

Hệ thống giao đồ ăn trong thực tế đòi hỏi sự tùy biến rẽ nhánh cực kỳ linh hoạt (Ví dụ: Trà sữa phải chọn Size, chọn Mức đá, Mức đường, Thêm Topping). Để giải quyết triệt để bài toán này mà không làm phình to dữ liệu gốc của bảng Food, kiến trúc hệ thống thiết kế ra bộ đôi lớp FoodOptionGroup và FoodOption.

Bảng 28. Lớp food option group

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + food_id: Int (FK → Food.id) + name: String + is_required: Boolean + selection_type: Enum	Không có	N Group – 1 Food 1 Group – N FoodOption

Bảng 29. Lớp food option

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + group_id: Int (FK → FoodOptionGroup.id) + name: String + price: Decimal + description: String + image_url: String	Không có	N Option – 1 Group

◆ Lớp Order:

Lớp Order (Đơn hàng) là thực thể phức tạp nhất và mang nặng tính nghiệp vụ (Business Logic) nhất trong hệ thống. Lớp này mô phỏng toàn bộ một Hợp đồng giao dịch điện tử. Không giống như các thực thể tĩnh, Order là một cỗ máy trạng thái (State Machine) chuyển biến liên tục qua các giai đoạn: Chờ xác nhận, Đang chuẩn bị, Đang giao và Hoàn thành. Nó là tâm điểm giao thoa, chứa khóa ngoại liên kết đồng thời 3 chủ thể: User (Khách), Restaurant (Nhà hàng) và Shipper (Tài xế). Các thuật toán lõi của phần mềm như tính toán khoảng cách, cộng dồn phí vận chuyển và quy đổi tổng hóa đơn đều được đóng gói và giao phó cho các phương thức cục bộ của lớp này xử lý an toàn.

Bảng 30. Lớp order

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + user_uid: String (FK → User.uid) + restaurant_id: Int (FK → Restaurant.id) + shipper_id: Int (FK → Shipper.id, Nullable) + status: Enum + total_price: Decimal + delivery_fee: Decimal + delivery_address: String + payment_method: String	createOrder() calculateHaversineFee() cancelOrderByUser() confirmOrderByRes() markAsReady() getOrderHistory()	N Order – 1 User N Order – 1 Restaurant N Order – 1 Shipper 1 Order – N OrderItem

+ delivery_lat: Float + delivery_lng: Float + shipper_lat: Float + shipper_lng: Float + created_at: DateTime		
--	--	--

◆ Lớp OrderItem:

Lớp OrderItem là một thực thể cầu nối (Associative Entity) mang tính chất lịch sử bất biến. Trong khi lớp Order chỉ quản lý thông tin vĩ mô của một giao dịch (Tổng tiền, Shipper, Trạng thái), thì OrderItem đóng vai trò bóc tách và ghi nhận chi tiết vi mô (Từng món ăn, Số lượng, Giá cả). Một đặc điểm kỹ thuật quan trọng của lớp này là nó bắt buộc phải "chụp nhanh" (Snapshot) và lưu trữ thuộc tính price cùng tham số cấu hình dạng chuỗi selected_options (dạng JSON) tại đúng thời điểm thanh toán. Kỹ thuật này đảm bảo rằng: nếu trong tương lai lớp Food có bị chủ nhà hàng thay đổi giá bán hoặc xóa cấu hình món, thì biên lai lịch sử của khách hàng nằm tại OrderItem vẫn giữ nguyên được tính toàn vẹn và mức giá trị hợp pháp ban đầu. Lớp này nhận dữ liệu chuyển hóa trực tiếp từ CartItem và ràng buộc vòng ngoài nguyên ngặt với lớp Order (Quan hệ N-1).

Bảng 31. Lớp order item

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + order_id: Int (FK → Order.id) + food_id: Int (FK → Food.id) + quantity: Int + price: Decimal + selected_options: Json	Không có	N OrderItem – 1 Order N OrderItem – 1 Food

♦ Lớp CartItem:

Lớp CartItem mặc dù có vòng đời rất ngắn nhưng lại là bước đệm then chốt quyết định tỷ lệ chuyển đổi đơn hàng. Thực thể này hoạt động như một bộ nhớ tạm thời lưu trữ ý định mua sắm của Khách hàng. Sự thông minh của kiến trúc hệ thống nằm ở chỗ lớp CartItem được gán logic xử lý xung đột: nó sở hữu các hàm thao tác như addToCart(), updateCartItem() nhưng tự động vô hiệu hóa nếu nhận diện sản phẩm được thêm vào có cấu trúc ID sai lệch với restaurant_id hiện tại (Ngăn chặn gộp đơn chéo nhà hàng). Sau khi khách hàng thực hiện checkout, toàn bộ dữ liệu của lớp này sẽ được chuyển hóa thành OrderItem và bản thân nó bị hủy bỏ để thu hồi bộ nhớ.

Bảng 32. Lớp cart item

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + user_uid: String (FK → User.uid) + food_id: Int (FK → Food.id) + restaurant_id: Int (FK → Restaurant.id) + quantity: Int + selected_options: Json + created_at: DateTime	addToCart() updateCartItem() removeFromCart() clearCart()	N CartItem – 1 User N CartItem – 1 Food N CartItem – 1 Restaurant

♦ Lớp Favorite:

Lớp Favorite là một đại diện đặc biệt của kỹ thuật thiết kế trải nghiệm người dùng (UX Design). Nếu xét thuần túy về chuỗi cung ứng, lớp này không tham gia vào vòng đời Đặt hàng hay Thanh toán. Tuy nhiên, nó là nhân tố cốt lõi để giữ chân người dùng (Customer Retention). Bản chất lớp Favorite là một bảng ánh xạ liên kết nhiều chiều (N-N) giữa Khách hàng (User) và Nhà hàng (Restaurant) hoặc Món ăn (Food). Các chức năng như `addFavorite()`, `removeFavorite()` và đặc biệt là thuật toán rà soát tức thời `checkIsFavorite()` giúp ứng dụng cá nhân hóa giao diện, hiển thị nhanh các món tủ đã lưu ngay trên cùng màn hình Trang chủ mà người dùng không cần phải tốn công tìm kiếm lại.

Bảng 33. Lớp favorite

Thuộc tính	Hành vi	Quan hệ
+ id: Int (PK) + user_uid: String (FK → User.uid) + food_id: Int (FK → Food.id) + restaurant_id: Int (FK → Restaurant.id) + created_at: DateTime	addFavorite() removeFavorite() checkIsFavorite()	N Favorite – 1 User N Favorite – 1 Food N Favorite – 1 Restaurant

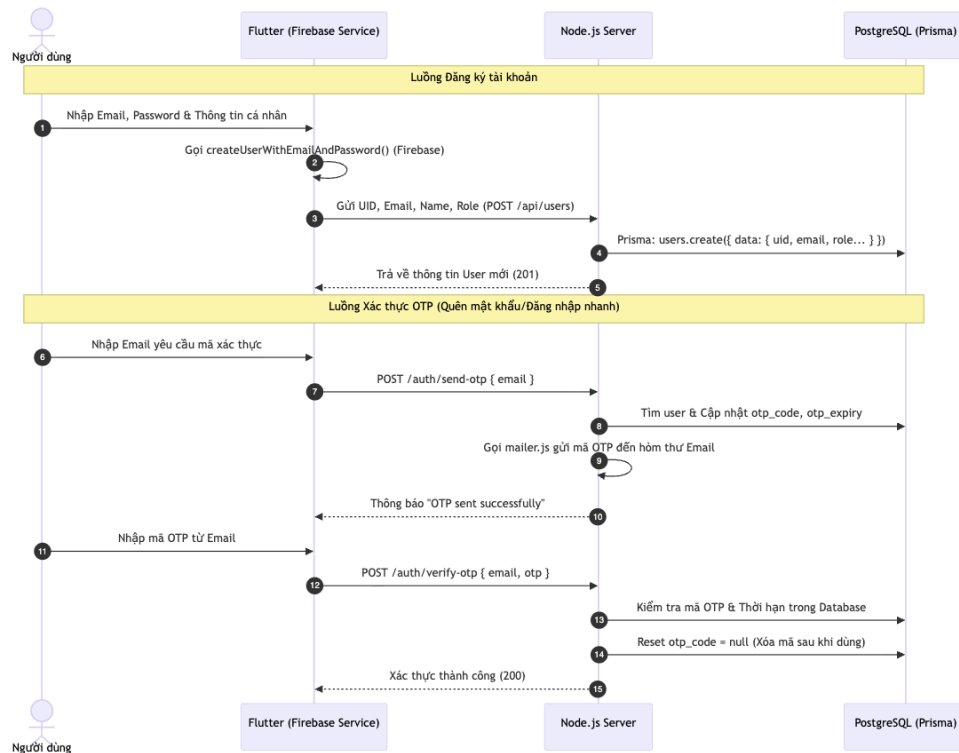
2.8. Biểu đồ trình tự (Sequence Diagram)

Biểu đồ trình tự (Sequence Diagram) trong hệ thống này được xây dựng nhằm mô tả rõ ràng sự tương tác dữ liệu và chuyển giao thông điệp giữa các Layer (lớp) của kiến trúc phần mềm theo trục thời gian. Toàn bộ các Use Case được đặc tả dựa trên 4 thành phần trụ cột của hệ thống:

- Frontend (App Flutter): Nơi tương tác trực tiếp với các tác nhân (User, Restaurant, Shipper). Xử lý giao diện và quản lý trạng thái client-side.
- Backend API (Node.js/Express): Đóng vai trò là cầu nối nghiệp vụ cốt lõi, tiếp nhận request JWT Token, xử lý thuật toán (tính khoảng cách Haversine, phân quyền) và Response định dạng JSON.
- Cơ sở dữ liệu (PostgreSQL qua Prisma ORM): Lớp lưu trữ dữ liệu với sự hỗ trợ của các kỹ thuật đảm bảo tính toàn vẹn thông tin.

Hệ thống Realtime (Supabase / Firebase): Cung cấp các WebSockets để đồng bộ hóa trạng thái tức thời (như GPS shipper, trạng thái đơn hàng) mà không cần Polling làm nghẽn máy chủ.

2.8.1. Biểu đồ trình tự use case “Xác thực người dùng”



Hình 17. Biểu đồ trình tự xác thực người dùng

Luồng đăng ký tài khoản:

- Người dùng nhập Email, Password và thông tin cá nhân trên ứng dụng.
- Ứng dụng gọi Firebase để tạo tài khoản bằng.
- Sau khi tạo thành công, trên App Firebase sinh ra một uid. Ứng dụng gửi toàn bộ UID, Email, Name đến Node.js Server thông qua API.
- Server sử dụng Prisma ORM để lưu thông tin người dùng vào bảng users trong cơ sở dữ liệu PostgreSQL.

Tài khoản được tạo thành công cả ở hệ thống xác thực (Firebase) và hệ thống lưu trữ nghiệp vụ (Postgres) với mã trạng thái 201 Created.

Luồng xác thực OTP:

- Bước gửi OTP:
 - + Người dùng nhập email để yêu cầu xác thực.

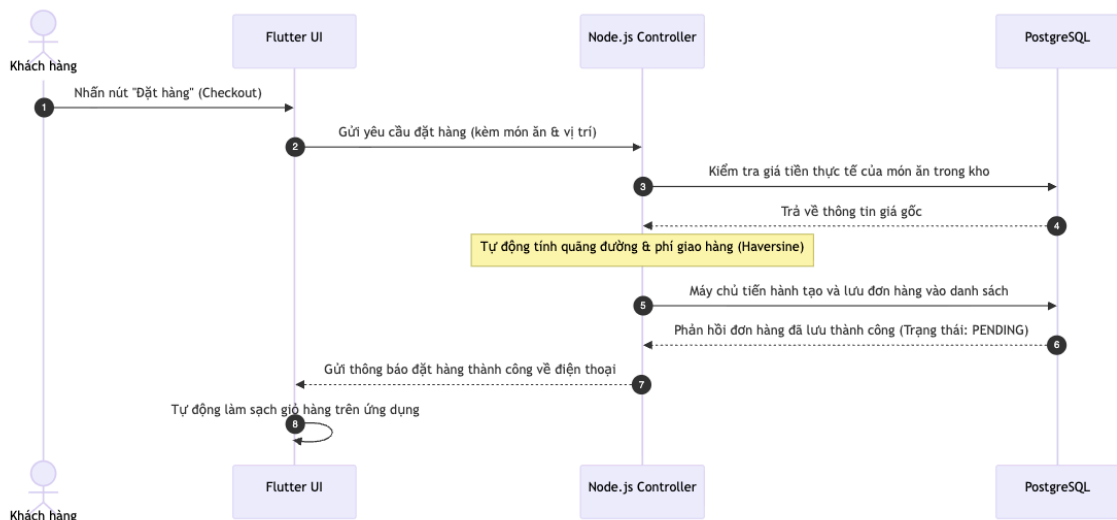
- + Ứng dụng gửi request POST đến server.
- + Server kiểm tra user trong database có tồn tại hay không. Nếu có, tạo ngẫu nhiên mã 6 số (otp_code) và gán thời gian hết hạn là 5 phút
- + Server gọi dịch vụ gửi mail để gửi mã OTP đến email người dùng.

- Bước xác thực OTP:

- + Ứng dụng gửi request POST đến server.
- + Server kiểm tra sự tồn tại của OTP và thời hạn.
- + Nếu hợp lệ, server cập nhật otp_code = null (xóa OTP sau khi dùng).

Kết quả: Server trả về kết quả 200 (OK) – Người dùng xác thực hợp lệ để đổi mật khẩu hoặc đăng nhập.

2.8.2. Biểu đồ trình tự use case “Đặt đơn hàng”



Hình 18. Biểu đồ trình tự đặt đơn hàng

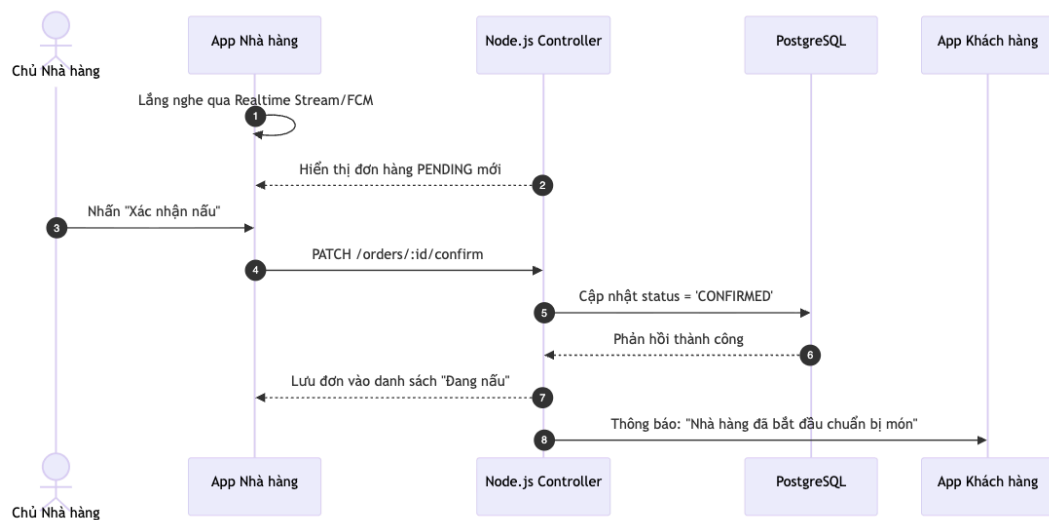
Luồng khởi tạo đơn hàng:

- Người dùng truy cập giỏ hàng, xác nhận địa chỉ giao hàng và phương thức thanh toán. Bấm "Thanh toán".

- App gửi request POST /api/orders bao gồm tọa độ giao nhận (lat, lng), payment_method, và mảng danh sách items.
- Server Node.js tiếp nhận request và duyệt qua mảng items:
 - + Query DB lấy đơn giá (price) của từng món ăn ở thời điểm hiện tại để chống gian lận thay đổi giá phía client.
 - + Cộng dồn giá các Options (nếu có lựa chọn thêm đồ ăn kèm).
- Server dùng hàm để tính khoảng cách giữa khách hàng và nhà hàng. Từ đó nhân với hệ số để ra phí giao hàng.
- App dùng Prisma \$transaction tạo một record ở bảng orders, đồng thời insert hàng loạt (bulk insert) vào bảng order_items.

Kết quả: Đơn hàng được tạo với trạng thái ban đầu là PENDING. Dữ liệu sẽ sẵn sàng để bắn Realtime về cho phía Nhà hàng.

2.8.3. Biểu đồ trình tự use case “Xác nhận đơn hàng”



Hình 19. Biểu đồ trình tự xác nhận đơn hàng

Luồng xử lý:

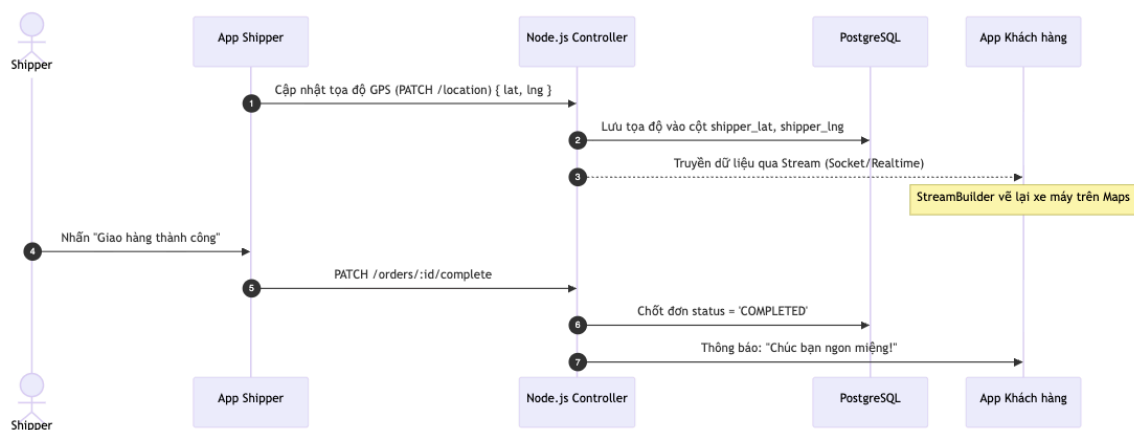
- Nhờ Supabase socket, App nhà hàng nhận được tín hiệu "Có đơn mới". Nhà hàng mở danh sách đơn, gọi API GET lấy thông tin chi tiết các đơn PENDING.

- Chủ quán bấm nút "Xác nhận đơn". App bắn request PATCH cập nhật trạng thái đơn hàng.

- Node.js kiểm tra xác thực JWT của nhà hàng, trả đúng vào id của đơn hàng và update trạng thái đơn hàng UPDATED.

Kết quả: Đơn hàng chuyển trạng thái CONFIRMED. Hệ thống tự động đẩy đơn hàng này "ra chợ" (available orders pool) để các tài xế quanh đó chuẩn bị nhìn thấy và bắt đơn.

2.8.4. Biểu đồ trình tự use case “Giao hàng”



Hình 20. Biểu đồ trình tự giao hàng

Luồng nhận đơn và cập nhật GPS:

- Shipper thấy đơn hàng đang CONFIRMED nên bấm vào nút Nhận đơn cập nhật trạng thái đơn hàng. Lúc này shipper_id được gán vào đơn.

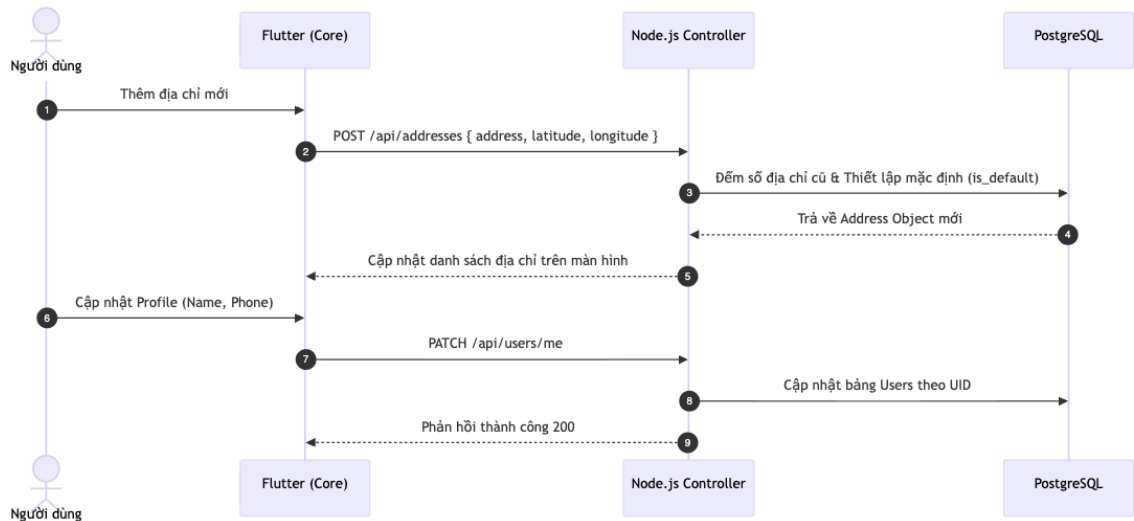
- Khi Shipper chạy xe tới quán cơm, lấy được đồ ăn, Shipper bấm Bắt đầu giao. Đơn hàng chuyển sang DELIVERING.

- Background Tracking: App của Shipper dùng dịch vụ định vị liên tục lấy GPS điện thoại và gọi API cập nhật tọa độ shipper_lat, shipper_lng lên Server.

- Supabase lắng nghe thay đổi tọa độ này, đồng bộ Realtime về màn hình điện thoại của User.

Kết quả: User thấy chiếc xe máy di chuyển trên bản đồ. Khi Shipper bấm Hoàn thành, đơn kết thúc ở ngưỡng COMPLETED.

2.8.5. Biểu đồ trình tự use case “Quản lý tài khoản và Địa chỉ”



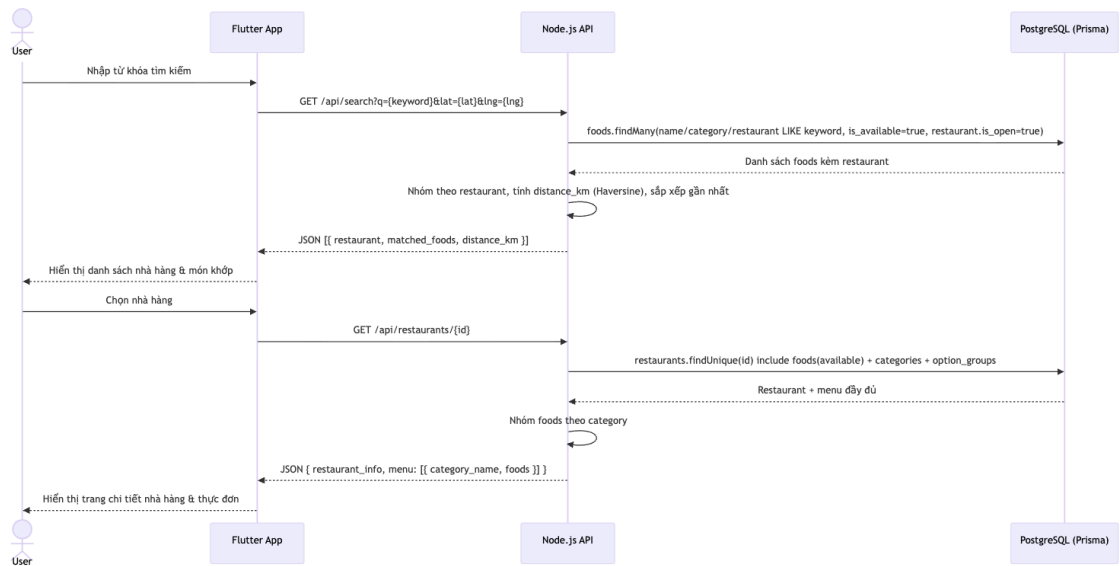
Hình 21. Biểu đồ trình tự quản lý tài khoản và địa chỉ

Luồng Quản lý thông tin & Địa chỉ

- Ứng dụng gọi GET lấy profile và danh sách các chi nhánh số địa chỉ.
- Khi User muốn cập nhật địa chỉ giao hàng, App lấy tọa độ mới gửi POST tạo mới địa chỉ. Phía DB insert dữ liệu với khóa liên kết user_uid.

Kết quả: Lần mua hàng tới, User có thể chọn nhanh từ db số địa chỉ được lưu.

2.8.6. Biểu đồ trình tự use case “Tìm kiếm và xem thực đơn”



Hình 22. Biểu đồ trình tự tìm kiếm và xem thực đơn

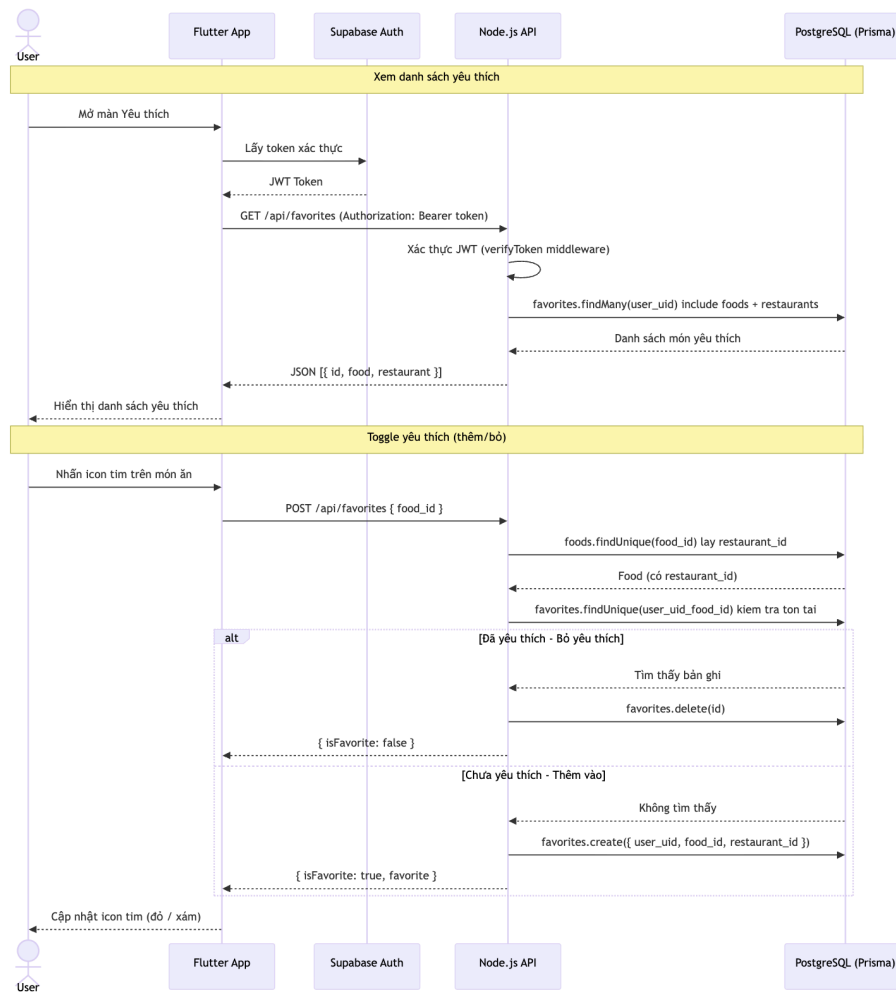
Luồng Tìm kiếm nhà hàng / món ăn;

- Khách hàng nhập từ khóa. Ứng dụng gửi request GET lấy danh sách món ăn, nhà hàng kèm từ khóa và tọa độ.
- Node.js Server truy vấn tìm món ăn/ nhà hàng khớp từ khóa.
- Server nhóm các món ăn theo nhà hàng, tính toán khoảng cách và sắp xếp nhà hàng gần nhất trước khi trả JSON về App.

Luồng Xem chi tiết thực đơn:

- Ứng dụng gọi API GET lấy danh sách món ăn.
- Server truy vấn lấy thông tin nhà hàng, danh mục món ăn (categories), danh sách món ăn và các nhóm tùy chọn (option_groups).
- Sau khi format data lại, Server trả dữ liệu về để Ứng dụng phân trang hiển thị thực đơn chi tiết.

2.8.7. Biểu đồ trình tự use case “Quản lý món yêu thích”

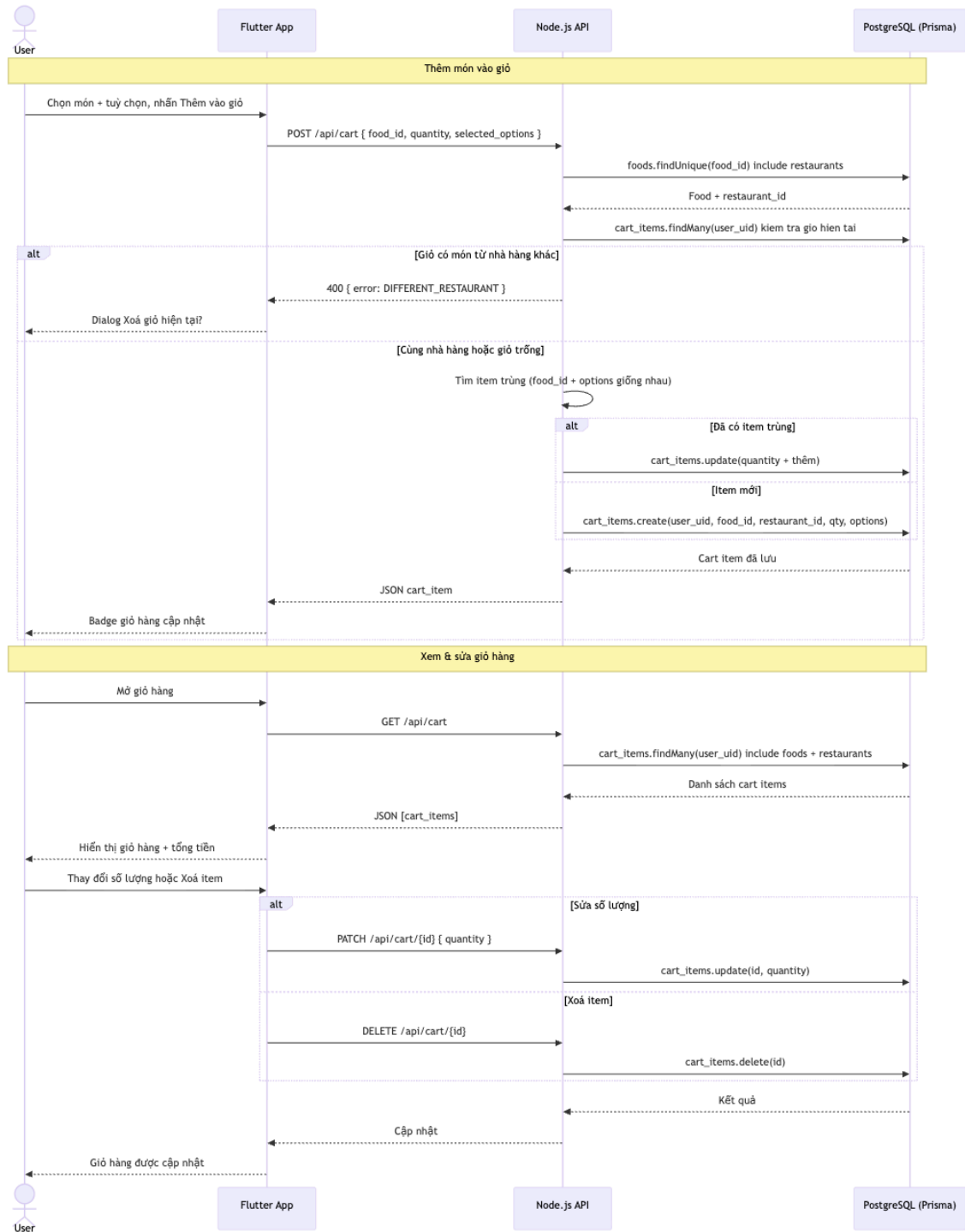


Hình 23. Biểu đồ trình tự quản lý món yêu thích

Luồng Thêm/Bỏ yêu thích:

- Người dùng nhấn vào biểu tượng "Tim" trên một món ăn.
 - App lấy token xác thực và gọi API POST.
 - Server xác thực quyền user. Sau đó truy xuất database kiểm tra xem bản ghi yêu thích (user_uid, food_id) đã tồn tại chưa bằng Unique Constraint.
 - Nếu đã tồn tại: Xóa (delete) bản ghi đó. Nếu chưa tồn tại: Tạo mới bản ghi.
- Kết quả: Nút tim trên giao diện cập nhật màu sắc dựa theo giá trị isFavorite server trả về.

2.8.8. Biểu đồ trình tự use case “Quản lý giỏ hàng”



Hình 24. Biểu đồ trình tự quản lý giỏ hàng

Luồng Thêm vào giỏ hàng:

- App gọi API POST thêm món ăn vào giỏ hàng.
- Server kiểm tra giỏ hàng hiện tại:

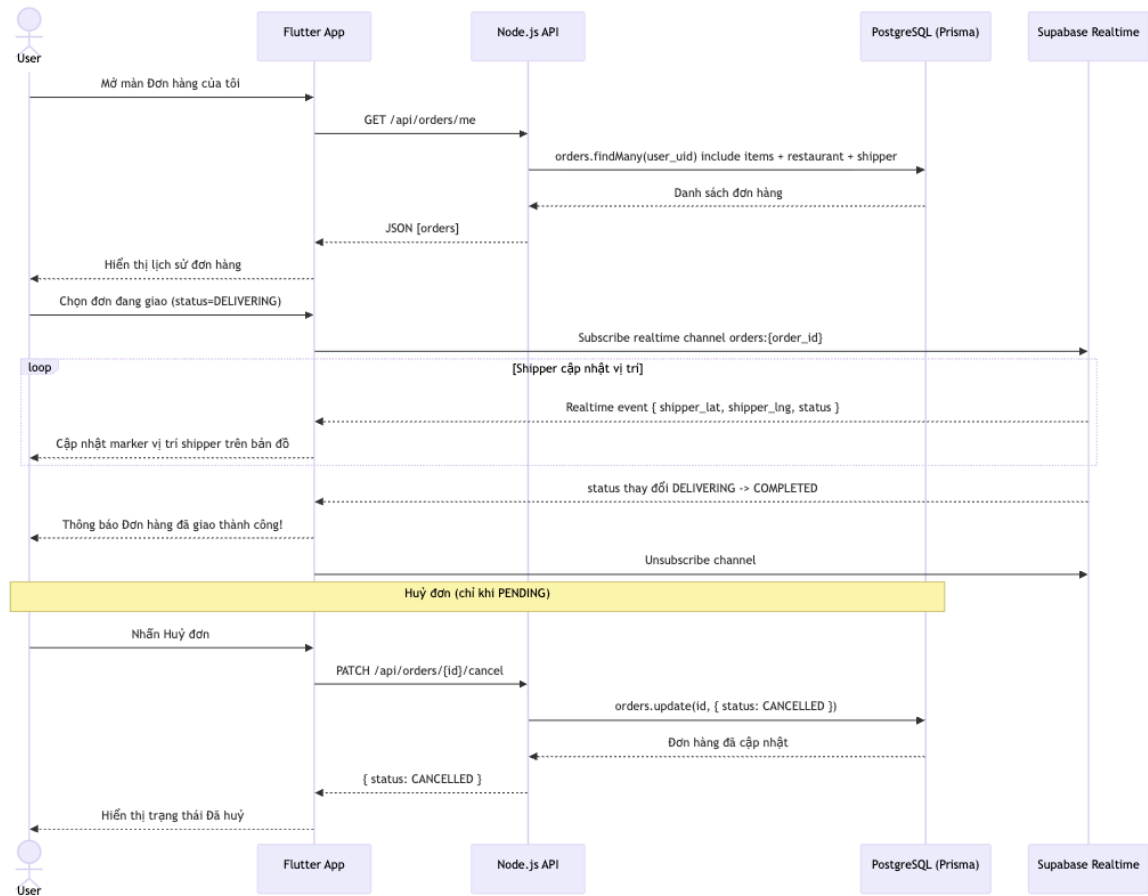
- + Nếu trong giỏ có món của nhà hàng khác: Server vớt lỗi (Error code: DIFFERENT_RESTAURANT). App chặn không cho chéo quán.
- + Nếu phù hợp: Server so sánh Options, nếu trùng lặp hoàn toàn thì cộng dồn quantity vào database. Ngược lại tạo dòng cart_items mới chứa file JSON options riêng.

Luồng Xem, Sửa giỏ hàng:

- Xem giỏ hàng: Khi User mở màn hình giỏ, App gọi API GET lấy dữ liệu giỏ hàng. Server dùng Prisma tìm theo user_uid, gộp thông tin món ăn và tính toán tổng chi phí để trả về định dạng mảng cho App vẽ Giao diện.
- Tăng, giảm số lượng: Khi bấm nút (+/-), App tự động bắn API PATCH cập nhật giỏ hàng kèm giá trị số lượng thay đổi. Server lập tức update trường quantity trong CSDL.
- Xóa món / Xóa giỏ:
 - + Nếu bấm xóa 1 món hoặc giảm số lượng về 0, App gọi API DELET để xóa đích danh bản ghi đó.
 - + Nếu User chấp nhận dọn sạch giỏ (để chuyển sang đặt quán khác), App sẽ kích hoạt lệnh DELETE xóa tất cả theo user_uid.

Kết quả: Dữ liệu giỏ hàng được đồng bộ hoàn toàn trên Database CSDL thay vì lưu tạm trên máy khách (Local Storage), giúp User nếu đổi thiết bị đăng nhập vẫn giữ nguyên nguyên giỏ hàng đang chọn

2.8.9. Biểu đồ trình tự use case “Theo dõi đơn hàng”



Hình 25. Biểu đồ trình tự theo dõi đơn hàng

Luồng Xem lịch sử đơn hàng:

- Người dùng mở tab "Đơn hàng của tôi". Ứng dụng gửi API GET lấy toàn bộ danh sách đơn hàng.

- Server Node.js dùng Prisma ORM query danh sách các đơn hàng dựa trên user_uid. Bao gồm cả dữ liệu liên kết (include) của Quán ăn, Tài xế, và Món ăn. Server trả mảng orders dạng JSON về App để hiển thị danh sách lịch sử.

Luồng Theo dõi qua Socket:

- Khi tài xế đang giao (status = DELIVERING), App phía người dùng sẽ subscribe vào Supabase realtime channel ứng với order_id.

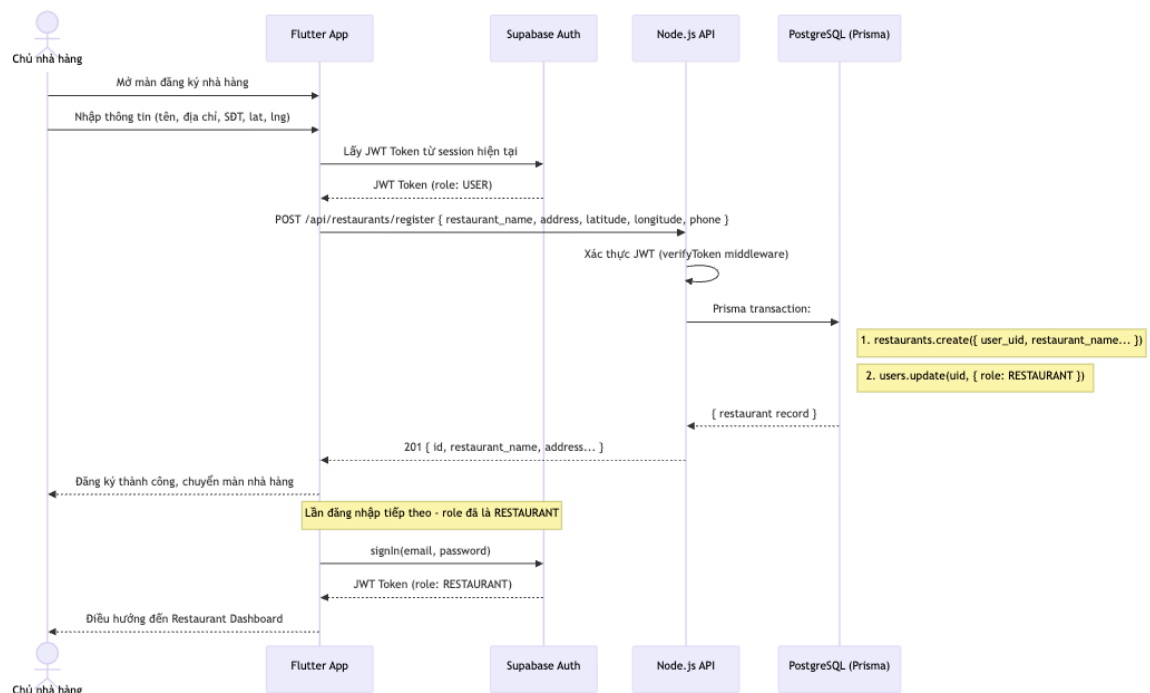
- Mỗi khi App của tài xế broadcast tọa độ GPS lên Server, Database cập nhật và Supabase rải sự kiện tới điện thoại của User.

Kết quả: Người dùng Thẻ đổi được trực tiếp lộ trình đơn hàng mà không cần liên tục làm mới làm hao pin thiết bị.

Luồng Hủy đơn hàng:

- Hệ thống hỗ trợ xử lý ngoại lệ: Chỉ khi đơn hàng mới đặt (status = PENDING) và quán chưa nấu, nút "Hủy đơn" mới xuất hiện.
- User bấm hủy, App gọi API PATCH cập nhật trạng thái đơn hàng.
- Server kiểm tra trạng thái và cập nhật status = CANCELLED. Phản hồi lại cho App để đổi màu hiển thị sang Đã hủy.

2.8.10. Biểu đồ trình tự use case “Xác thực tài khoản nhà hàng”



Hình 26. Biểu đồ trình tự xác thực tài khoản nhà hàng

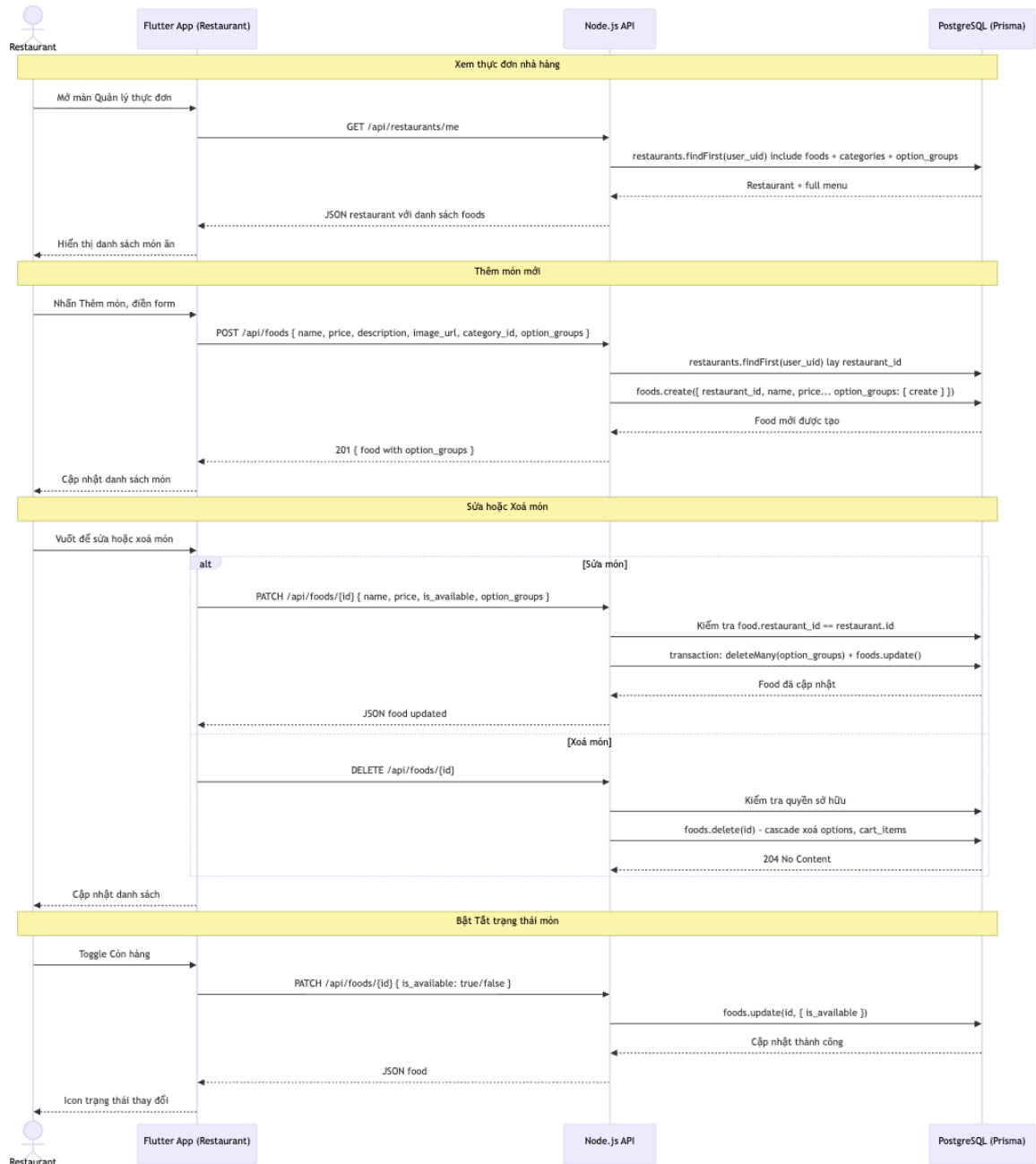
Luồng Đăng ký & Nâng cấp Role:

- Người dùng (role: USER) nhập thông tin cửa hàng: Tên, SĐT, Địa chỉ, Tọa độ bản đồ lên App, đính kèm JWT token.
- Node.js Server xử lý an toàn: Dùng khối Transaction (Giao dịch nguyên tử) để thực hiện đồng thời 2 việc:

+ Tạo record bảng restaurants.

+ Sửa cột role = RESTAURANT trong bảng users. Nếu 1 hành động fail, tất cả được Roll-back.

2.8.11. Biểu đồ trình tự use case “Quản lý món ăn”



Hình 27. Biểu đồ trình tự quản lý món ăn

Luồng xem thực đơn:

- Nhà hàng truy cập màn hình quản lý thực đơn.

- Ứng dụng gửi request GET đến server.
- Server truy vấn dữ liệu bằng Prisma, lấy thông tin nhà hàng kèm danh sách món (foods, categories, option_groups).
- Server trả về dữ liệu JSON chứa toàn bộ thực đơn.
- Ứng dụng hiển thị danh sách món ăn cho nhà hàng.

Luồng thêm món mới:

- Nhà hàng chọn thêm món và nhập thông tin (tên, giá, mô tả...).
- Ứng dụng gửi request POST tạo món ăn.
- Server xác định restaurant_id từ user.
- Server tạo món mới trong database, bao gồm cả option_groups.
- Server trả về kết quả 201 Created.
- Ứng dụng cập nhật lại danh sách món.

Luồng cập nhật món:

- Nhà hàng chọn chỉnh sửa món.
- Ứng dụng gửi request PATCH cập nhật thông tin món ăn.
- Server kiểm tra quyền sở hữu món (food.restaurant_id).
- Server cập nhật thông tin món và các option liên quan (transaction).
- Server trả về dữ liệu món đã cập nhật.
- Ứng dụng hiển thị dữ liệu mới.

Luồng xóa món:

- Nhà hàng chọn xóa món.
- Ứng dụng gửi request DELETE xóa món ăn.
- Server kiểm tra quyền sở hữu.
- Server xóa món và các dữ liệu liên quan (option_groups, cart_items).

- Server trả về 204 No Content.

- Ứng dụng cập nhật lại danh sách.

Luồng bật/tắt trạng thái món:

- Nhà hàng bật/tắt trạng thái còn hàng.

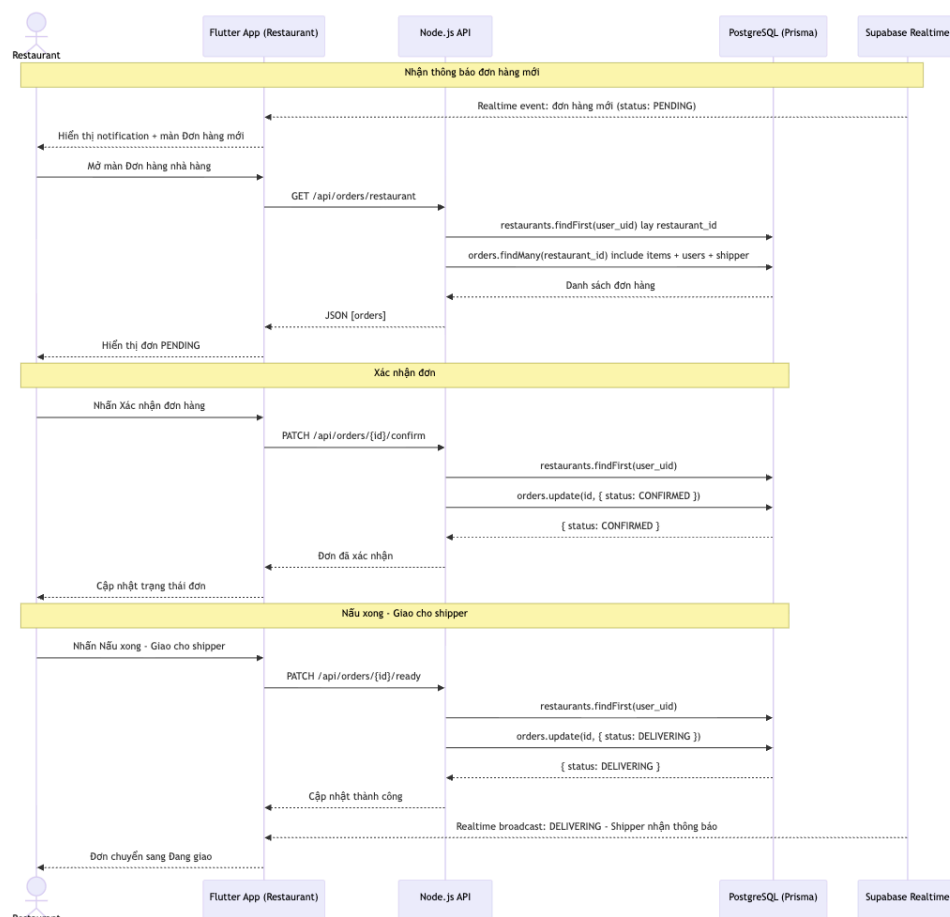
- Ứng dụng gửi request PATCH cập nhật trạng thái món ăn.

- Server cập nhật trạng thái trong database.

- Server trả về dữ liệu món đã cập nhật.

- Giao diện cập nhật icon trạng thái tương ứng.

2.8.12. Biểu đồ trình tự use case "Xác nhận món đã nấu xong"



Hình 28. Biểu đồ trình tự xác nhận món đã nấu xong

Luồng nhận đơn hàng mới:

- Khi có đơn hàng mới, hệ thống phát sự kiện realtime với trạng thái PENDING.

- Ứng dụng nhà hàng nhận thông báo và hiển thị notification.
- Nhà hàng truy cập màn hình đơn hàng.
- Ứng dụng gửi request GET lấy toàn bộ đơn hàng đến server.
- Server truy vấn database để lấy danh sách đơn hàng kèm thông tin liên quan (users, shipper).
- Server trả về danh sách đơn hàng dưới dạng JSON.
- Ứng dụng hiển thị các đơn hàng với trạng thái PENDING.

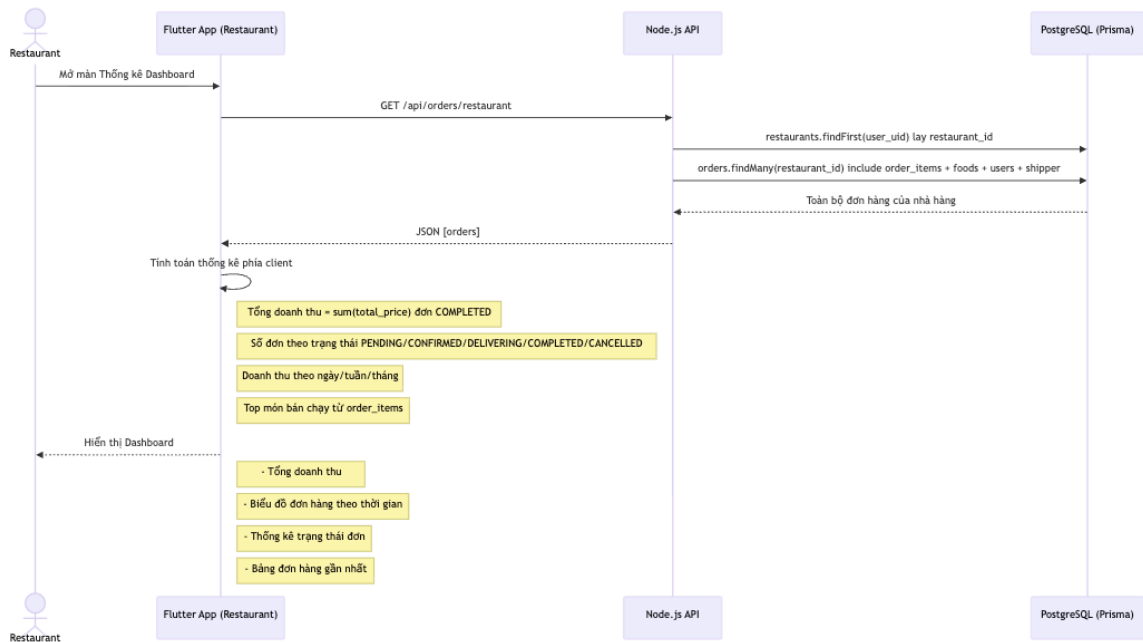
Luồng xác nhận đơn hàng:

- Nhà hàng nhấn xác nhận đơn.
- Ứng dụng gửi request PATCH cập nhật trạng thái đơn hàng.
- Server kiểm tra quyền của nhà hàng.
- Server cập nhật trạng thái đơn hàng thành CONFIRMED trong database.
- Server trả về kết quả cập nhật.
- Ứng dụng cập nhật trạng thái đơn hàng trên giao diện.

Luồng bàn giao cho shipper:

- Sau khi chuẩn bị xong, nhà hàng chọn “Giao cho shipper”.
- Ứng dụng gửi request PATCH cập nhật trạng thái đơn hàng.
- Server cập nhật trạng thái đơn hàng thành DELIVERING.
- Server trả về kết quả cập nhật thành công.
- Hệ thống phát sự kiện realtime với trạng thái DELIVERING để shipper nhận đơn.
- Ứng dụng hiển thị trạng thái “Đang giao”.

2.8.13. Biểu đồ trình tự use case “Xem thống kê doanh thu”



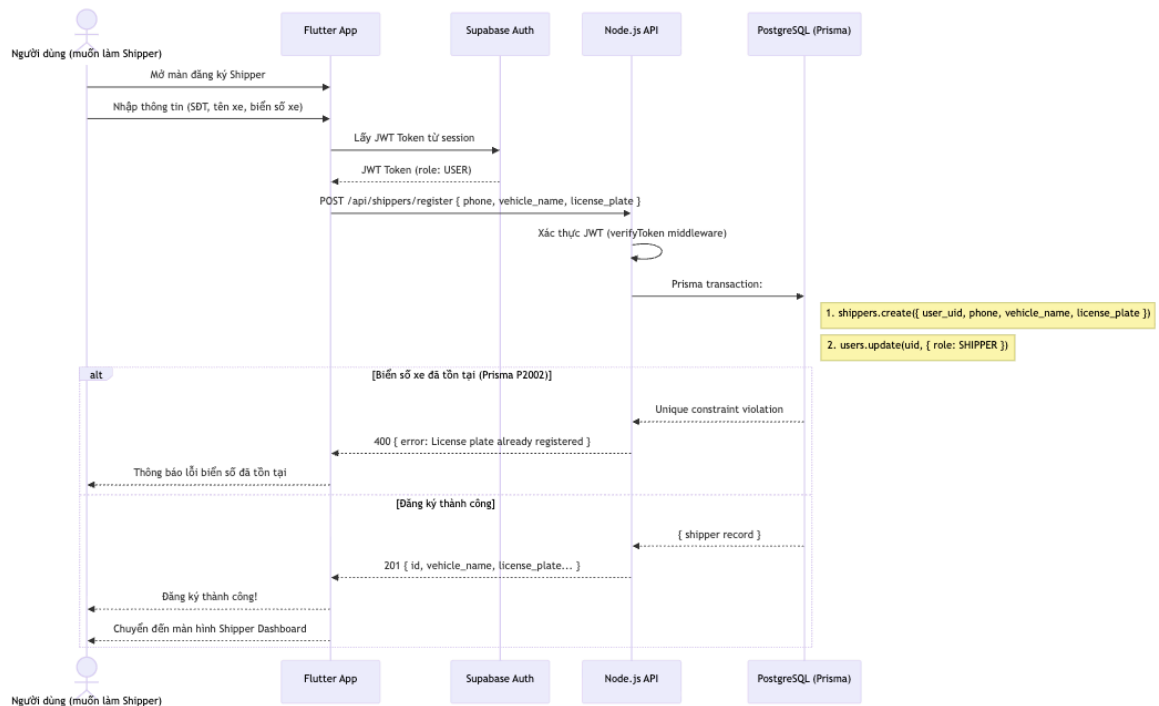
Hình 29. Biểu đồ trình tự xem thống kê doanh thu

Luồng Tổng hợp báo cáo:

- Bấm vào tab "Thống kê". App gọi API GET lấy dữ liệu để hiển thị.
- Lọc các đơn từ DB có trạng thái thỏa mãn COMPLETED.
- Thực hiện tính tổng cộng dồn trên trường total_price. Xuất dữ liệu thống kê ra mảng.

Kết quả: Render được biểu đồ Line và Bảng xếp hạng món bán chạy cho đối tác xem.

2.8.14. Biểu đồ trình tự use case “Xác thực tài khoản shipper”



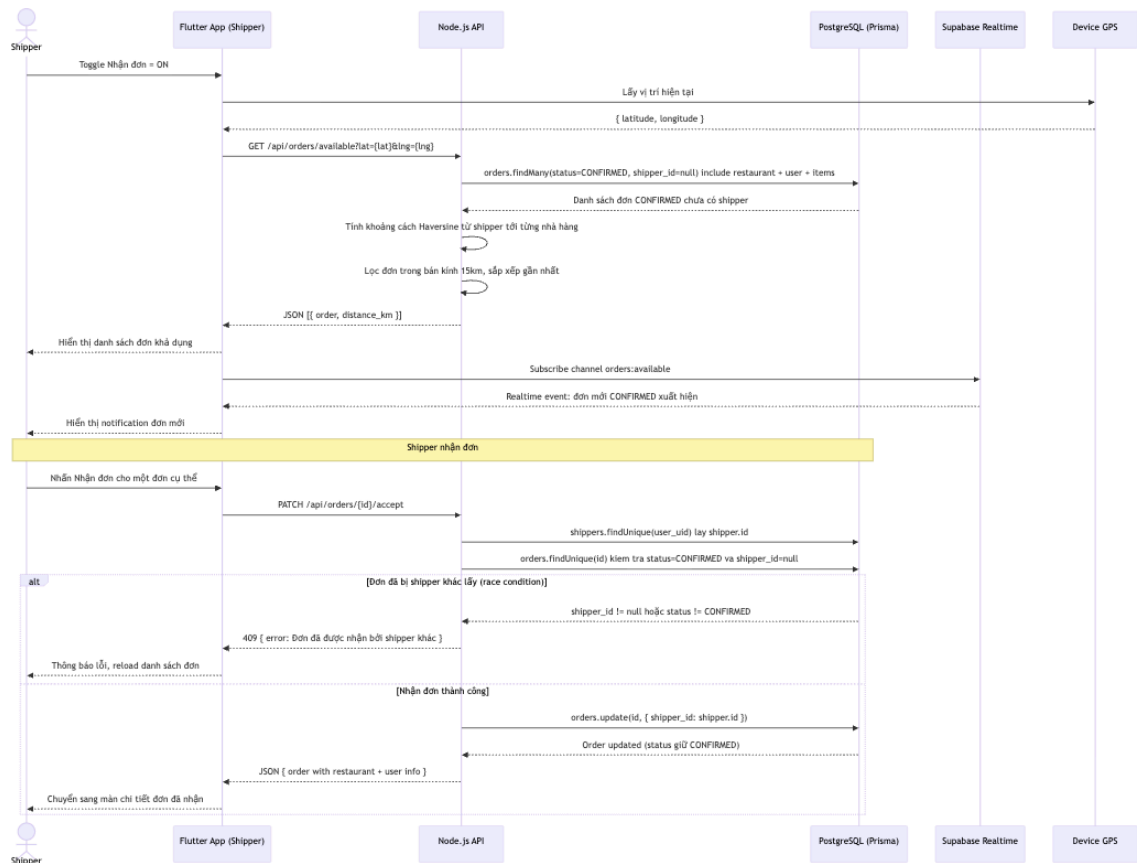
Hình 30. Biểu đồ trình tự xác thực tài khoản shipper

Luồng Đăng ký trở thành Shipper:

- Người dùng mở màn hình đăng ký và cung cấp các thông tin cần thiết bao gồm: Số điện thoại, tên xe và biển số xe.
- Ứng dụng Flutter truy xuất JWT Token từ phiên làm việc hiện tại của Supabase Auth để xác thực danh tính người dùng (lúc này vẫn mang role USER).
- Ứng dụng gửi request POST kèm theo các thông tin đã nhập và Token đến Server.
- API kiểm tra tính hợp lệ của Token thông qua Middleware trước khi xử lý logic nghiệp vụ.
- Để đảm bảo tính đồng nhất, Server thực hiện đồng thời hai thao tác:
 - + Tạo bản ghi thông tin shipper mới trong bảng shippers.

- + Cập nhật lại quyền hạn của người dùng thành SHIPPER trong bảng users.
- Trường hợp lỗi :Nếu biến số xe đã tồn tại trong hệ thống, Server trả về lỗi 400. Ứng dụng sẽ hiển thị thông báo lỗi cụ thể cho người dùng.
- Hoàn tất thành công: Khi các bước lưu dữ liệu hoàn tất, Server trả về mã 201. Ứng dụng hiển thị thông báo chúc mừng và chuyển hướng người dùng sang màn hình Shipper Dashboard.

2.8.15. Biểu đồ trình tự use case “Bật chế độ bắt đơn”



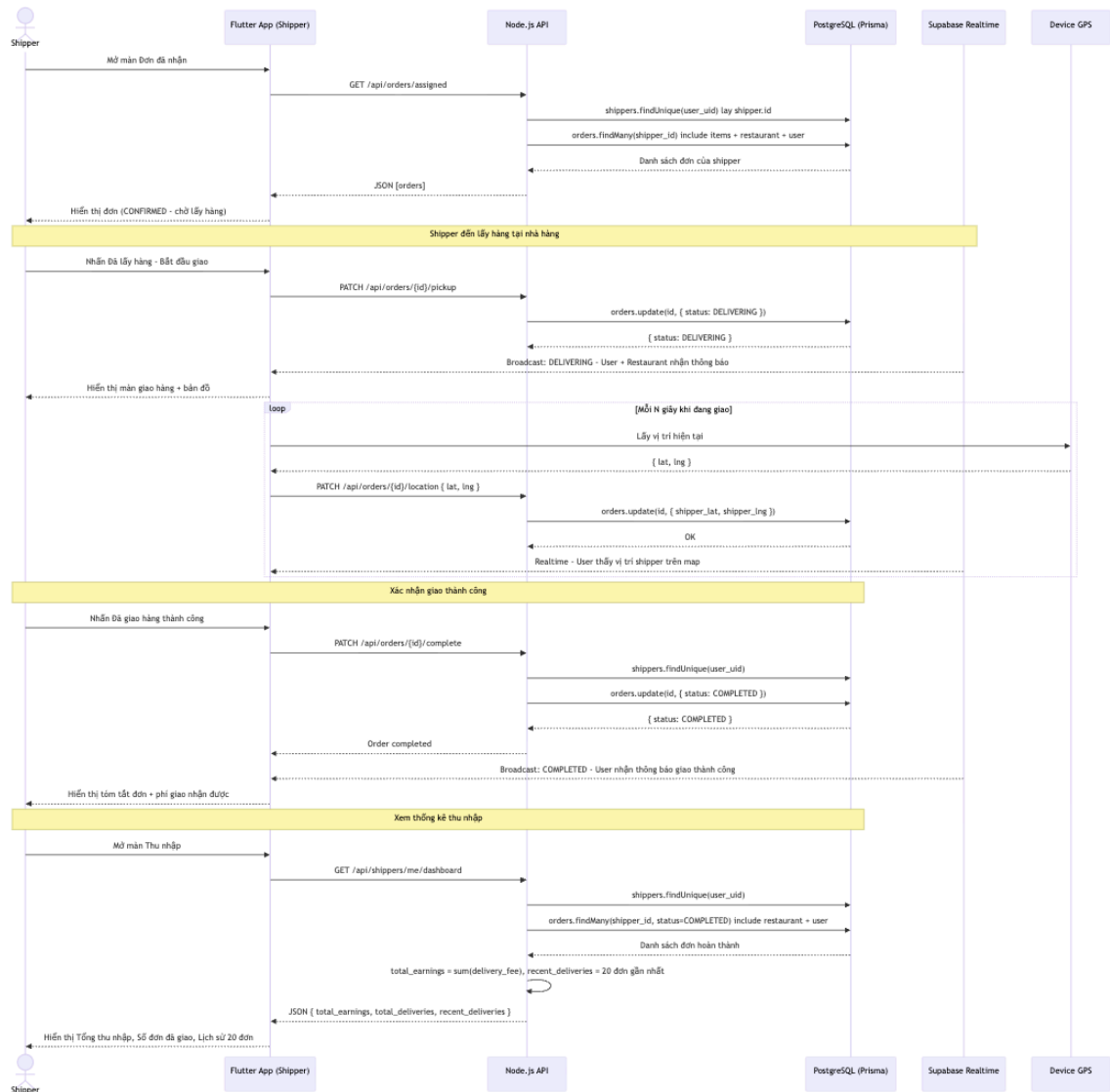
Hình 31. Biểu đồ trình tự bắt chế độ bắt đơn

Tìm kiếm và Cập nhật đơn hàng khả dụng:

- Shipper bật chế độ "Nhận đơn", ứng dụng Flutter lấy tọa độ GPS hiện tại từ thiết bị.

- Ứng dụng gửi request GET lấy những đơn hàng chưa được chọn kèm tọa độ đến Server.
- Truy vấn và Lọc:
 - + Server dùng Prisma lấy danh sách đơn hàng có trạng thái CONFIRMED và chưa có shipper.
 - + Tính toán khoảng cách từ Shipper đến nhà hàng bằng công thức.
 - + Lọc các đơn hàng trong bán kính 15km và sắp xếp theo thứ tự ưu tiên gần nhất.
- Hiển thị & Realtime: Ứng dụng hiển thị danh sách đơn khả dụng, đồng thời đăng ký (Subscribe) kênh orders:available của Supabase Realtime để nhận thông báo tức thời ngay khi có đơn hàng mới xuất hiện.

2.8.16. Biểu đồ trình tự use case “Giao thành công và xem thống kê thu nhập”



Hình 32. Biểu đồ trình tự giao hàng thành công và xem thống kê doanh thu

Lấy hàng và Bắt đầu giao:

- Shipper mở màn hình Đơn đã nhận. Ứng dụng gửi request GET để lấy danh sách các đơn hàng đang chờ lấy.
- Khi đến nhà hàng, Shipper nhấn "Đã lấy hàng". Ứng dụng gửi request PATCH để cập nhật trạng thái đơn hàng.
- Cập nhật trạng thái đơn thành DELIVERING trong Database và phát tín hiệu Broadcast qua Supabase Realtime để thông báo cho Khách hàng và Nhà hàng.

- Theo dõi vị trí (Loop): Trong suốt quá trình giao, ứng dụng tự động lấy tọa độ từ GPS và gửi request cập nhật vị trí mỗi N giây. Server cập nhật tọa độ liên tục vào Database để khách hàng có thể theo dõi shipper trên bản đồ theo thời gian thực.

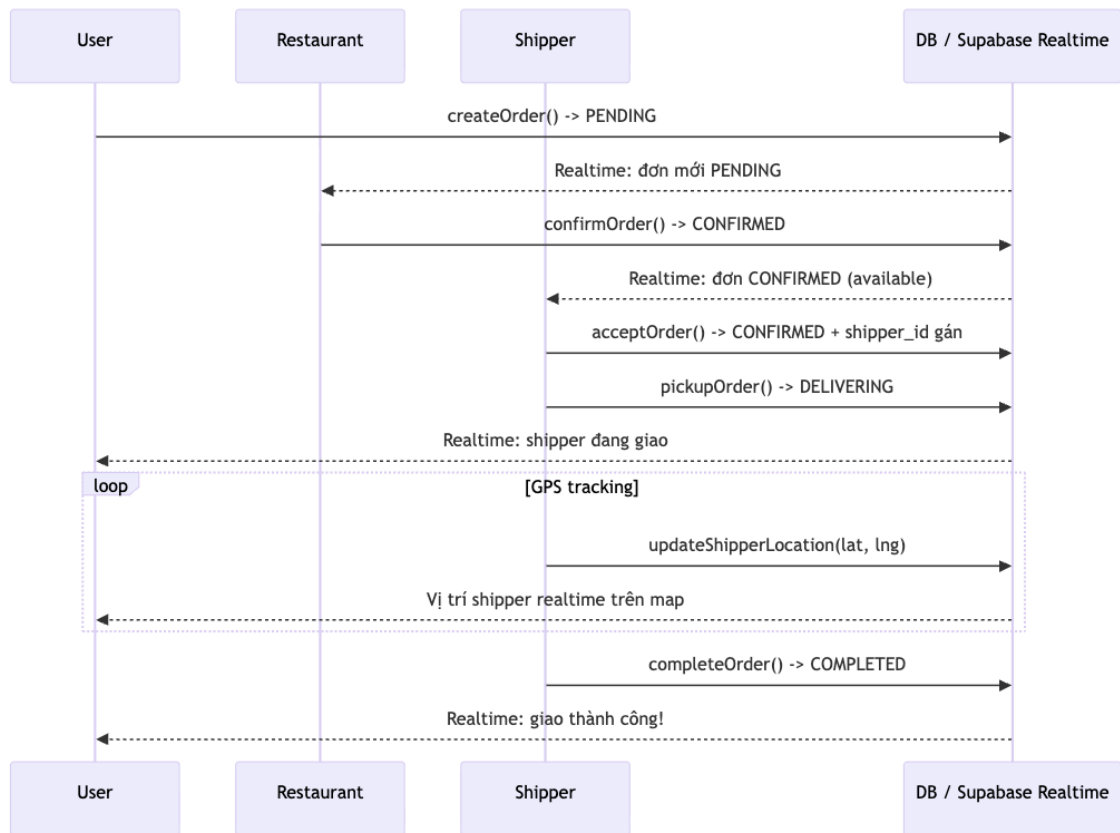
Xác nhận giao hàng thành công:

- Khi đến nơi, Shipper nhấn "Đã giao hàng thành công". Ứng dụng gửi request PATCH cập nhật trạng thái đơn hàng.
- Hệ thống cập nhật trạng thái đơn thành COMPLETED qua Prisma và gửi thông báo hoàn tất đến khách hàng.
- Ứng dụng hiển thị màn hình tổng kết đơn hàng kèm theo phí giao hàng mà shipper nhận được cho đơn đó.

Xem thống kê thu nhập:

- Shipper truy cập màn hình thu nhập. Ứng dụng gửi request GET lấy toàn bộ dữ liệu thu nhập cá nhân của shipper.
- Server truy vấn tất cả đơn hàng đã hoàn thành của shipper đó.
- Thực hiện tính toán: tổng phí giao hàng, tổng số đơn đã giao, và lấy danh sách 20 đơn gần nhất.
- Ứng dụng render dữ liệu thu nhập tổng quát, biểu đồ hoặc danh sách lịch sử giao hàng để shipper quản lý tài chính cá nhân.

2.8.17. Biểu đồ trình tự luồng trạng thái đơn hàng



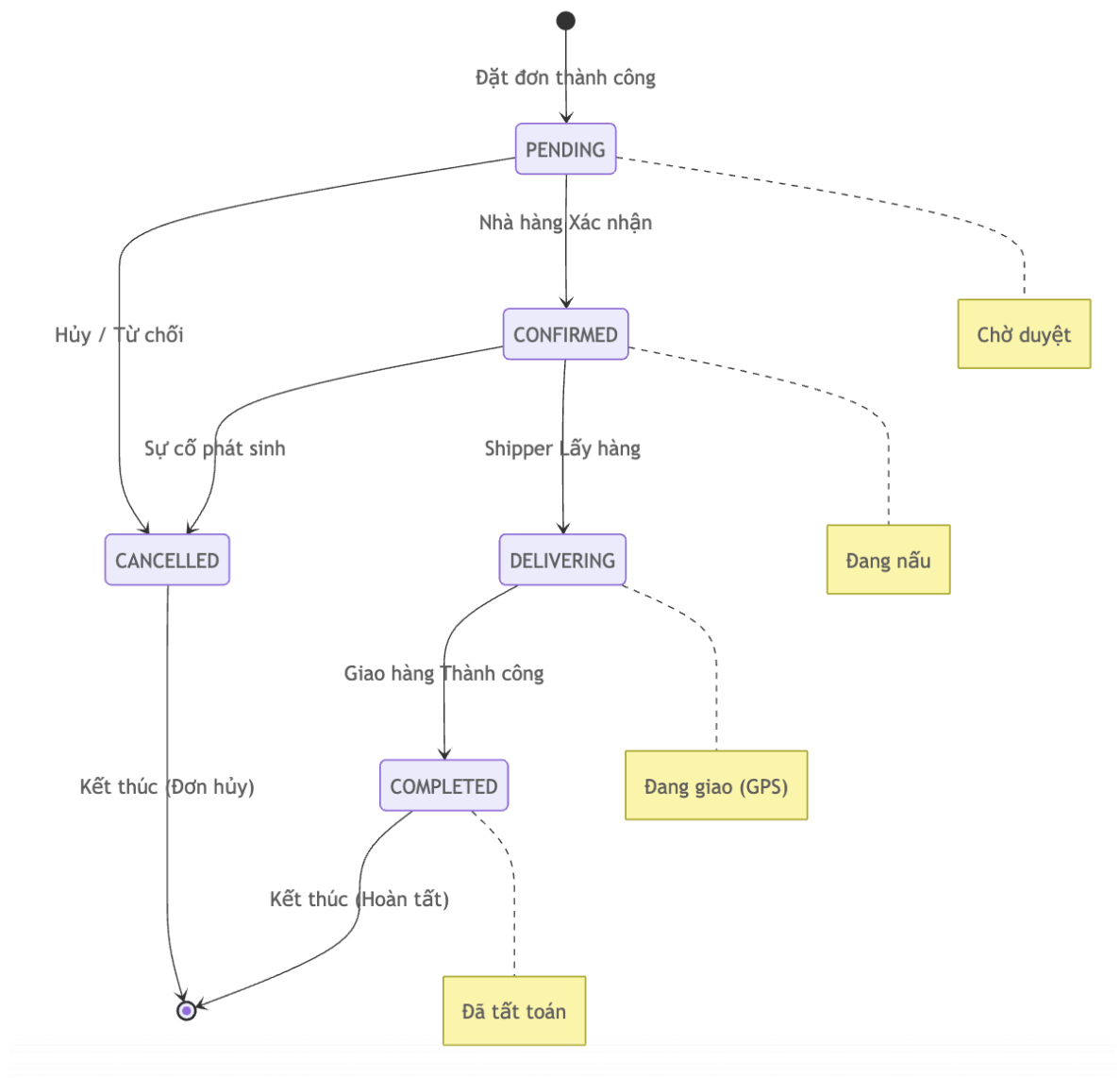
Hình 33. Biểu đồ trình tự luồng trạng thái đơn hàng

Luồng Tổng quát Quy trình Đặt và Giao hàng:

- Khởi tạo đơn hàng: Người dùng (User) thực hiện đặt hàng thông qua hàm `createOrder()`. Hệ thống ghi nhận đơn hàng vào Database với trạng thái ban đầu là `PENDING`.
- Thông báo cho Nhà hàng: Ngay khi có đơn hàng mới, Supabase Realtime phát tín hiệu đến ứng dụng của Nhà hàng để thông báo có đơn hàng đang chờ duyệt.
- Xác nhận đơn hàng: Nhà hàng kiểm tra và nhấn xác nhận qua hàm `confirmOrder()`. Trạng thái đơn hàng được cập nhật thành `CONFIRMED`.
- Điều phối Shipper: Sau khi đơn hàng được xác nhận, hệ thống phát sự kiện realtime thông báo đơn hàng đã sẵn sàng (`available`) cho các Shipper trong khu vực.

- Tiếp nhận đơn hàng: Shipper thực hiện `acceptOrder()`. Lúc này, hệ thống sẽ gán `shipper_id` vào đơn hàng để xác định người chịu trách nhiệm vận chuyển.
- Bắt đầu giao hàng: Khi Shipper đến lấy hàng tại quán, họ nhấn `pickupOrder()`. Trạng thái đơn hàng chuyển sang `DELIVERING`, đồng thời gửi thông báo realtime cho Người dùng biết đơn hàng đang trên đường giao.
- Theo dõi thời gian thực (GPS Tracking): Trong suốt quá trình di chuyển (vòng lặp loop):
 - + Ứng dụng Shipper liên tục cập nhật tọa độ (lat, lng) lên hệ thống qua hàm `updateShipperLocation`.
 - + Hệ thống truyền tải dữ liệu này realtime đến bản đồ của Người dùng để họ theo dõi vị trí chính xác của Shipper.
- Hoàn tất quy trình: Khi Shipper giao hàng thành công và nhấn `completeOrder()`, đơn hàng chuyển sang trạng thái cuối cùng là `COMPLETED`. Một thông báo xác nhận giao hàng thành công sẽ được gửi đến ứng dụng của Người dùng.

2.8.2. Biểu đồ trạng thái (State Diagram)



Hình 34. Biểu đồ trạng thái đơn hàng

Biểu đồ mô tả vòng đời của một đơn hàng từ khi được khởi tạo cho đến khi kết thúc, bao gồm các trạng thái chính và điều kiện chuyển đổi:

- **PENDING** (Chờ duyệt): Trạng thái mặc định ngay khi người dùng đặt đơn thành công. Tại đây, đơn hàng chờ nhà hàng kiểm tra và phản hồi.
- **CONFIRMED** (Đang nấu/Xác nhận): Xảy ra khi nhà hàng chấp nhận đơn hàng. Hệ thống chuyển sang giai đoạn chuẩn bị món ăn.

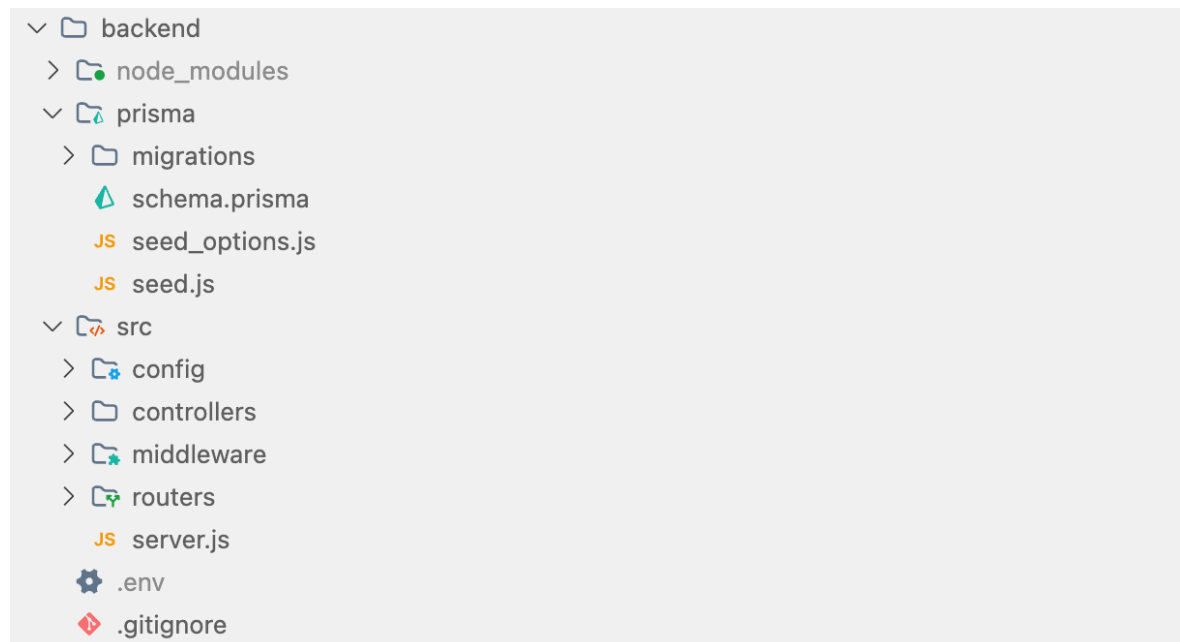
- DELIVERING (Đang giao/GPS): Trạng thái khi Shipper đã đến nhà hàng và lấy hàng thành công. Trong giai đoạn này, vị trí của Shipper được cập nhật liên tục qua GPS để khách hàng theo dõi.
- COMPLETED (Đã tất toán/Hoàn tất): Trạng thái cuối cùng của một đơn hàng thành công sau khi Shipper xác nhận đã giao hàng tận tay khách hàng.
- CANCELLED (Đã hủy): Trạng thái kết thúc không mong muốn. Đơn hàng có thể bị hủy từ:
 - + Trạng thái PENDING: Do người dùng chủ động hủy hoặc nhà hàng từ chối nhận đơn.
 - + Trạng thái CONFIRMED: Do các sự cố phát sinh ngoài ý muốn (hết nguyên liệu đột xuất, không tìm thấy shipper...).

CHƯƠNG III. THIẾT KẾ ỨNG DỤNG

3.1. Thiết kế Back-end

Để đảm bảo ứng dụng di động hoạt động ổn định và hiệu quả, hệ thống Backend được xây dựng trên nền tảng Node.js kết hợp với framework Express.js. Kiến trúc Backend được thiết kế theo tiêu chuẩn RESTful API, giúp phân tách rõ ràng các thành phần trong hệ thống, từ đó tăng khả năng mở rộng, bảo trì và tái sử dụng mã nguồn.

Hệ thống được tổ chức theo mô hình phân tầng (layered architecture), bao gồm các thành phần chính sau:



Hình 35. Cấu trúc thư mục backend

◆ Tầng Router (src/routers):

Tầng Router chịu trách nhiệm tiếp nhận và định tuyến các HTTP Request từ ứng dụng Flutter, bao gồm các phương thức như GET, POST, PATCH và DELETE.

Hệ thống được chia thành nhiều router riêng biệt theo từng chức năng, ví dụ:

- authRouter: xử lý các chức năng xác thực người dùng
- ordersRouter: xử lý các nghiệp vụ liên quan đến đơn hàng

- restaurantsRouter: quản lý thông tin nhà hàng
- shippersRouter: quản lý tài xế giao hàng

Việc tách riêng router giúp hệ thống dễ quản lý và mở rộng khi số lượng API tăng lên.

◆ Tầng Middleware (src/middleware)

Middleware đóng vai trò như một lớp trung gian xử lý trước khi request được chuyển đến controller. Đây cũng là lớp đảm bảo bảo mật cho hệ thống.

Chức năng chính:

- Xác thực người dùng (Authentication) thông qua JSON Web Token
- Kết hợp với Supabase Auth để kiểm tra tính hợp lệ của phiên đăng nhập
- Kiểm tra quyền truy cập (Authorization) đối với từng tài nguyên

◆ Tầng Controller (src/controllers)

Controller là nơi xử lý logic nghiệp vụ chính của hệ thống (Business Logic).

Tại đây, các request sau khi được xác thực sẽ được xử lý và trả về kết quả tương ứng.

Một số chức năng tiêu biểu:

- ordersController: xử lý logic đơn hàng, bao gồm tính toán khoảng cách bằng công thức Haversine
- cartController: xử lý giỏ hàng và gộp các tùy chọn món ăn
- authController: xử lý xác thực, bao gồm phát sinh và kiểm tra mã OTP

Việc tách biệt controller giúp code rõ ràng, dễ kiểm thử và bảo trì.

◆ Tầng Database / ORM (src/config/prisma)

Tầng này đảm nhiệm việc giao tiếp với cơ sở dữ liệu PostgreSQL thông qua Prisma ORM.

Ưu điểm:

- Truy vấn dữ liệu an toàn hỗ trợ tự động sinh schema và migration
- Hỗ trợ transaction đảm bảo tính toàn vẹn dữ liệu
- Nhờ đó, hệ thống giảm thiểu lỗi truy vấn và tăng độ tin cậy khi xử lý các nghiệp vụ phức tạp.

◆ Nhận xét

- Kiến trúc Backend được thiết kế theo mô hình phân tầng giúp:
- Tăng tính modular và dễ mở rộng
- Dễ dàng bảo trì và nâng cấp hệ thống
- Đảm bảo tính bảo mật thông qua middleware
- Tối ưu hiệu năng và độ tin cậy nhờ Prisma ORM

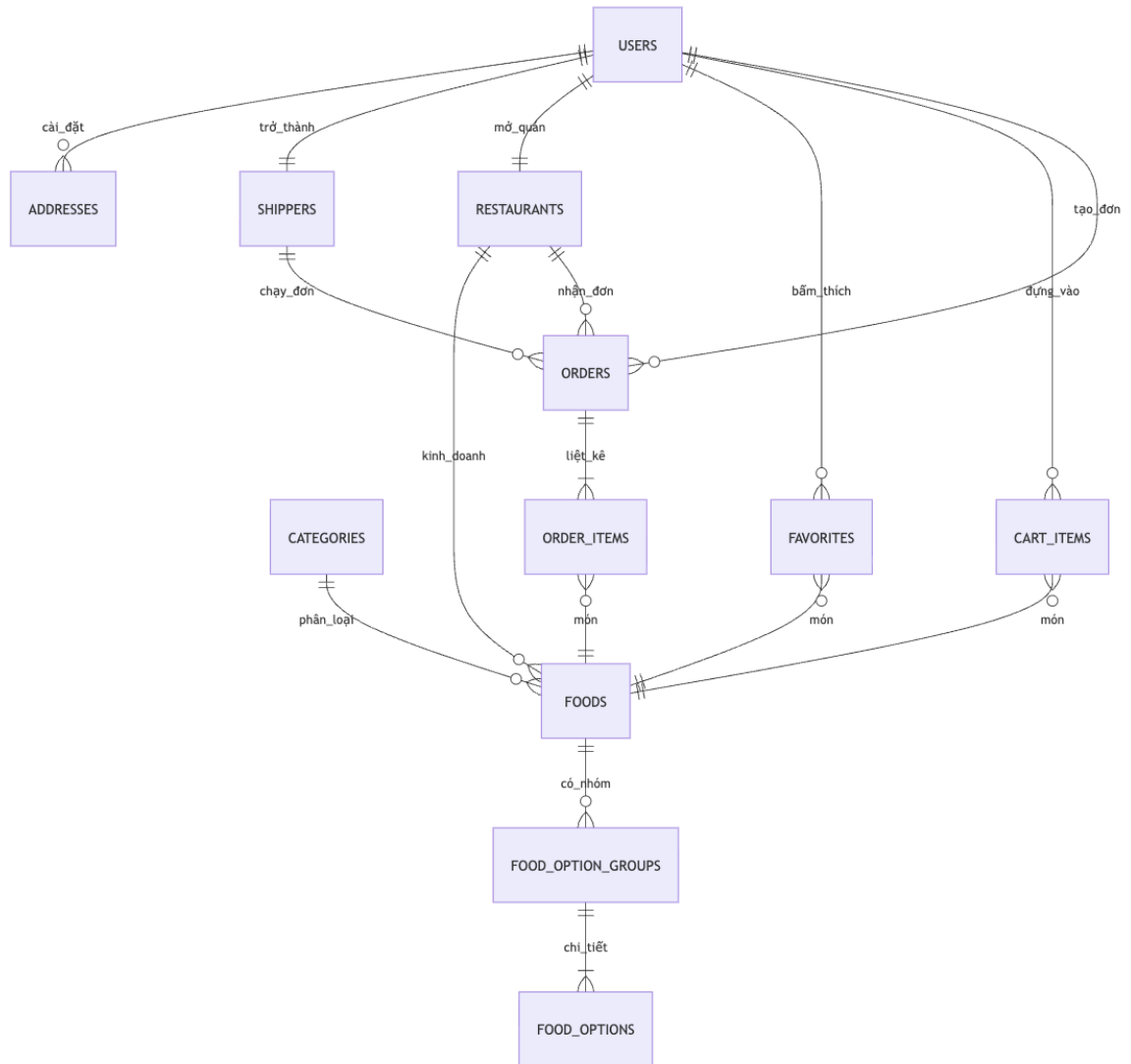
3.2. Thiết kế mô hình dữ liệu

Hệ thống cơ sở dữ liệu được thiết kế và triển khai trên hệ quản trị cơ sở dữ liệu PostgreSQL. Toàn bộ dữ liệu được tổ chức theo mô hình quan hệ với tổng cộng 13 bảng, liên kết chặt chẽ với nhau thông qua thực thể trung tâm là bảng users.

Thiết kế này đảm bảo:

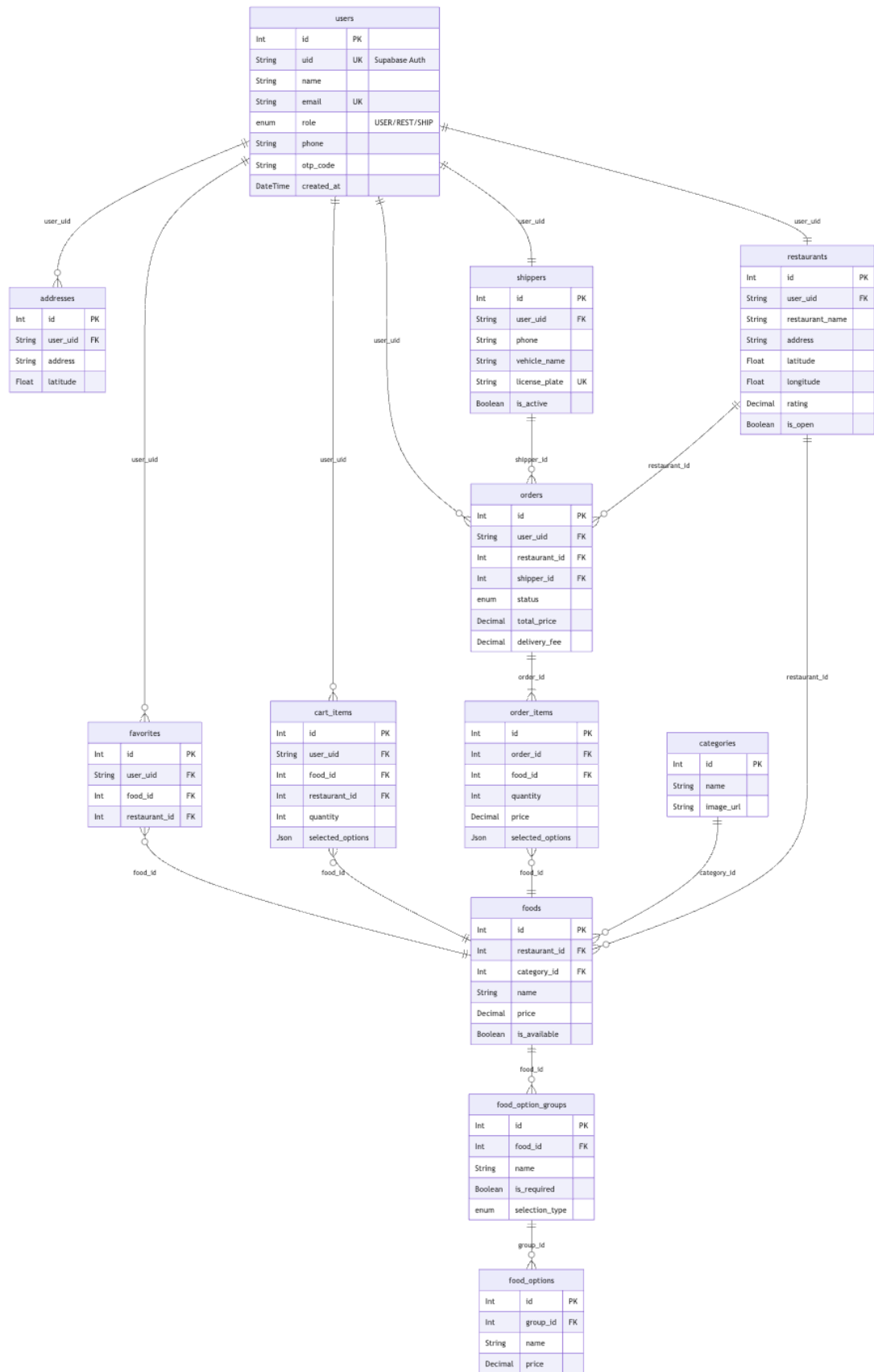
- Tính nhất quán dữ liệu
- Khả năng mở rộng trong tương lai
- Hỗ trợ hiệu quả cho các nghiệp vụ như quản lý người dùng, đơn hàng, nhà hàng và vận chuyển

3.2.1. Biểu đồ ERD mức logic



Hình 36. Biểu đồ ERD mức logic

3.2.2. Biểu đồ ERD mức vật lý



Hình 37. Biểu đồ ERD mức vật lý

3.3. Thiết kế giao diện hệ thống

Thiết kế giao diện người dùng và trải nghiệm người dùng đóng vai trò quan trọng trong việc xây dựng một ứng dụng giao đồ ăn hiệu quả và có tính cạnh tranh. Đối với hệ thống này, giao diện được phát triển bằng framework Flutter, cho phép triển khai đồng nhất và đảm bảo hiệu năng trên nhiều nền tảng di động khác nhau. Tổng thể giao diện được xây dựng dựa trên các nguyên tắc thiết kế cốt lõi nhằm đảm bảo tính thẩm mỹ, hiệu quả sử dụng và thân thiện với người dùng.

Trước hết, hệ thống tuân theo nguyên tắc tối giản và rõ ràng. Các thành phần giao diện được tinh giản, loại bỏ những yếu tố không cần thiết nhằm giúp người dùng tập trung vào nội dung chính, đặc biệt là hình ảnh món ăn và các nút. Các thành phần như container, ô nhập liệu và nút bấm được chuẩn hóa thông qua hệ thống biến màu sắc và kiểu dáng thống nhất, góp phần giảm tải nhận thức và nâng cao tốc độ thao tác.

Bên cạnh đó, ứng dụng sử dụng hệ thống màu sắc chủ đạo mang tính nhận diện cao, với tông màu xanh lá làm màu nhấn, tạo cảm giác tươi mới, an toàn và thân thiện với người dùng. Hệ thống cũng hỗ trợ chế độ hiển thị linh hoạt, cho phép tự động chuyển đổi giữa giao diện sáng và tối dựa trên cài đặt của thiết bị, từ đó cải thiện trải nghiệm thị giác và giảm mỏi mắt khi sử dụng trong điều kiện ánh sáng yếu. Kiểu chữ và các biểu tượng được thiết kế theo phong cách hiện đại, bo góc mềm mại, đảm bảo tính trực quan và dễ nhận diện chức năng.

Về trải nghiệm người dùng, hệ thống hướng đến việc tối ưu hóa quy trình sử dụng bằng cách giảm thiểu số lượng thao tác cần thiết để hoàn thành một chức năng. Quy trình đặt hàng được thiết kế liền mạch, hạn chế chuyển đổi màn hình không cần thiết. Đồng thời, hệ thống tích hợp cơ chế cập nhật dữ liệu theo thời gian thực thông qua Supabase Realtime, cung cấp thông tin liên tục về

trạng thái đơn hàng và vị trí giao hàng, giúp người dùng dễ dàng theo dõi và tăng mức độ tin cậy trong quá trình sử dụng.

Tổng thể, hệ thống được tổ chức thành ba ứng dụng (hoặc ba phân hệ giao diện chính), tương ứng với ba nhóm đối tượng người dùng khác nhau:

- Ứng dụng dành cho khách hàng (Customer App): Cho phép người dùng tìm kiếm và khám phá món ăn, thực hiện đặt hàng, thanh toán và theo dõi trạng thái đơn hàng theo thời gian thực. Giao diện được thiết kế tập trung vào yếu tố trực quan và hình ảnh nhằm tăng tính hấp dẫn.
- Ứng dụng dành cho nhà hàng (Restaurant App): Hỗ trợ quản lý thực đơn, tiếp nhận và xử lý đơn hàng, đồng thời cung cấp các báo cáo doanh thu. Giao diện ưu tiên sự rõ ràng và khả năng thao tác nhanh, chính xác.
- Ứng dụng dành cho tài xế (Shipper App): Tối ưu cho việc nhận đơn, định vị và theo dõi lộ trình giao hàng. Giao diện được thiết kế phù hợp với điều kiện sử dụng ngoài trời, đảm bảo khả năng hiển thị rõ ràng và dễ thao tác.

Việc phân tách các phân hệ một cách rõ ràng, đồng thời duy trì ngôn ngữ thiết kế thống nhất giúp hệ thống đáp ứng tốt nhu cầu đặc thù của từng nhóm người dùng mà vẫn đảm bảo tính nhất quán về nhận diện thương hiệu.

3.3.1. Giao diện phía khách hàng

Giao diện khách hàng được thiết kế tập trung vào việc khám phá món ăn và trải nghiệm đặt hàng mượt mà. Khách hàng thường đưa ra quyết định dựa trên hình ảnh và đánh giá, do đó giao diện cần tối ưu hóa không gian hiển thị hình ảnh và thông tin từ nhà hàng.

3.3.1.1. Màn hình đăng nhập và đăng kí

The image displays two mobile application screens for user authentication. Both screens have a status bar at the top showing the time as 13:30, a back arrow, and a battery level of 95%.

Left Screen (Login):

- Header: "Fresh & Delicious" with the tagline "Discovery your next favotire meal".
- Form fields: "Email Address" (with an envelope icon) and "Password" (with a lock icon and a toggle for visibility).
- Buttons: "Sign In" (green) and "Continue with Google" (white with Google logo).
- Link: "Forget Password?" (green text).
- Footer: "Don't have an account? [Create Account](#)" (green text).

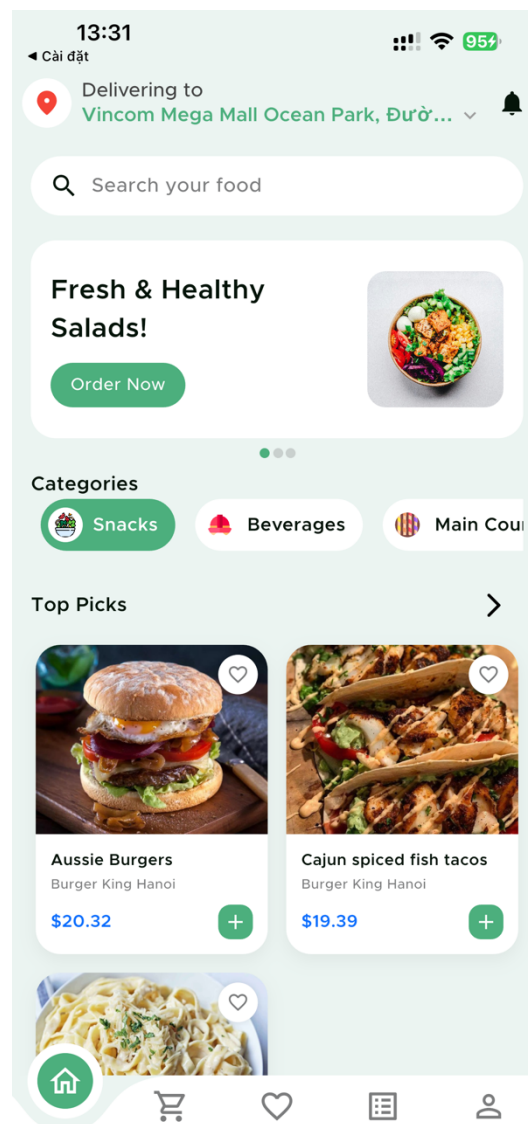
Right Screen (Register):

- Header: "Register" with the instruction "Enter Your Personal Information".
- Form fields: "Username", "Email", "Name", "Phone Number", "Password", and "Confirm Password" (each with a toggle for visibility).
- Buttons: "Register" (green) and "Login" (green text link).
- Footer: "Already have an account? [Login](#)" (green text).

Hình 38. Màn hình đăng nhập và đăng kí

Cung cấp giao diện xác thực qua email và mật khẩu hoặc qua các dịch vụ bên thứ ba. Biểu mẫu lấy thông tin được thiết kế thân thiện, báo lỗi rành mạch.

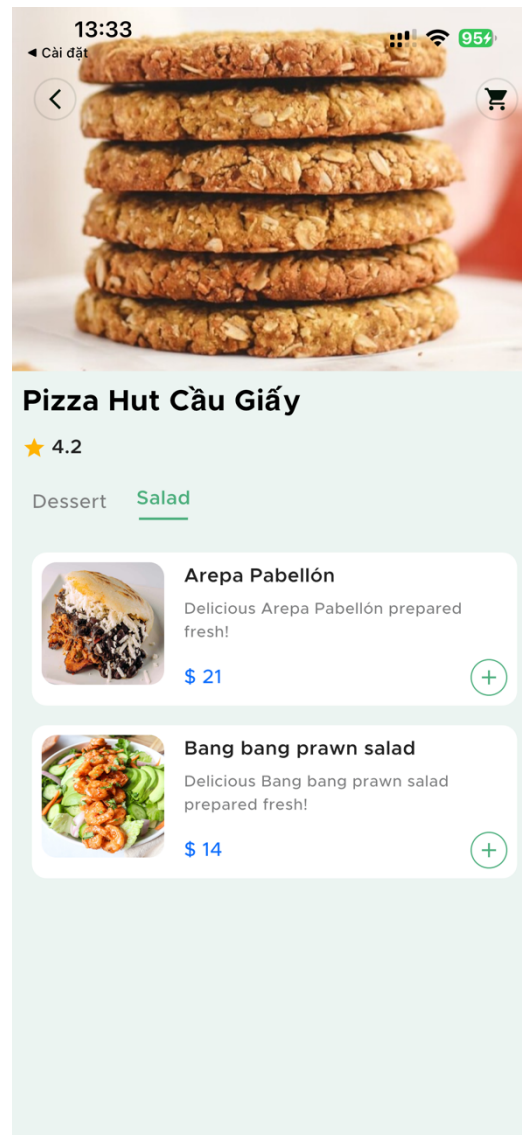
3.3.1.2. Màn hình trang chủ



Hình 39. Màn hình trang chủ

Hiện thị thanh tìm kiếm nổi bật ở phía trên, theo sau là các phân mục món ăn, banner hiển thị các chương trình khuyến mãi, và danh sách các nhà hàng đề xuất.

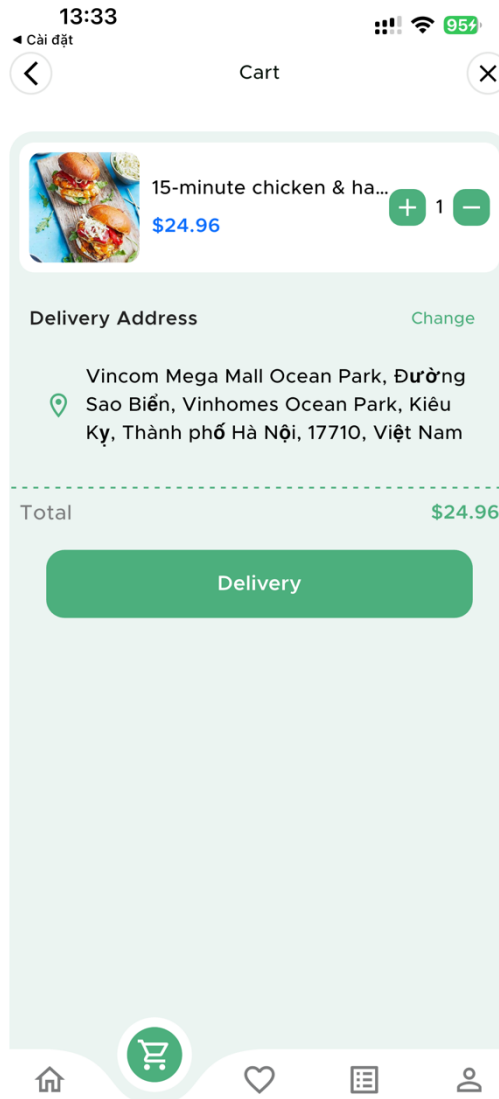
3.3.1.3. Màn hình chi tiết nhà hàng và thực đơn



Hình 40. Màn hình chi tiết nhà hàng và thực đơn

Khi người dùng chọn vào 1 nhà hàng, giao diện sẽ cuộn đến phần thực đơn được phân chia theo danh mục (đồ ăn chính, nước uống,...). Mỗi món ăn đều có ảnh minh họa, tên, mô tả ngắn, giá cả và nút thêm vào giỏ hàng.

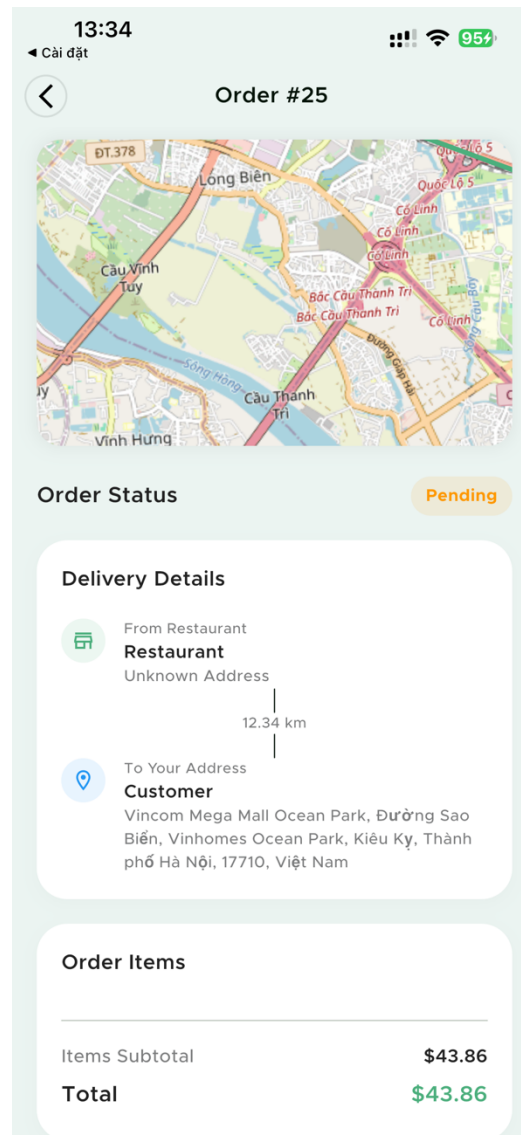
3.3.1.4. Màn hình giỏ hàng và thanh toán



Hình 41. Màn hình giỏ hàng và thanh toán

Tóm tắt các món hàng đã chọn, cho phép thay đổi số lượng. Hệ thống tính toán và hiển thị tự động tổng tiền món, phí giao hàng và tổng cộng thanh toán. Khách hàng có thể chọn địa điểm giao (từ bản đồ) và hình thức thanh toán.

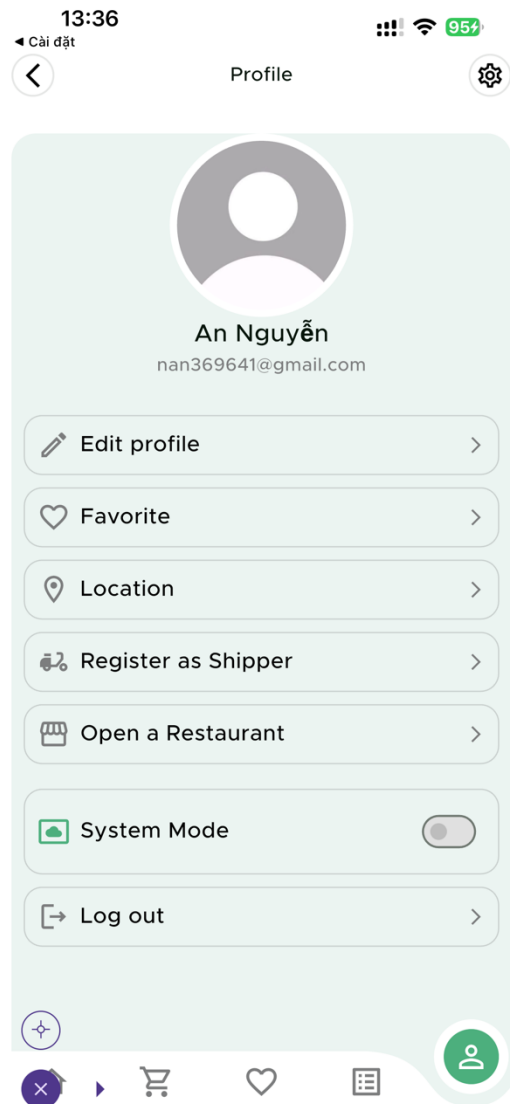
3.3.1.5. Màn hình theo dõi đơn hàng



Hình 42. Màn hình theo dõi đơn hàng

Đây là màn hình quan trọng, hiển thị tiến trình của đơn hàng theo thời gian. Bản đồ tích hợp giúp khách hàng theo dõi vị trí trực tiếp của Shipper.

3.3.1.6. Màn hình hồ sơ



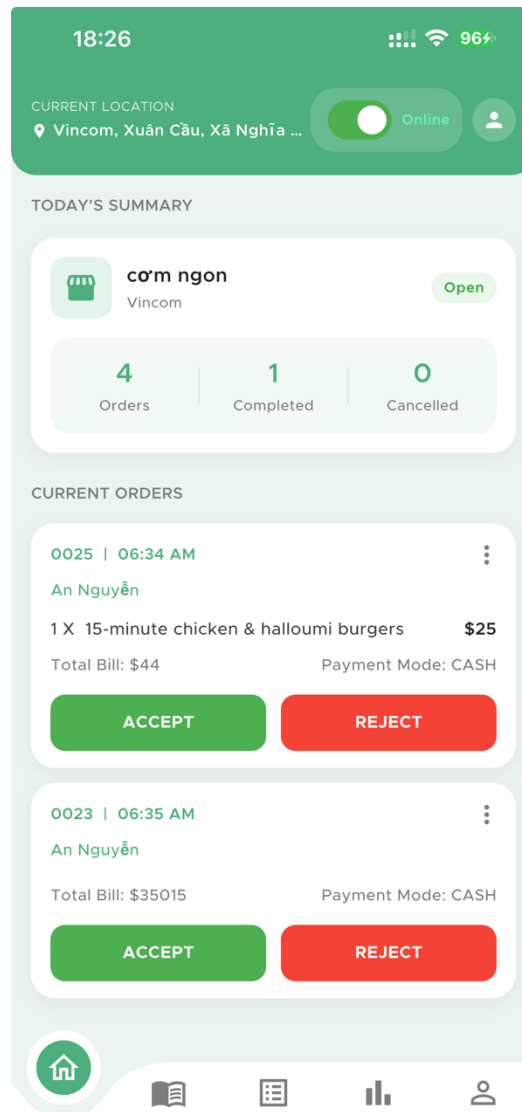
Hình 43. Màn hình hồ sơ

Nơi khách hàng quản lý thông tin tài khoản, danh sách địa chỉ lưu sẵn, và lịch sử đơn hàng.

3.3.2. Giao diện phía nhà hàng

Đối với nhà hàng, môi trường làm việc thường bận rộn và nhiều áp lực, không có nhiều thời gian thao tác. Do đó, giao diện cho đối tác nhà hàng đòi hỏi sự tinh gọn, chữ to, dễ thao tác và có thông báo rõ rệt khi có đơn mới.

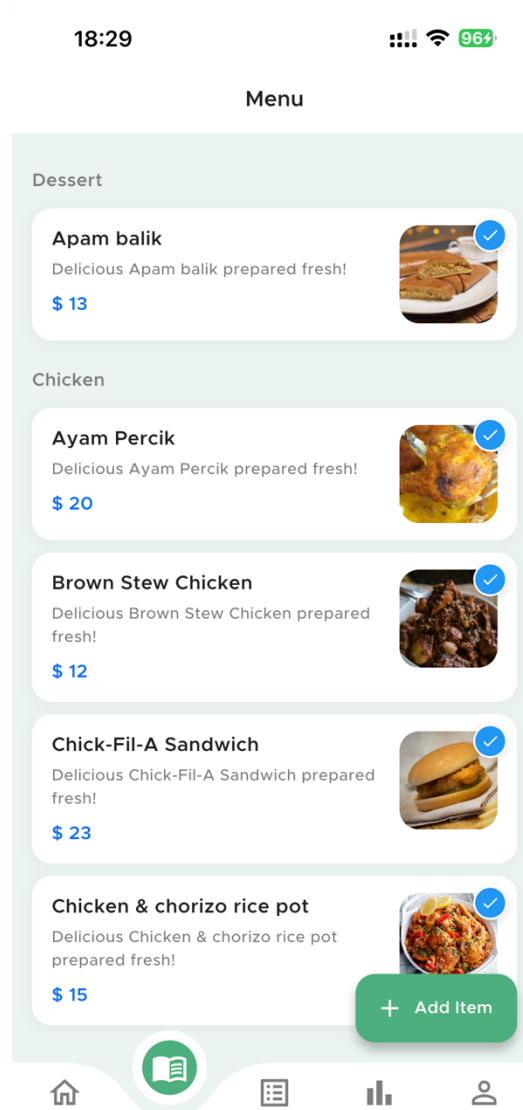
3.3.2.1. Màn hình quản lý đơn hàng



Hình 44. Màn hình quản lý đơn hàng

Là trung tâm điều khiển chính. Màn hình phân loại đơn hàng theo trạng thái "Đơn mới", "Đang chuẩn bị", và "Sẵn sàng giao" hoặc "Lịch sử". Các nút bấm "Chấp nhận" hoặc "Từ chối" được thiết kế lớn và phân biệt màu sắc rõ ràng để tránh thao tác nhầm.

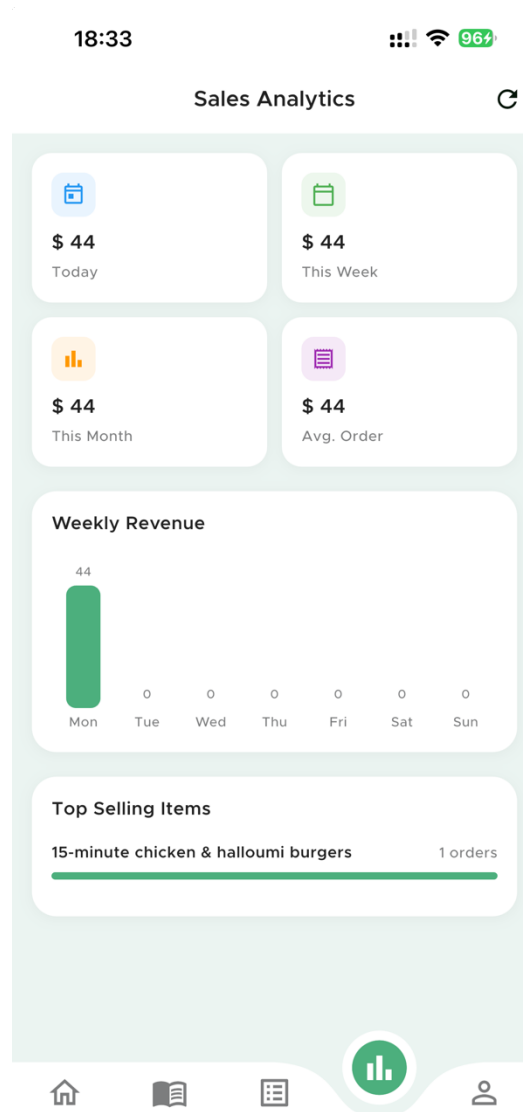
3.3.2.2. Màn hình quản lý thực đơn



Hình 45. Màn hình quản lý thực đơn

Giao diện cho phép nhà hàng bật/tắt trạng thái hết hàng của từng món ăn một cách nhanh chóng chỉ bằng một nút gạt. Chủ cửa hàng cũng có thể thêm mới hoặc cập nhật giá món ăn.

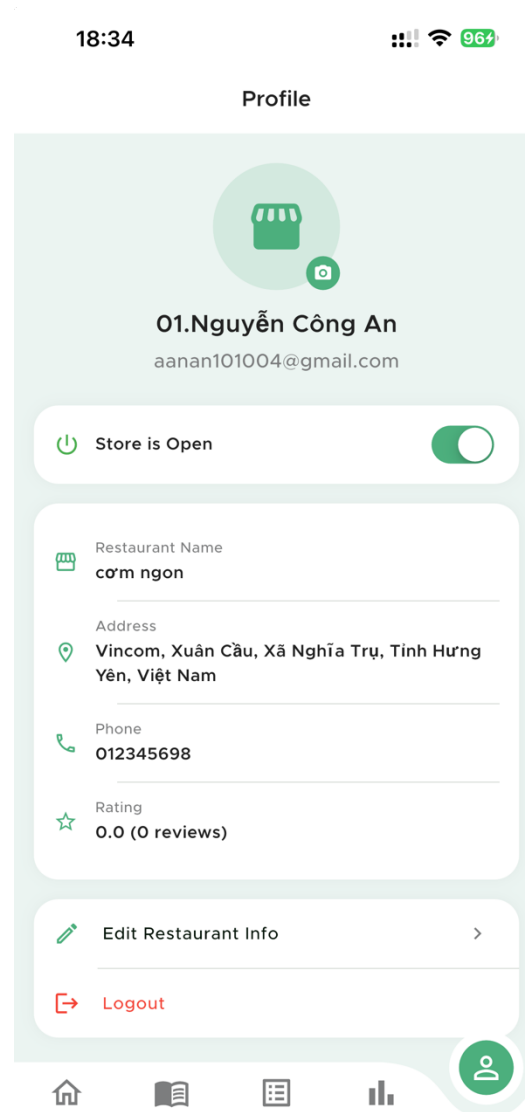
3.3.2.3. Màn hình thống kê thu nhập



Hình 46. Màn hình thống kê thu nhập

Bảng dashboard đơn giản hiển thị số lượng đơn hàng đã bán, doanh thu theo ngày/tuần bằng các biểu đồ trực quan, giúp nhà hàng đánh giá tình hình kinh doanh tổng quan.

3.3.2.4. Màn hình hồ sơ cửa hàng



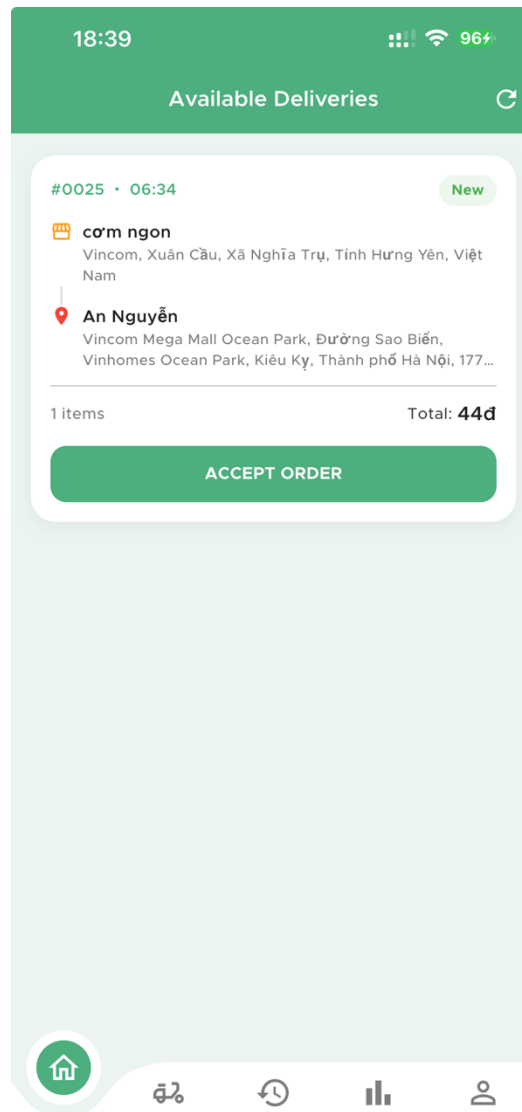
Hình 47. Màn hình hồ sơ cửa hàng

Cung cấp biểu mẫu để cập nhật tên quán, thay đổi trạng thái đóng/mở cửa của quán, địa chỉ chính xác trên bản đồ.

3.3.3. Giao diện phía shipper

Giao diện của Shipper được tùy chỉnh đặc biệt cho việc di chuyển liên tục ngoài trời, đòi hỏi chế độ tối hoặc độ tương phản siêu cao để nhìn rõ dưới ánh nắng, cùng các nút bấm để thao tác dễ dàng khi đang lái xe hoặc đeo găng tay.

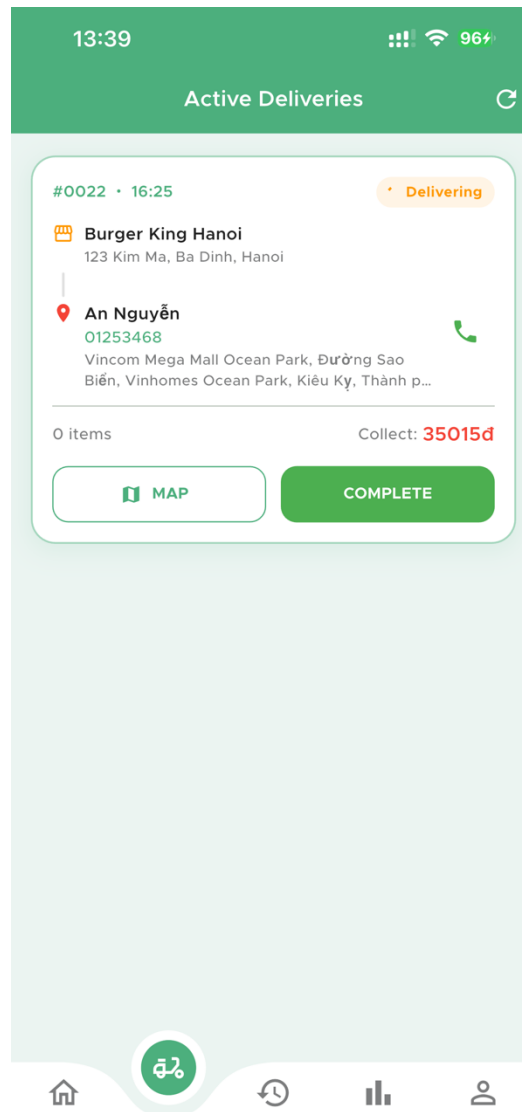
3.3.3.1. Màn hình danh sách đơn hàng đang chờ



Hình 48. Màn hình danh sách đơn hàng đang chờ

Khi có đơn hàng phát sinh, màn hình hiển thị thể thông báo nổi bật có tích hợp cả âm thanh, cung cấp thông tin: điểm lấy hàng (Khoảng cách từ vị trí hiện tại đến nhà hàng), điểm giao hàng, quãng đường tổng cộng và phí giao hàng ước tính.

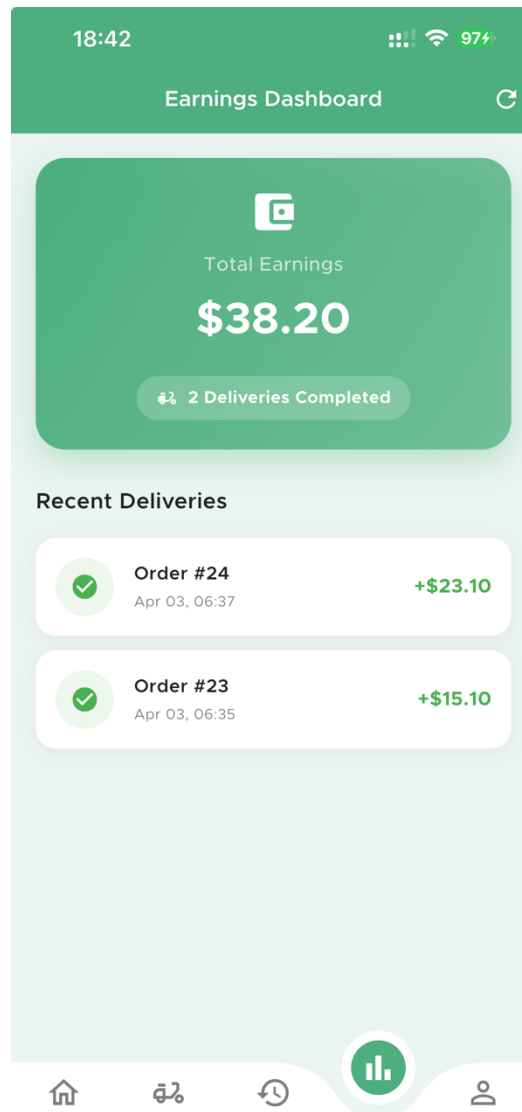
3.3.3.2. Màn hình chi tiết đơn hàng



Hình 49. Màn hình chi tiết đơn hàng

Hiển thị các đơn hàng đang được tài xế thực hiện giao. Mỗi đơn được trình bày dưới dạng thẻ thông tin bao gồm nhà hàng, khách hàng, địa chỉ giao hàng, trạng thái đơn và số tiền cần thu. Người dùng có thể nhanh chóng mở bản đồ để điều hướng hoặc xác nhận hoàn thành đơn hàng thông qua các nút chức năng trực tiếp trên giao diện.

3.3.3.3. Màn hình thống kê thu nhập



Hình 50. Màn hình thống kê thu nhập

Cung cấp tổng quan thu nhập của tài xế, hiển thị tổng số tiền đã kiếm được cùng số lượng đơn hàng hoàn thành. Ngoài ra, hệ thống còn liệt kê các đơn giao gần đây kèm thời gian và khoản thu nhập tương ứng, giúp người dùng dễ dàng theo dõi hiệu suất làm việc và quản lý doanh thu cá nhân.

CHƯƠNG IV. CÀI ĐẶT VÀ KIỂM THỬ

4.1. Cài đặt

4.1.1. Cài đặt Back-end

Hệ thống Back-end là bộ não trung tâm xử lý dữ liệu của toàn bộ ứng dụng giao đồ ăn, tính toán khoảng cách cước phí và phát luồng tín hiệu (Realtime).

4.1.1.1. Cài đặt môi trường và công nghệ sử dụng

- Môi trường thực thi (Runtime): Cài đặt Node.js môi trường cục bộ . Tại thư mục gốc của Node.js, tệp tin định dạng package.json được khởi tạo bằng lệnh `npm init` với cấu hình `"type": "module"` để hỗ trợ cú pháp ES6 (import/export), giúp mã nguồn hiện đại và an toàn hơn.

- Các thư viện cốt lõi : Quá trình cài đặt sử dụng lệnh “`npm install express prisma @prisma/client jsonwebtoken bcryptjs dotenv`”.

+ `express`: Khởi tạo máy chủ HTTP linh hoạt.

+ `@prisma/client` và `prisma`: Trình ánh xạ đối tượng ORM Type-safe cho phép khởi tạo các truy vấn thao tác tới PostgreSQL mà không cần viết SQL thuần túy phức tạp.

+ `jsonwebtoken` và `bcryptjs`: Tiêu chuẩn hóa quy trình bảo mật. Mật khẩu người dùng trước khi lưu xuống DB bị băm (Hash) một chiều bằng mức độ muối (salt) của `bcryptjs`. Token giao tiếp sử dụng chuẩn JWT không trạng thái (Stateless).

- Cơ sở dữ liệu đám mây: Khởi tạo project mới trên nền tảng Supabase. Lấy các cặp khóa Connection String cung cấp sẵn (bao gồm chuỗi kết nối gộp pool `DATABASE_URL` và chuỗi trực tiếp `DIRECT_URL`). Khai báo các chuỗi này vào tệp tin `.env` đóng. Tiếp theo, sử dụng lệnh `npx prisma db push` để Prisma tự động phân tích tệp `schema.prisma` và ánh xạ tạo thành 13 bảng vật lý trên nền tảng Supabase.

4.1.1.2. Mô hình kiến trúc

Cấu trúc cây thư mục Back-end được cài đặt theo mô hình MVC tách biệt trách nhiệm chặt chẽ, hỗ trợ khả năng phân tán mã nguồn.

- Thư mục prisma/: Chứa tệp schema.prisma định nghĩa toàn bộ mô hình dữ liệu (Users, Foods, Orders, Restaurants...) với các ràng buộc ngoại (Foreign Key Constraint) và khóa phụ thuộc. Tại đây chứa các script Seed (khởi tạo dữ liệu mẫu đầu vào).
- Thư mục src/routers/: Định nghĩa luồng dẫn API API Routing. Chịu trách nhiệm phân loại phương thức HTTP (GET, POST, PUT, DELETE).
- Thư mục src/controllers/: Tập trung các logic nghiệp vụ (Business Logic) phức tạp nhất. Phân tách thành: authController (Đăng nhập, đăng ký), cartController (Thêm món, cộng dồn giá), và quan trọng nhất là orderController dùng để xử lý quy trình khởi tạo đơn giao hàng và tìm tài xế dựa trên công thức bản đồ thực tế.
- Thư mục src/middleware/: Cài đặt các hàng rào chặn request rác. Tệp auth.js giải mã chuỗi Bearer Token từ Headers của Client; nếu JWT hết hạn, máy chủ lập tức từ chối và phản hồi trạng thái HTTP 401 Unauthorized.

4.1.1.3. Kết quả đạt được

- Lệnh npm start hay npm run dev thi hành thành công tệp server.js, máy chủ lắng nghe tại biến môi trường PORT=3000.
- PostgreSQL vận hành trơn tru khi thực thi logic.

4.1.2. Cài đặt Front-end

4.1.2.1. Cấu hình môi trường

Để xây dựng App đồng nhất trên cả 2 nền tảng iOS và Android, hệ thống áp dụng bộ khung phát triển Flutter SDK:

- Thiết lập hệ thống: Hệ thống khởi chạy lệnh flutter create delivery_apps. Cấu trúc tệp tin pubspec.yaml được quản lý chặt chẽ để thêm các packages quan trọng. Lệnh flutter pub get được sử dụng để đồng bộ hóa mã nguồn mã mở.
- Kiểm soát trạng thái hệ thống (State Management): Sử dụng kiến trúc provider. Lý do lựa chọn Provider là nhờ khả năng tách bạch tầng Logic khỏi tầng View (Giao diện), cho phép chia sẻ mượt mà trạng thái của "Giỏ đồ ăn" hoặc "Thông tin User đang đăng nhập" đến mọi màn hình bất kỳ mà không cần truyền biến trung gian liên tục.
- Định tuyến luồng màn hình (Go Router): go_router thay thế định tuyến mặc định để thiết lập cơ chế chuyển màn hình hiện đại. Có khả năng khóa ứng dụng (Redirect), chặn người dùng chưa đăng nhập tiếp cận màn hình Đặt Hàng.
- Tích hợp Bản đồ và Định vị: Bộ ba thư viện flutter_map, geolocator và latlong được tích hợp. Khi Shipper cấp quyền truy cập vị trí, hệ thống ngay lập tức định vị kinh độ, vĩ độ và khởi họa lại bản đồ dẫn đường (Routing Map) mà không cần dùng đến Google API trả phí.

4.1.2.2. Tổ chức giao diện và cấu trúc hệ thống

App được thiết kế tuân theo hệ thống nhận diện Sáng/Tối (Light/Dark Mode) tự động dựa trên nhân app_color.dart. Cấu trúc quy mô lớn chia làm:

- lib/core/Chứa định nghĩa kiểu chữ metropolis tùy biến, các biến hằng biểu diễn màu xanh lá chủ đạo, lớp giao diện mạng, mạng lưới truy vấn HTTP Client và quản lý Session đăng nhập cục bộ.
- lib/features/ Tách ra riêng biệt 3 tính năng lớn:
 - + Phân hệ Customer: UI thiên về hình ảnh lớn (Cached Network Image), thẻ nhà hàng và giỏ hàng lướt ngang.
 - + Phân hệ Restaurant: Bảng điều khiển tối giản chức năng bật/tắt món. Phân vùng lớn để bấm Xác nhận đơn hàng tránh nhầm lẫn.

- + Phân hệ Shipper: Chế độ bản đồ tối. Vuốt trượt mượt mà (Slidable) để kết thúc tiến trình chuyển xe.

4.1.2.3. Kết quả đạt được của hệ thống Front-end

- Ứng dụng được chạy thành công với dung lượng cực kì tối ưu, không có lỗi xung đột thư viện
- Giao diện kết xuất đồ họa mượt không xảy ra hiện tượng tràn phần tử vượt quá ranh giới giới hạn khung hình.

4.2. Kiểm thử

4.2.1. Kế hoạch kiểm thử

Để đảm bảo hệ thống ứng dụng giao đồ ăn đi vào hoạt động trơn tru, đáp ứng đủ các tiêu chuẩn kỹ thuật cũng như tiêu chuẩn nghiệp vụ, giai đoạn kiểm thử được lên kế hoạch và phân bổ một cách bài bản, chặt chẽ dựa trên 4 yếu tố cốt lõi:

◆ Mục tiêu kiểm thử:

- Xác minh tính đúng đắn (Verification): Đảm bảo tất cả các chức năng cốt lõi (Đăng nhập, Thêm giỏ hàng, Thanh toán, Chuyển phát đơn, v.v.) của 3 phân hệ (Khách, Nhà hàng, Shipper) đều hoạt động chính xác theo đặc tả yêu cầu ở các chương trước.
- Phát hiện và loại bỏ lỗi (Bug Tracking): Phát hiện sớm các lỗi hệ thống (Crash app, RenderFlex overflow) và các lỗi về bảo mật logic (Người dùng chưa ủy quyền mà truy cập được chức năng, v.v.).
- Đánh giá trải nghiệm người dùng (UX/UI & Performance): Đo lường tính trơn tru khi cuộn trang hiển thị món ăn, tính ổn định trong việc hiển thị bản đồ định vị, và tốc độ phản hồi (Latency) của thông báo thời gian thực (Realtime notification) giữa các thiết bị.

- ◆ Phạm vi kiểm thử (Scope of Testing) Quá trình kiểm thử tập trung rà soát toàn bộ các thành phần của hệ thống bao gồm:

- Phạm vi Back-end (Server side): Kiểm tra các điểm cuối API (API Endpoints), kiểm tra tính toàn vẹn của mô hình cơ sở dữ liệu trên Prisma, xác minh khả năng chặn request vô lý của Middleware auth.js.

- Phạm vi Front-end (Client side): Kiểm tra tính thân thiện của giao diện Flutter trên nhiều tỷ lệ màn hình thực tế (Responsive). Kiểm tra tính mạch lạc của thanh điều hướng Navigator.

- Phạm vi Tích hợp luồng (Integration Workflow): Rà soát dòng chảy (Flow) của một đơn hàng kể từ nút "Xác nhận đặt" của khách, truyền qua tài khoản "Nhà hàng", và chuyển tiếp đến màn hình bản đồ của "Tài xế".

- ◆ Môi trường kiểm thử (Testing Environment) Môi trường kiểm thử được cấu hình sát với điều kiện triển khai thực tế nhất có thể:

- Về phần cứng (Hardware):

+ Môi trường máy chủ cục bộ (Host Machine): Laptop/PC cá nhân đóng vai trò làm máy chủ biên dịch (Node.js/Nodemon) phục vụ qua mạng Local.

+ Môi trường Client giả lập và thiết bị thực: Sử dụng tính năng Android Emulator / iOS Simulator của máy trạm để test các độ phân giải ảo. Áp dụng test trên thiết bị điện thoại di động thực vật lý (iPhone/Android smartphone) để trải nghiệm cảm ứng và vượt ngoài thực tế.

- Về phần mềm & Công cụ (Software & Tools):

+ Hệ điều hành thiết bị: Android 11.0+ và iOS 15.0+.

+ Công cụ kiểm thử API: Sử dụng phần mềm Postman để bắn độc lập khối lệnh JSON lên máy chủ Node.js.

+ Bảng điều khiển đám mây: Sử dụng Supabase Dashboard để đối soát xem Database vật lý có thực sự ghi nhận thêm các Record thành công hay không.

- Điều kiện kết nối (Network): Thử nghiệm ứng dụng trong cả hai môi trường mạng tốc độ cao (Wi-Fi tiêu chuẩn gia đình) và môi trường mạng di động 4G để kiểm tra khả năng bắt lỗi mất kết nối (Network Error Handling).

- ♦ Chiến lược kiểm thử (Testing Strategy) Hệ thống sử dụng các chiến lược kiểm thử chính sau đây để bao phủ toàn bộ vòng đời ứng dụng:

- Chiến lược Kiểm thử Hộp đen (Black-box Testing): Đây là chiến lược chủ đạo ở lớp giao diện Front-end. Người kiểm thử không cần quan tâm đến code logic bên dưới, mà sẽ đóng vai thuần túy là Khách / Shipper / Chủ nhà hàng, nhập các dữ liệu từ bàn phím màn hình và quan sát sự phản hồi trực quan trên màn hình.

- Chiến lược Kiểm thử Tích hợp (Integration Testing): Đánh giá tính kết dính và tương tác chéo giữa hệ thống Client App và hệ thống Server API. Đảm bảo dữ liệu từ App đẩy lên được Database đọc, lưu trữ và phản hồi khớp mã Status 200/201.

- Chiến lược kiểm tra xác thực dữ liệu: Ở lớp back-end API, thử cố ý gửi các Payload không đầy đủ qua chuẩn RESTful để đo lường xem máy chủ có báo cáo bắt lỗi đúng trường thuộc tính cần thiết hay bị app crash.

4.2.2. Kịch bản kiểm thử

Bảng 34. Danh mục kịch bản kiểm thử

Test Case	Yêu cầu	Tiền điều kiện	Các bước thực hiện	Kết quả mong đợi	Trạng thái
01	Bảo mật cơ chế xác thực OTP (Reset Password /Login)	User nhập một Email đã tồn tại trong bảng users trên CSDL. đang ở Log in	1. Nhập email và bấm "Gửi OTP". 2. Nhập một mã OTP sai (hoặc OTP đã hết hạn). 3. Bấm Xác nhận.	Backend báo lỗi 4xx. Client bắt lỗi hiển thị Flushbar thông báo: "Mã OTP không hợp lệ hoặc đã hết". Không tiến vào màn hình chính.	Pass
02	Cơ chế phân luồng chuyển hướng dựa trên quyền	Tài khoản đang có role: 'SHIPPER'. Đang ở màn LoginView().	1. Đăng nhập thành công, có Token. 2. Lớp App Router gọi hàm lấy thông tin	Thay vì vào màn hình Khách, hàm tự động kiểm tra và đẩy chính xác sang màn hình shipper. Các phân hệ khác không được truy cập.	Pass
03	Ràng buộc tính đồng nhất giỏ hàng	Cart của khách đang chứa Phở của Nhà hàng A	1. Khách hàng lướt xem menu nhà hàng B 2. Chọn món com rang bấm nút add.	Controller Giỏ hàng phát hiện có sự sai lệch mã ID nhà hàng. Hiển thị hộp thoại cảnh báo: "Bạn có muốn xóa giỏ cũ để tạo giỏ của	Pass

				Nhà hàng mới không?". Khách không thể đặt cùng lúc 2 quán.	
05	Tương tác Realtime giữa Nhà hàng và Đơn mới	Máy của nhà hàng đang mở ở màn hình hiển thị các đơn hàng và sau khi người dùng bấm thành toán	1. Nhà hàng không tải lại trang. Chi quan sát màn hình.	Ngay khi có payload từ Supabase WebSockets hiển thị đơn hàng của người dùng với trạng thái PENDING	Pass

4.2.3. Kết quả kiểm thử

Sau quá trình thực thi các kịch bản kiểm thử đã xây dựng trên cả môi trường giả lập (Emulator/Simulator) và thiết bị di động thực tế, hệ thống ghi nhận đối với các chức năng cốt lõi. Kết quả này cho thấy hệ thống hoạt động ổn định và đáp ứng đúng các yêu cầu chức năng đã đề ra. Dưới đây là các đánh giá chi tiết theo từng tiêu chí:

♦ Tính chính xác và toàn vẹn dữ liệu:

- Xử lý ngoại lệ (Exception Handling): Hệ thống thể hiện khả năng xử lý tốt các trường hợp ngoại lệ. Cụ thể, trong các test case như TC-01 và TC-03, khi người dùng nhập sai mã OTP hoặc thực hiện thao tác không hợp lệ (thêm món từ nhiều nhà hàng khác nhau vào cùng một giỏ hàng), hệ thống backend đã phát hiện và từ chối yêu cầu một cách chính xác. Đồng thời, phía client hiển thị thông báo lỗi rõ ràng cho người dùng thông qua các thành phần giao diện (ví dụ: thông báo dạng snackbar/flushbar), đảm bảo ứng dụng không bị gián đoạn hoặc xảy ra lỗi nghiêm trọng như crash hay treo ứng dụng.

- Cơ chế phân quyền (Authorization): Hệ thống áp dụng cơ chế phân quyền dựa trên vai trò người dùng một cách chặt chẽ. Trong test case TC-02, sau khi đăng

nhập thành công, hệ thống tự động điều hướng người dùng đến đúng phân hệ tương ứng (khách hàng, nhà hàng hoặc tài xế). Các phân hệ khác không thể truy cập trái phép, đảm bảo tính bảo mật và nhất quán trong luồng xử lý.

◆ **Hiệu năng và xử lý thời gian thực:**

- **Tính toán khoảng cách và phí vận chuyển:** Hệ thống sử dụng công thức để tính toán khoảng cách giữa các vị trí địa lý, từ đó xác định phí vận chuyển một cách chính xác. Kết quả tính toán được kiểm tra và đảm bảo tính nhất quán trong toàn bộ quy trình xử lý đơn hàng (TC-07).

- **Cập nhật dữ liệu thời gian thực:** Việc tích hợp cơ chế realtime thông qua Supabase Realtime cho phép hệ thống cập nhật trạng thái đơn hàng gần như tức thời. Trong test case TC-05, khi người dùng đặt đơn hàng, phía nhà hàng nhận được thông báo đơn mới với độ trễ thấp. Đồng thời, các thay đổi trạng thái đơn hàng từ phía tài xế cũng được cập nhật liên tục đến phía khách hàng, đảm bảo tính đồng bộ và nâng cao trải nghiệm người dùng.

4.3.Hạn chế và hướng cải thiện

4.3.1. Các hạn chế hiện tại của hệ thống

- **Thiếu hụt cổng thanh toán và Ví điện tử (Payment Limitations):** Hệ thống đang chạy độc bản ở nghiệp vụ "Trả tiền mặt khi nhận hàng" (COD). Thiếu sót Cổng thanh toán (Payment Gateway) làm giảm đi độ tiện ích cốt lõi của thương mại điện tử, gây bất tiện trong việc thanh toán trực tiếp và tiềm ẩn rủi ro bùng đơn (Cancel rate) đối với cả nhà hàng lẫn tài xế.

- **Thuật toán Vận tải (Logistics) còn cứng nhắc:** Ứng dụng chỉ mới gán quan hệ 1 tài xế duyệt 1 đơn hàng dựa trên khoảng cách. Hệ thống chưa xử lý được tình trạng giao thông thực tế và chưa đáp ứng được tính năng "Ghép đơn" (Batch/Group Order) cho chung một tiến trình vận chuyển để tối ưu hóa nhân lực.

- Chưa đảm bảo quyền riêng tư giao tiếp (In-app Communication): Trong thực tế, khách và tài xế thường phải gọi điện để hẹn lấy hàng, làm lộ thông tin số điện thoại cá nhân. Ứng dụng hiện tại chưa tích hợp hộp thoại trò chuyện trực tiếp trên App.
- Hạn chế Khai phá dữ liệu (Data Analytics/AI Lack): Người dùng khách hàng vẫn phải thực hiện tìm kiếm thủ công thủ công. Chưa có thuật toán đề xuất món ăn thông minh khai thác từ dữ liệu lịch sử mua hàng hay khung giờ đặt hàng.
- Về chịu tải máy chủ (Scalability Limit): Do môi trường triển khai đang chạy trên Local Server và gói giới hạn, hệ thống chưa từng vượt qua các cuộc kiểm tra khi số truy vấn đồng thời trong khung giờ vàng đặt thức ăn.

4.3.2. Định hướng phát triển và mở rộng

4.3.2.1. Số hóa luồng vận hành dòng tiền

♦ Về phía ứng dụng khách hàng:

- Tại bước Checkout (Thanh toán), ứng dụng Customer Call API tạo đơn rỗng trên Back-end với trạng thái AWAITING_PAYMENT (Chờ tiền).
- Back-end khởi tạo chuỗi chữ ký bảo mật (Signature/Checksum) và đẩy mã giao dịch về App. App điều hướng tạm thời (Redirect) hoặc mở Modal liên kết sang App MoMo/VNPay để khách hàng quét mã hoặc nhập thẻ.
- Khi Khách hàng trừ tiền thành công, Cổng thanh toán bên thứ 3 lập tức bắn một gói tin hiệu ngấm (Webhook/IPN) về máy chủ Node.js của chúng ta. Back-end xác thực Webhook, và kích hoạt trạng thái đơn thành PENDING. Lúc này, tiếng chuông thông báo có đơn mới vang lên ở thiết bị của Nhà hàng.

♦ Về phía Hệ thống chi trả cho Nhà hàng:

- Trong quá trình thanh toán điện tử, dòng tiền của Khách hàng không được chuyển trực tiếp vào tài khoản ngân hàng của Chủ nhà hàng. Thay vào đó, máy chủ của ứng dụng gốc sẽ đóng vai trò Dịch vụ ký quỹ (Escrow).

- Hệ thống thiết kế thêm một khái niệm là "Ví điện tử Nhà hàng". Khi Shipper xác nhận giao hàng đến tay khách (Status = COMPLETED), máy chủ kích hoạt hàm Payout(). Hàm này tự động trích lập tỷ lệ chiết khấu của nền tảng (Ví dụ: thu 20% phí duy trì App), sau đó cộng 80% doanh thu còn lại của đơn hàng đó vào Restaurant_Wallet.

- Định kỳ (cuối ngày hoặc cuối tuần), Nhà hàng thực hiện lệnh "Rút tiền". Back-end gọi API payout của Ngân hàng để giải ngân từ tài khoản tổng của App sang tài khoản ngân hàng cá nhân của Chủ quán.

♦ Về phía Thu nhập của Shipper:

- Thay vì thu tiền mặt vắt và từ người nhận và trả tiền mặt tại quán, Shipper chỉ cần thực hiện đúng chức năng giao hàng.

- Hệ thống khởi tạo bổ sung "Ví tài xế". Khi kiện hàng giao thành công, phí giao hàng tự động được hệ thống Back-end đổ thẳng vào Ví tài xế. Shipper có thể dùng số dư trong ví này để Rút tiền công về thẻ ATM.

4.3.2.2. Nâng cấp thuật toán vận tải và tối ưu giao hàng

Để cải thiện hiệu quả vận hành và giảm thời gian giao hàng, hệ thống cần được nâng cấp thuật toán phân phối đơn hàng .

Trước hết, thay vì chỉ lựa chọn tài xế dựa trên khoảng cách gần nhất, hệ thống có thể tích hợp thêm các yếu tố như:

- Tình trạng giao thông theo thời gian thực
- Số lượng đơn hàng đang xử lý của tài xế
- Thời gian dự kiến hoàn thành đơn hàng

Bên cạnh đó, cần triển khai cơ chế ghép đơn (Batch/Group Order) cho phép một tài xế nhận và giao nhiều đơn hàng trong cùng một lộ trình nếu các điểm giao có vị trí gần nhau. Điều này giúp tối ưu hóa tài nguyên, giảm chi phí vận hành và tăng thu nhập cho tài xế.

Ngoài ra, việc tích hợp các dịch vụ bản đồ như Google Maps API hoặc Mapbox API sẽ hỗ trợ tính toán tuyến đường tối ưu, từ đó nâng cao độ chính xác và hiệu quả của quá trình giao hàng.

4.3.2.3. Tích hợp hệ thống giao tiếp nội bộ

Để đảm bảo quyền riêng tư và nâng cao trải nghiệm người dùng, hệ thống cần bổ sung chức năng giao tiếp trực tiếp giữa các bên thông qua ứng dụng. Cụ thể, hệ thống có thể triển khai:

- Chat thời gian thực giữa khách hàng và tài xế
- Gửi tin nhắn giữa nhà hàng và khách hàng khi cần xác nhận đơn

Giải pháp này không chỉ hạn chế việc lộ thông tin cá nhân (số điện thoại) mà còn tăng tính chuyên nghiệp và bảo mật cho hệ thống.

4.3.2.4. Ứng dụng phân tích dữ liệu và trí tuệ nhân tạo

Hiện tại, hệ thống chưa khai thác hiệu quả dữ liệu người dùng. Trong tương lai, có thể tích hợp các giải pháp phân tích dữ liệu nhằm nâng cao trải nghiệm và tăng doanh thu. Một số hướng phát triển bao gồm:

- Hệ thống gợi ý món ăn: Dựa trên lịch sử đặt hàng, sở thích và hành vi người dùng để đề xuất món ăn phù hợp
- Phân tích hành vi người dùng: Xác định khung giờ cao điểm, món ăn phổ biến để hỗ trợ nhà hàng tối ưu kinh doanh
- Cá nhân hóa giao diện: Hiển thị nội dung phù hợp với từng người dùng

Các mô hình học máy (Machine Learning) có thể được áp dụng để cải thiện độ chính xác của hệ thống gợi ý theo thời gian.

4.3.2.5. Nâng cao khả năng chịu tải và mở rộng hệ thống

Để đáp ứng nhu cầu sử dụng trong thực tế với số lượng người dùng lớn, hệ thống cần được nâng cấp về hạ tầng và kiến trúc. Các giải pháp đề xuất bao gồm:

- Triển khai trên nền tảng Cloud (AWS, GCP, Azure) thay vì sử dụng Local Server
- Áp dụng Load Balancing để phân phối tải giữa nhiều máy chủ
- Sử dụng Message Queue (RabbitMQ, Kafka) để xử lý các tác vụ bất đồng bộ như gửi thông báo, xử lý đơn hàng
- Tách kiến trúc Microservices nhằm tăng khả năng mở rộng và bảo trì

KẾT LUẬN

Sau quá trình nghiên cứu, phân tích, thiết kế và triển khai, đề tài “Phát triển ứng dụng giao đồ ăn trực tuyến AppFoodDelivery” đã được hoàn thành và đạt được các mục tiêu đề ra ban đầu. Kết quả của đề tài không chỉ thể hiện ở việc xây dựng thành công một hệ thống ứng dụng hoàn chỉnh mà còn góp phần làm rõ quy trình phát triển một sản phẩm phần mềm theo hướng hiện đại, từ giai đoạn khảo sát yêu cầu đến triển khai và kiểm thử.

Về mặt lý thuyết, đề tài đã tổng hợp và hệ thống hóa một cách có hệ thống các kiến thức nền tảng liên quan đến phát triển ứng dụng di động đa nền tảng, kiến trúc phần mềm Client–Server và phương pháp phân tích, thiết kế hướng đối tượng. Những cơ sở này đóng vai trò quan trọng trong việc định hướng thiết kế và xây dựng hệ thống một cách khoa học và hợp lý. Bên cạnh đó, đề tài đã nghiên cứu và lựa chọn các công nghệ phù hợp với xu hướng phát triển hiện nay như Flutter/Dart cho phát triển giao diện người dùng, Node.js/Express.js cho hệ thống Backend, Prisma ORM cho quản lý và thao tác dữ liệu, cùng với PostgreSQL làm hệ quản trị cơ sở dữ liệu. Việc tích hợp Supabase Realtime và Firebase Authentication không chỉ giúp nâng cao khả năng đồng bộ dữ liệu theo thời gian thực mà còn đảm bảo các yêu cầu về xác thực và bảo mật trong hệ thống.

Về mặt thực tiễn, hệ thống AppFoodDelivery đã được xây dựng với đầy đủ các chức năng cơ bản của một nền tảng giao đồ ăn trực tuyến. Hệ thống bao gồm ba phân hệ chính dành cho Khách hàng, Nhà hàng và Tài xế, cho phép thực hiện các nghiệp vụ như đặt món, quản lý đơn hàng, theo dõi trạng thái giao hàng và quản lý thông tin tài khoản. Kiến trúc hệ thống được tổ chức theo mô hình rõ ràng, đảm bảo tính phân tách giữa các thành phần và thuận lợi cho việc bảo trì, mở rộng trong tương lai. Cơ sở dữ liệu được thiết kế với 13 bảng có mối quan hệ chặt chẽ, đáp ứng yêu cầu lưu trữ và xử lý dữ liệu một cách nhất quán và hiệu quả. Quá trình kiểm thử được thực hiện trên các chức năng

chính cho thấy hệ thống hoạt động ổn định, đáp ứng tốt các yêu cầu chức năng đã đặt ra.

Tuy nhiên, bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế cần được tiếp tục cải thiện. Cụ thể, hệ thống chưa tích hợp các công thanh toán trực tuyến, điều này làm giảm tính tiện lợi trong quá trình sử dụng thực tế. Thuật toán điều phối đơn hàng và tài xế hiện tại còn ở mức cơ bản, chưa tối ưu về mặt thời gian và chi phí vận chuyển. Ngoài ra, hệ thống chưa được kiểm thử tải trong môi trường thực tế với số lượng lớn người dùng đồng thời, do đó chưa đánh giá được đầy đủ khả năng chịu tải và mở rộng. Những hạn chế này là cơ sở quan trọng để định hướng cho các nghiên cứu và phát triển tiếp theo.

Thông qua quá trình thực hiện đề tài, tác giả đã tích lũy được nhiều kiến thức chuyên môn cũng như kinh nghiệm thực tiễn trong việc xây dựng ứng dụng di động, thiết kế và phát triển hệ thống Backend, cũng như quản lý cơ sở dữ liệu. Đồng thời, đề tài cũng giúp rèn luyện tư duy phân tích, thiết kế hệ thống và giải quyết các vấn đề phát sinh trong quá trình triển khai thực tế. Những kết quả đạt được không chỉ đáp ứng yêu cầu của đề tài tốt nghiệp mà còn tạo nền tảng vững chắc để tiếp tục nghiên cứu, hoàn thiện và mở rộng hệ thống trong tương lai, hướng tới việc xây dựng một ứng dụng có tính ứng dụng cao và phù hợp với nhu cầu thực tiễn.

TÀI LIỆU THAM KHẢO

Tiếng Việt

- [1] Nguyễn Thị Thanh Huyền, Ngô Thị Bích Thủy, Phạm Thị Kim Phương (2011), *Giáo trình phân tích thiết kế hệ thống*, NXB Giáo dục Việt Nam.
- [2] Vũ Thị Dương, Phùng Đức Hòa, Nguyễn Thị Hương Lan (2015), *Giáo trình phân tích thiết kế hướng đối tượng*, NXB Khoa học và Kỹ thuật.
- [3] Nguyễn Văn Hiếu (2022), *Lập trình Flutter cơ bản và nâng cao*, NXB Hồng Đức.
- [4] Trần Minh Tuấn (2021), *Phát triển ứng dụng di động đa nền tảng với Flutter và Firebase*, Luận văn tốt nghiệp, Trường Đại học Bách Khoa Hà Nội.
- [5] Lê Văn Bình (2020), *Xây dựng ứng dụng quản lý chi tiêu cá nhân trên Android*, Đồ án tốt nghiệp, Trường Đại học Công nghệ Thông tin TP.HCM.
- [6] Bộ Giáo dục và Đào tạo (2020), *Giáo trình Cơ sở dữ liệu NoSQL*, NXB Giáo dục Việt Nam.
- [7] Trường Đại học Công nghiệp Hà Nội – Khoa Công nghệ Thông tin (2023), *Hướng dẫn viết khóa luận tốt nghiệp ngành Công nghệ Thông tin*, Tài liệu nội bộ.

Tiếng Anh

- [8] Google Developers (2024), *Flutter Documentation*. Truy cập tại: <https://docs.flutter.dev> (truy cập ngày 20/05/2025).
- [9] Google Firebase (2024), *Firebase Documentation*. Truy cập tại: <https://firebase.google.com/docs> (truy cập ngày 20/05/2025).
- [10] Priyanka Tyagi (2022), *Mastering Flutter: Build Modern Mobile Apps with Google's UI Toolkit*, Apress.
- [11] Momentum Works (2026), *Food Delivery Platforms in Southeast Asia 6.0*.

[12] Hanoi Times (2026), “Vietnam’s online food delivery market jumps 19% in 2025 as duopoly tightens grip”, ngày 29/01/2026.

[13] The Investor (2026), “Vietnam’s online food delivery market tops \$2.1 bln as foreign apps dominate”, ngày 29/01/2026.

[14] VnExpress (2026), “Người Việt chi 2,1 tỷ USD đặt đồ ăn qua ứng dụng năm 2025”, ngày 28/01/2026.

Tài liệu trực tuyến

[15] Flutter (n.d.), <https://flutter.dev/docs>

[16] Dart (n.d.), <https://dart.dev/guides>

[17] Pub.dev (n.d.), <https://pub.dev>

[18] Node.js (n.d.), <https://nodejs.org/en/docs>

[19] Express.js (n.d.), <https://expressjs.com>

[20] Prisma (n.d.), <https://www.prisma.io/docs>

[21] Supabase Database (n.d.), <https://supabase.com/docs/guides/database>

[22] PostgreSQL (n.d.), <https://www.postgresql.org/docs/>

[23] Supabase Realtime (n.d.), <https://supabase.com/docs/guides/realtime>

[24] Flutter Awesome (n.d.), <https://flutterawesome.com>

[25] Flutter Map Package (n.d.), https://pub.dev/packages/flutter_map

[26] Firebase Authentication (n.d.), <https://firebase.google.com/docs/auth>

TIẾN ĐỘ THỰC HIỆN

Tuần	Thời gian	Nội dung	Ghi chú
1	02/03 - 08/03	Tìm hiểu và khảo sát nghiệp vụ của hệ thống giao hàng trực tuyến, nghiên cứu các ứng dụng giao đồ ăn phổ biến trên thị trường như GrabFood, ShopeeFood để xác định các chức năng chính của hệ thống. Tiến hành phân tích yêu cầu hệ thống, xác định các tác nhân tham gia như khách hàng, tài xế giao hàng và quản trị viên. Hoàn thiện phần cơ sở lý thuyết liên quan đến mô hình Client–Server, kiến trúc ứng dụng di động và xây dựng đặc tả Use Case cho các chức năng chính của hệ thống.	
2	09/03 - 15/03	Thực hiện phân tích và thiết kế hệ thống dựa trên các yêu cầu đã xác định. Tiến hành xây dựng các biểu đồ UML như Use Case Diagram, Activity Diagram và Sequence Diagram để mô tả luồng hoạt động của hệ thống. Thiết kế cơ sở dữ liệu bằng mô hình ERD, xác định các bảng dữ liệu như người dùng, đơn hàng, sản phẩm, cửa hàng và đánh giá. Đồng thời phác thảo giao diện người dùng (UI/UX) cho ứng dụng nhằm hình dung cấu trúc và trải nghiệm sử dụng của hệ thống.	
3	16/03 - 22/03	Khởi tạo hệ thống và xây dựng cấu trúc tổng thể của hệ thống. Thiết lập môi trường phát triển, cấu hình hệ thống Back-end và Front-end. Xây	

		dựng kiến trúc hệ thống theo hướng tách biệt các tầng như Presentation, Business Logic và Data Access. Thiết lập cơ sở dữ liệu ban đầu và cấu hình kết nối giữa ứng dụng và cơ sở dữ liệu. Đồng thời xây dựng khung giao diện cơ bản cho ứng dụng di động bằng Flutter.	
4	23/03 - 29/03	Phát triển chức năng xác thực người dùng bao gồm đăng ký, đăng nhập và quản lý thông tin tài khoản. Xây dựng API phục vụ việc xác thực và phân quyền người dùng. Triển khai các màn hình giao diện cơ bản trên ứng dụng như màn hình đăng nhập, đăng ký và trang chủ. Thiết lập kết nối giữa Front-end và Back-end để thực hiện các thao tác xác thực và hiển thị dữ liệu ban đầu.	
5	30/03 - 05/04	Xây dựng các chức năng chính của hệ thống giao hàng như hiển thị danh sách cửa hàng, danh sách món ăn và chi tiết sản phẩm. Phát triển chức năng đặt hàng, thêm sản phẩm vào giỏ hàng và gửi yêu cầu đặt hàng đến hệ thống. Đồng thời triển khai chức năng quản lý đơn hàng cho người dùng và xây dựng giao diện hiển thị thông tin đơn hàng.	
6	06/04 - 12/04	Phát triển chức năng quản lý đơn hàng dành cho tài xế giao hàng bao gồm nhận đơn, cập nhật trạng thái giao hàng và xác nhận hoàn thành đơn hàng. Xây dựng trang quản trị dành cho quản trị viên để quản lý người dùng, cửa hàng và đơn	

		hàng. Đồng thời hoàn thiện việc đồng bộ dữ liệu giữa ứng dụng di động và hệ thống Back-end.	
7	13/04 - 19/04	Thực hiện kiểm thử hệ thống nhằm phát hiện và khắc phục các lỗi phát sinh trong quá trình sử dụng. Kiểm tra các chức năng chính như đăng nhập, đặt hàng, cập nhật trạng thái đơn hàng và hiển thị dữ liệu. Tối ưu hiệu năng xử lý của hệ thống, cải thiện trải nghiệm người dùng và bắt đầu viết nội dung báo cáo cho chương cài đặt và triển khai hệ thống.	
8	20/04 - 26/04	Rà soát và hoàn thiện toàn bộ mã nguồn của hệ thống. Kiểm tra lại các chức năng chính để đảm bảo hệ thống hoạt động ổn định. Hoàn thiện nội dung báo cáo đồ án, chỉnh sửa định dạng theo đúng quy định và chuẩn bị slide thuyết trình. Chuẩn bị dữ liệu mẫu và kịch bản demo để trình bày trong buổi bảo vệ đồ án.	